결과리포트

LAB3_FPGA 실험

제출일: 2026년 10월 11일

이준화 교수님

전자전기컴퓨터공학부

팀 B

학번: 2025440031

이름: 김영서

목차

<예비보고서>

1. 실험 개요와 목적

2. 실험의 이론적 배경

3. 실험에 필요한 장치

4. 모드별 입출력 표

5. 각 verilog 파일의 역할

6. 사전 시뮬레이션 결과

<결과보고서>

7. 포트 및 핀 번호 입력

8. Vivado 내의 실험 과정 및 결과

9. 실험 결과와 예상 결과 간의 비교

10. 증빙 및 소스 commit 링크(git hub)

1. 실험 개요와 목적

1.1. 7개의 회로 즉, PWM LED 밝기 제어, RGB LED 색상 제어, 피에조 구동, 스텝모터 제어, MM 시계, 문자 LCD 제어 및 UART 에코 회로를 설계하고 그 과정 안에서 FPGA를 이용한 다양한 입출력 장치의 제어 원리를 이해한다. 특히 50 MHz 단일 클록을 기준으로 하여 카운터와 클록 이네이블을 활용하여 동작 주기를 생성하고, 또한 듀티비, 출력 주파수 및 상태 변화가 각각의 회로의 동작에 미치는 영향을 확인한다.

1.2. 각각의 응용 회로들을 Verilog HDL을 통해 구현하고, 테스트 벤치를 이용하여 입력 변화에 따른 출력과 내부 상태를 검증한다. VS Code에서 Icarus 시뮬레이션을 수행하여 예상값과 실제 출력 파형을 비교하고, 파라미터나 RTL의 일부를 수정했을 때 나타나는 변화와 복구 결과를 분석하여 디버깅 과정을 거친다. 이를 통해 정상 동작과 경계 조건을 확인하고, 회로의 기능을 체계적으로 검증하는 방법을 익히고 원리를 이해한다.

1.3. 사전 시뮬레이션 단계에서 검증했던 RTL과 테스트벤치에서의 결과를 Vivado에서의 결과와 교차 확인하고, 핀 및 클록 제약을 적용하여 합성, 구현, 타이밍 분석과 비트스트림 생성을 vivado 프로그램 내에서 수행한다. 이후 앞선 단계가 모두 진행된 FPGA 보드에서 각각의 회로를 실험하여 LED 밝기와 색상, 피에조 음 출력, 모터 회전, 시계 및 문자 표시, UART 송수신 동작을 관찰하고 기록한다. 실험 과정을 기록한 내용들을 통해서 시뮬레이션 결과와 실제 하드웨어 동작을 비교하고, 이후 핀 제약이 알맞게 이루어졌는지 확인한다. 외에도 각각의 회로를 실제 보드에 구현하면서 기존에 알고 있던 지식이 맞는지 확인하는 과정을 가지면서 점검하는 시간을 갖는다.

2. 실험의 이론적 배경

2.1. 단일 클록과 Clock Enable

▶ FPGA에서의 순차 논리회로는 클록 에지를 기준으로 내부 상태와 출력값을 갱신하면서 진행된다. 이번 실험에서는 50MHz 클록을 공통으로 사용할 것이며, 클록의 주기는 20ns로 사용하여 진행할 예정이다. LED의 밝기 제어, 모터 회전, 시계 계수처럼 서로 다른 동작 속도가 필요한 경우, 카운터를 이용하여 각각 다른 주기를 생성하여 다른 동작 속도를 할당해 줄 수 있다. 하지만, 클록 이네이블 방식을 이용하여 다른 주기를 생성할 예정이다.

▶ Clock enable 방식은 기존 클록을 유지하면서 진행되지만, 중간에 특정 조건을 만족하는 상승 에지에서만 상태를 변경하는 방식이다. 예를 들어 이를 이용하면 50MHz 클록에서 5천만 번째의 클록마다 한 번 상태를 갱신하면 1초 간격의 동작을 구현할 수 있게 된다. 이번 실험에서는 카운터 출력으로 별도의 클록을 만드는 대신, 동일한 클록 안에서 상태 갱신 조건을 제어하여 회로를 구성한다.

2.2. 비동기 입력의 동기화와 디바운스

▶ 버튼과 UART의 수신 신호는 FPGA의 내부 클록과 상관없이 다른 시점에 변화하는 비동기 입력이다. 이러한 입력이 클록 에지 부근에서 변하게 되면 플립플롭의 출력이 일시적으로 불안정해지는 메타 안정성이 발생할 수 있다. 이것을 줄이기 위해서 입력을 두 단계의 플립플롭에 순차적으로 입력하여 통과시켜 내부 클록에 동기화한다.

▶ 기계식 버튼은 한 번 눌러도 짧은 시간 동안 접점이 반복적으로 붙고 떨어지는 바운스 현상이 발생한다. 디바운스 회로는 입력이 일정 시간 동안 동일한 상태로 유지되는지 확인하고 이를 유효한 입력으로 받아들인다. 이번 실험에서의 LED PWM 회로에서는 50MHz 기준 1백만 클록, 즉 20ms를 안정 시간으로 사용할 것이다

.

▶ 이후 버튼의 유효한 클릭을 한 클록 길이의 펄스로 변환하면, 버튼을 길게 누르더라도 밝기 단계가 계속 증가하는 것을 방지할 수 있다. 이에 따라 동기화, 디바운스, 단일 펄스 생성은 각각 입력의 안정적인 수용, 접점 잡음 제거, 동작 횟수 제한과 같은 역할을 수행한다.

2.3. PWM과 LED 밝기 제어

▶ PWM(Pulse Width Modulation)은 일정한 주기 안에서 신호가 HIGH로 유지되는 시간을 조절하는 방식이다. 여기서 듀티비는 한 주기에서 HIGH 구간이 차지하는 비율이며, 다음과 같은 식으로 나타낼 수 있다.

$$\text{듀티비}(\%) = \frac{HIGH\text{상태 유지 시간}}{100*(\text{전체 주기})}$$

▶ LED를 빠르게 켜고 끄면 결과적으로 개별 점멸보다 평균적인 밝기 변화로 인식된다. LED가 HIGH 상태에서 켜지는 경우 듀티비가 증가할수록 켜져 있는 시간이 길어지므로 일반적으로 밝게 보인다. 다만 듀티비는 사람이 봤을 때 느껴지는 밝기에 정확히 비례하는 것은 아니다.

▶ 본 실험에서는 50MHz 클록으로 1kHz PWM 신호를 생성하므로 한 주기는 5만 클록, 즉, 1ms이다. 카운터 값과 듀티비 각각에 해당하는 기준값을 비교하여 PWM 출력을 결정한다. 예를 들어 듀티비가 30%이면 한 주기 중 만 오천 클록 동안 HIGH가 된다. 실험에서 버튼을 누를 때마다 듀티비를 10%씩 증가시키고, 100% 이후에 한 번 누르면 다시 0%로 순환하도록 구성한다.

2.4. RGB LED와 색 혼합

▶ RGB LED는 빨강(R), 초록(G), 파랑(B)의 빛을 조합하여 다양한 색을 표현하는 장치이다. 빛의 삼원색을 통해 여러 색을 구현하는 방식으로 이해하면 된다. 각 색의 밝기를 조절하면 세 빛의 상대적인 세기가 달라지면서 혼합된 색상이 변한다.

▶ 본 실험에서는 앞서 설명한 세 개의 PWM 채널을 사용하여 R, G, B의 듀티비를 각각 제어할 예정이다. 각각 채널은 같은 PWM 주기를 사용하지만 HIGH 구간의 길이는 독립적으로 설정할 것이기 때문에 원하는 색의 버튼을 눌렀을 때 해당 채널의 듀티비만 변경되고 다른 채널의 설정은 유지되어야 한다.

▶ 시뮬레이션에서는 각 채널의 HIGH 클록 수와 채널 간 독립성을 확인한다. 실제 보드에서는 같은 듀티비를 적용하더라도 LED별 발광 특성과 사람의 시각적 민감도에 따라 각 색의 밝기가 다르게 느껴질 수 있으므로, 정확한 수치를 통해 독립적으로 실행되는지 확인하는 과정을 거쳐야 한다.

2.5. 피에조 구동과 주파수 분주

▶ 피에조 소자는 소리를 발생시키는 소자로서 전압 변화에 따라 기계적으로 변형되는 성질을 이용하여 소리를 발생시킨다.

▶ 주기적인 전기 신호를 인가하면 소자가 진동하며, 입력하는 신호의 주파수는 발생하는 소리의 음높이에 영향을 주게 된다.

▶ 본 실험에서는 50MHz 클록을 이용하여 약 294Hz의 사각파를 생성한다. 사각파 한 주기에는 출력 반전이 두 번 필요하기 때문에, 절반의 주기에 해당하는 카운터 값은 아래와 같이 계산한다.

▶ 반주기 클록 수 = 입력 클록 주파수 / (2 × 목표 출력 주파수)

▶ 카운터가 설정한 절반의 주기에 도달할 때마다 출력을 반전하면 HIGH와 LOW 구간이 반복되게 만들 수 있다. 정수 나눗셈으로 반주기 클록 수를 결정하므로 정확한 값으로 나누지 못해 실제 출력 주파수에는 이론에 따른 목표값과 작은 차이가 생길 수 있다.

▶ 시뮬레이션에서는 연속된 출력 반전 사이의 클록 수를 확인하여 분주 동작을 수치적으로 검증한다.

2.6. 스텝모터의 위상 제어

▶ 스텝모터는 회전자를 일정한 각도씩 이동시키는 모터로서 코일에 전류를 공급하는 순서를 바꾸는 방식을 이용하여 이동시킨다. 코일 구동 패턴을 순차적으로 변경하면 정방향으로 회전하고, 반대로 순서를 변경하면 역방향으로 회전하게 된다.

▶ 본 실험에서는 0011 → 0110 → 1100 → 1001의 네 가지 구동 패턴을 사용한다. 위에서 설명했듯이 방향 입력에 따라 상태의 진행 순서를 변경하게 되고, enable 입력을 통해 상태 이동 여부를 제어한다. enable이 0이면 이론적으로 현재 상태와 코일 출력 패턴을 유지해야 하므로, 정지 동작과 코일 전원 차단은 서로 구분해야 한다.

▶ 기본 단계 전환 주파수는 100Hz이며, 50MHz 클록에서 오십만 클록마다 다음 패턴으로 이동한다. 즉, 결과적으로 각 단계 사이의 시간 간격은 10ms가 된다. 실제 회전 속도는 이 전환 주파수뿐 아니라 모터의 스텝각과 감속비에 따라 달라진다. 모터 구동에 필요한 전류는 FPGA 출력 핀으로 직접 공급하지 않고 드라이버 회로를 통해 공급한다.

2.7. BCD 카운터와 MM 시계

▶ BCD(Binary-Coded Decimal)는 십진수 한 자리를 4비트로 할당하여 이진수로 표현하는 방식이다.

▶ MM 시계는 분의 십의 자리와 일의 자리, 초의 십의 자리와 일의 자리를 각각 관리하여 각각 시간을 표시한다.

▶ 초의 일의 자리는 0부터 9까지 증가하고, 9 다음에는 0으로 돌아가면서 그 과정마다 초의 십의 자리에 자리 올림하게 된다. 초의 십의 자리는 0부터 5까지 계수하므로 00:59 다음에는 01:00이 된다. 같은 방식으로 분을 계수하여 59:59 다음에는 00:00으로 순환한다

.

▶ 이번 실험에서는 50MHz 클록을 계수하여 1초마다 시간 값을 갱신할 예정이다. 검증 시에는 일반적인 증가 동작뿐 아니라 00:09 → 00:10, 00:59 → 01:00, 59:59 → 00:00과 같이 특별하게 자리올림이 이루어지는 상황 및 전체 순환 조건을 확인해야 한다.

2.8. 7-Segment 디코딩과 동적 스캔

▶ 7-segment는 일곱 개의 발광 구간을 조합하여 숫자를 표시하는 장치이다. 디코더는 입력으로 받은 숫자를 각 세그먼트의 점등 패턴으로 변환한다. 실제 출력 패턴은 세그먼트의 연결 순서와 점등 극성에 맞게 설정해야 한다.

▶ 여러 자리의 표시장치가 세그먼트 신호를 공유하며 이용하는 경우에는 각 자리를 빠르게 번갈아 활성화하는 동적 스캔 방식을 사용한다. 한 순간에는 한 자리만 표시하지만, 충분히 빠르게 반복하면 사람이 느끼기에는 여러 자리가 동시에 표시되는 것처럼 느껴진다.

▶ 이번 실험에서는 네 개의 자리에 분과 초의 각각의 숫자를 순차적으로 출력한다. 자리 선택 신호와 해당 자리의 숫자 패턴이 일치해야하고, 숫자 계수를 위한 1초 간격과 표시를 위한 빠른 스캔 간격은 서로 독립적으로 관리한다.

2.9. 문자 LCD의 명령과 데이터 전송

▶ 문자 LCD는 제어 명령과 문자 데이터를 입력받아서 화면에 입력받은 문자를 표시하는 장치이다. 본 실험에서는 HD44780모델 호환 LCD를 8비트 쓰기 전용 방식으로 제어할 예정이다. RS 신호는 명령과 문자 데이터를 구분하며, 쓰기 동작에서는 R/W를 LOW로 유지한다.

▶ LCD를 사용하기 위해서 전원 인가 후 대기 시간을 확보하고, 기능 설정, 화면 표시 설정, 문자 입력 방향 설정, 화면 지우기 등의 초기화 명령을 순서대로 전달해야 한다. 초기화가 끝난 뒤에는 표시할 위치의 DDRAM 주소를 지정하고 문자 코드를 전송하는 과정을 거친다.

▶ Enable 신호를 이용해 데이터를 전달할 때에는 RS와 데이터가 미리 안정되어 있어야 하며, Enable의 HIGH 구간과 하강 에지 이후에도 필요한 유지 시간을 확보해야 한다. 외에도 명령 처리에도 시간이 필요하기 때문에 명령 사이에 대기 구간을 둘 것이다. 이번 실험에서는 LCD의 Busy Flag를 읽지 않는 쓰기 전용 방식이기에, 화면 지우기처럼 처리 시간이 긴 명령에는 별도의 충분한 대기 시간을 적용한다.

2.10. UART 직렬통신과 에코 동작

▶ UART(Universal Asynchronous Receiver/Transmitter)는 송신 측과 수신 측이 별도의 클록선을 공유하지 않은 상태에서 데이터를 직렬로 주고받는 통신 방식이다. 양쪽은 통신 속도와 데이터 형식을 동일하게 설정해야 하며, 수신기는 시작 비트를 기준으로 비트의 수신 시점을 결정한다.

▶ 본 실험에서는 9600bps, 8N1형식을 사용할 예정이다. 8N1은 데이터 8비트, 패리티 없음, 정지 비트 1개를 의미한다. 전송선은 대기 상태에서 HIGH를 유지하며, LOW인 시작 비트 이후 데이터가 가장 하위 비트부터 전송되고 마지막에 HIGH인 정지 비트가 이어진다.

▶ 9600bps에서 한 비트의 시간은 약 104.17μs이다. 50MHz 클록을 기준으로 이를 구현하기 위해서는 한 비트당 대략 5208클록이 필요하다. 수신 회로는 입력을 동기화하고 시작 비트를 확인하고, 이후 각 데이터 비트의 중앙 부근에서 값을 읽는다. 정지 비트가 올바르지 않으면 프레이밍 오류로 판단한다.

▶ 에코 회로는 정상적으로 수신한 바이트를 다시 송신하여 PC로 돌려보내는 회로이다. 이번 실험에서는 마지막으로 전달받은 수신값을 LED에도 표시하며, 송신기의 준비 상태인 ready와 전송 요청인 valid를 이용하여 데이터 전달 시점을 제어한다.

3. 실험에 필요한 장치

3.1. Visual Studio Code

- 본 실험에서 Verilog HDL 파일과 SystemVerilog Testbench를 작성하는 주된 편집 환경이다.
- src 폴더의 각 조합논리회로, 통합 모듈, 버튼 및 LCD 제어 모듈을 작성하고, sim 폴더의 Testbench와 FPGA 핀 설정을 위한 XDC 파일도 관리한다.
- 즉, 회로를 HDL이라는 소프트웨어 형태로 기술하는 첫 단계에 사용된다.

3.2. Icarus Verilog

- HDL로 작성한 회로를 실제 FPGA에 구현하기 전에 기능이 올바른지 확인하는 시뮬레이터이다.
- 이번 실험에서는 10개 mode와 각각의 256개 DIP 입력 조합을 검사하여 총 2560개의 경우를 자동으로 비교한다.
- 또한 reset, button bounce, long press, MODE 10에서 MODE 01로의 순환 및 LCD 동작도 Testbench에서 함께 검증한다.

3.3. VaporView

- Icarus Verilog 시뮬레이션에서 생성된 VCD 파일을 파형으로 표시하는 도구이다.
- 본 실험에서는 여러 신호를 관찰하여 버튼 입력 이후 모드가 바뀌는 과정과 LCD 데이터가 전송되는 과정을 시간축에서 확인할 예정이다.

3.4. AMD Vivado 2026.1

- Vivado는 Verilog로 기술한 논리회로를 실제 AMD/Xilinx FPGA에서 사용할 수 있는 회로로 변환하는 FPGA 설계 프로그램이다.
- 사전 시뮬레이션 이후 회로를 합성하고, FPGA 내부 자원에 회로를 배치하고 연결하는 과정을 수행하며, 마지막으로 FPGA에 기록할 비트 스트림을 생성한다.

FPGA와 이러한 소프트웨어 도구의 관계

- FPGA는 ASIC과 달리 반도체가 제조된 이후에도 논리 기능을 다시 설정할 수 있는 소자이다.
- ASIC은 특정 기능에 맞는 회로를 실제 반도체 제조 공정을 통해 고정하여 제작하므로, 설계에 문제가 발생하면 경우에 따라 설계를 수정한 뒤 다시 제작해야 한다.
- 반면, FPGA는 이미 제조된 칩의 programmable logic과 interconnect를 Bitstream으로 설정하기 때문에 새로운 반도체 제조 공정을 반복하지 않고도 회로를 변경할 수 있다.
- 이러한 위험성 측면, 시간적 측면에 따라 반도체 제조 공정에서는 FPGA를 사용한다.

4. 모드별 입출력 표

본 실험은 7개의 회로를 개별 프로젝트로 구현한다.
아래의 모드는 각 회로의 동작 조건을 의미한다. 모든 회로는 공통으로 50 MHz 클록인 clk_50mhz와 HIGH에서 활성화되는 리셋 입력 rst_p를 사용한다.

4.1. PWM LED 밝기 제어

동작 모드	입력 조건	출력 및 동작
초기화	rst_p=1	밝기 단계를 0으로 초기화하고 led[7:0]을 LOW로 출력한다.
밝기 유지	rst_p=0, 유효한 버튼 누름 없음	현재 듀티비의 PWM 신호를 8개 LED에 동일하게 출력한다.
밝기 증가	button의 유효한 누름 발생, 현재 단계 0~9	밝기 단계가 1 증가하며 듀티비가 10%p 증가한다.
최대 밝기	밝기 단계=10	듀티비가 100%가 되어 led[7:0]을 HIGH로 유지한다.
단계 순환	밝기 단계=10에서 버튼 누름	밝기 단계가 0으로 돌아가고 듀티비가 0%가 된다.

버튼 입력은 동기화와 디바운스를 거쳐 처리되며, 버튼을 계속 누르고 있으면 밝기가 반복 증가하지 않는다.

4.2. RGB LED PWM 제어

동작 모드	입력 조건	출력 및 동작
초기화	rst_p=1	R·G·B의 밝기 단계를 모두 0으로 초기화한다.
색상 유지	유효한 버튼 누름 없음	각 채널의 현재 듀티비를 유지한다.
빨강 밝기 변경	button_r의 유효한 누름 발생	led_r[3:0]의 듀티비가 한 단계 증가하고 G·B 설정은 유지된다.
초록 밝기 변경	button_g의 유효한 누름 발생	led_g[3:0]의 듀티비가 한 단계 증가하고 R·B 설정은 유지된다.
파랑 밝기 변경	button_b의 유효한 누름 발생	led_b[3:0]의 듀티비가 한 단계 증가하고 R·G 설정은 유지된다.
채널별 순환	해당 채널이 100%인 상태에서 버튼 누름	해당 채널의 듀티비만 0%로 돌아간다.

4.3. 피에조 단일 음 출력

동작 모드	입력 조건	출력 및 동작
초기화	rst_p=1	내부 카운터와 piezo 출력을 0으로 초기화한다.
반주기 계수	rst_p=0, 카운터가 반주기 종료값에 도달하지 않음	카운터를 증가시키고 piezo의 현재 출력값을 유지한다.
출력 반전	카운터가 HALF_PERIOD−1에 도달	카운터를 0으로 되돌리고 piezo 출력을 반전한다.
연속 음 출력	rst_p=0 상태 유지	출력 반전을 반복하여 약 294 Hz의 사각파를 생성한다.

4.4. 스텝모터 위상 제어

동작 모드	입력 조건	출력 및 동작
초기화	rst_p=1	상태를 0으로 초기화하고 stepmotor[3:0]=0011을 출력한다.
정지 및 상태 유지	동기화된 enable=0	단계 카운터를 0으로 초기화하고 현재 코일 출력 패턴을 유지한다.
정방향 구동	동기화된 enable=1, direction=0	단계 전환 시점마다 0011 → 0110 → 1100 → 1001 → 0011 순서로 출력한다.
역방향 구동	동기화된 enable=1, direction=1	단계 전환 시점마다 위 순서의 역순으로 출력한다.
단계 사이 유지	enable=1, 단계 전환 시점 이전	현재 코일 출력 패턴을 유지한다.

4.5. MM 시계

동작 모드	입력 및 내부 조건	출력 및 동작
초기화	rst_p=1	시간 값을 00:00으로 초기화하고 스캔 위치를 첫 자리로 설정한다.
시간 유지	rst_p=0, 1초 계수 완료 전	시간 값을 유지하면서 네 자리의 표시를 계속 스캔한다.
초 증가	1초 경과, 초의 일의 자리 0~8	초의 일의 자리를 1 증가시킨다.
초 자리올림	1초 경과, 초의 일의 자리=9, 십의 자리 0~4	초의 일의 자리를 0으로 바꾸고 십의 자리를 1 증가시킨다.
분 증가	1초 경과, 현재 초=59, 현재 분<59	초를 00으로 바꾸고 분을 1 증가시킨다.
전체 순환	1초 경과, 현재 시간=59:59	시간 값을 00:00으로 되돌린다.
표시 스캔	스캔 카운터의 주기 완료	seg_com[7:0]으로 표시 자리를 선택하고 seg_data[7:0]으로 해당 숫자 패턴을 출력한다.

4.6. 문자 LCD 제어

동작 모드	입력 및 내부 조건	출력 및 동작
초기화	rst_p=1	lcd_e, lcd_rs, lcd_data를 0으로 초기화한다. lcd_rw는 항상 0이다.
전원 안정 대기	리셋 해제 후 초기 대기 구간	lcd_e=0을 유지하고 LCD의 전원 안정 시간을 확보한다.
명령 전송	초기화 명령 또는 표시 주소 설정 단계	lcd_rs=0, lcd_rw=0으로 설정하고 lcd_data[7:0]에 명령 코드를 출력한다.
문자 전송	문자열 데이터 전송 단계	lcd_rs=1, lcd_rw=0으로 설정하고 lcd_data[7:0]에 문자 코드를 출력한다.
Enable 제어	명령 또는 데이터의 전송 단계	데이터 설정 후 lcd_e를 HIGH로 올렸다가 LOW로 내려 쓰기 동작을 수행한다.
명령 처리 대기	Enable 하강 이후	다음 전송까지 정해진 시간을 기다리며, 화면 지우기 명령에는 더 긴 대기 시간을 적용한다.
두 줄 표시 반복	초기화 및 문자열 전송 완료	첫째 줄에 “FPGA LAB3”, 둘째 줄에 “LCD CONTROLLER”를 표시하도록 주소와 데이터를 반복 전송한다.

4.7. UART 에코

동작 모드	입력 및 내부 조건	출력 및 동작
초기화	rst_p=1	송수신 상태를 초기화하고 uart_txd=1, led[7:0]=0으로 설정한다.
수신 대기	uart_rxd=1, 송수신 동작 없음	시작 비트를 기다린다. uart_txd는 HIGH를 유지하고 LED는 마지막 수신값을 유지한다.
데이터 수신	시작 비트 확인 후 데이터 비트 입력	9600 bps, 8N1 조건으로 데이터 8비트를 최하위 비트부터 수신한다.
정상 수신 및 에코 요청	내부 rx_valid=1, tx_ready=1	수신값을 LED에 표시하고 동일한 바이트의 송신을 요청한다.
에코 송신	송신 요청 수락	uart_txd로 시작 비트, 데이터 8비트, 정지 비트를 순서대로 출력한다.
송신 중 추가 수신	내부 rx_valid=1, tx_ready=0	LED는 새 수신값으로 갱신되지만, 해당 바이트의 에코 요청은 저장되지 않는다.
프레이밍 오류	정지 비트 확인 시 수신값이 LOW	내부 오류 신호를 발생시키며, 해당 프레임을 정상 수신 데이터로 전달하지 않는다.

5. 각 verilog 파일의 역할

파일명	역할
button_onepulse.v	버튼 입력을 2단 동기화하고 디바운스를 수행한다. 유효한 버튼 누름마다 한 클록 길이의 펄스를 생성하여 밝기 단계가 한 번씩 변경되도록 한다.
pwm_channel.v	주기 카운터와 밝기 단계에 따른 기준값을 비교하여 PWM 신호를 생성한다. LED PWM과 RGB PWM에서 각 채널의 듀티비를 제어한다.
lab3_led_pwm.v	LED 밝기 제어의 최상위 모듈이다. 버튼 펄스마다 밝기 단계를 증가시키고, 0~100%로 순환하는 PWM 신호를 8개 LED에 동일하게 출력한다.
tb_led_pwm.sv	LED PWM 회로의 테스트벤치이다. 리셋과 버튼 입력을 인가하고, 0%·30%·100% 및 0% 복귀 조건에서 한 주기의 HIGH 클록 수를 검사한다.
lab3_rgb_pwm.v	RGB LED 제어의 최상위 모듈이다. R·G·B 버튼에 따라 세 채널의 밝기 단계를 독립적으로 변경하고, 각 채널의 PWM을 네 개의 RGB LED에 출력한다.
tb_rgb_pwm.sv	RGB PWM 회로의 테스트벤치이다. R=20%, G=50%, B=80%의 HIGH 구간을 확인하고, R 채널만 변경했을 때 G·B 채널의 설정이 유지되는지 검사한다.
lab3_piezo.v	클록을 계수하여 설정된 반주기마다 출력을 반전한다. 기본 설정에서 피에조 구동을 위한 약 294 Hz의 사각파를 생성한다.
tb_piezo.sv	피에조 회로의 테스트벤치이다. 시간을 단축한 파라미터를 적용하여 연속된 출력 반전 사이의 간격이 예상한 클록 수와 일치하는지 검사한다.
lab3_stepper.v	스텝모터 제어 모듈이다. enable과 direction 입력을 동기화하고, 설정된 단계 간격마다 네 가지 코일 패턴을 정방향 또는 역방향으로 변경한다.
tb_stepper.sv	스텝모터 회로의 테스트벤치이다. 정방향 및 역방향의 코일 패턴 변화와 enable=0에서 현재 패턴이 유지되는지 검사한다.
mmss_counter.v	입력 클록을 계수하여 1초마다 분·초의 BCD 값을 갱신한다. 각 자리의 자리올림을 처리하고 59:59 다음에 00:00으로 순환한다.

sevenseg_decode.v	4비트 숫자 입력을 7-segment 표시 패턴으로 변환하여 각 숫자에 필요한 세그먼트 출력을 생성한다.
lab3_mmss_clock.v	MM 시계의 최상위 모듈이다. 시간 카운터와 세그먼트 디코더를 연결하고, 네 자리를 순차적으로 선택하여 분과 초를 표시한다.
tb_mmss_counter.sv	시간 카운터와 숫자 디코더의 테스트벤치이다. 초·분 자리올림, 59:59에서의 순환, 전체 3,600초 계수 및 숫자별 세그먼트 패턴을 검사한다.
lab3_character_lcd.v	문자 LCD의 초기화와 두 줄 문자열 출력을 제어한다. 명령·문자 데이터, RS 및 Enable 신호를 생성하고 전원 안정 시간과 명령별 대기 시간을 관리한다.
tb_character_lcd.sv	문자 LCD 회로의 테스트벤치이다. Enable 하강 에지에서 초기화 명령과 두 줄 데이터에 해당하는 총 40바이트의 순서, 값 및 RS 신호를 검사한다.
uart.v	UART 수신 모듈 uart_rx와 송신 모듈 uart_tx를 포함한다. 수신 입력 동기화, 시작·정지 비트 확인, 직렬·병렬 데이터 변환 및 송신 비트 타이밍을 처리한다.
lab3_uart_echo.v	UART 에코 회로의 최상위 모듈이다. 수신한 데이터를 LED에 표시하고, 송신기가 준비된 경우 동일한 데이터를 다시 송신하도록 제어한다.
tb_uart_echo.sv	UART 에코 회로의 테스트벤치이다. 독립적인 직렬 송수신 모델로 0x41, 0x5A, 0x0A를 전송하고, 반환 데이터와 LED 표시값을 비교하며 프레이밍 오류 여부를 확인한다.

6. 사전 시뮬레이션 결과

6.1. LED PWM

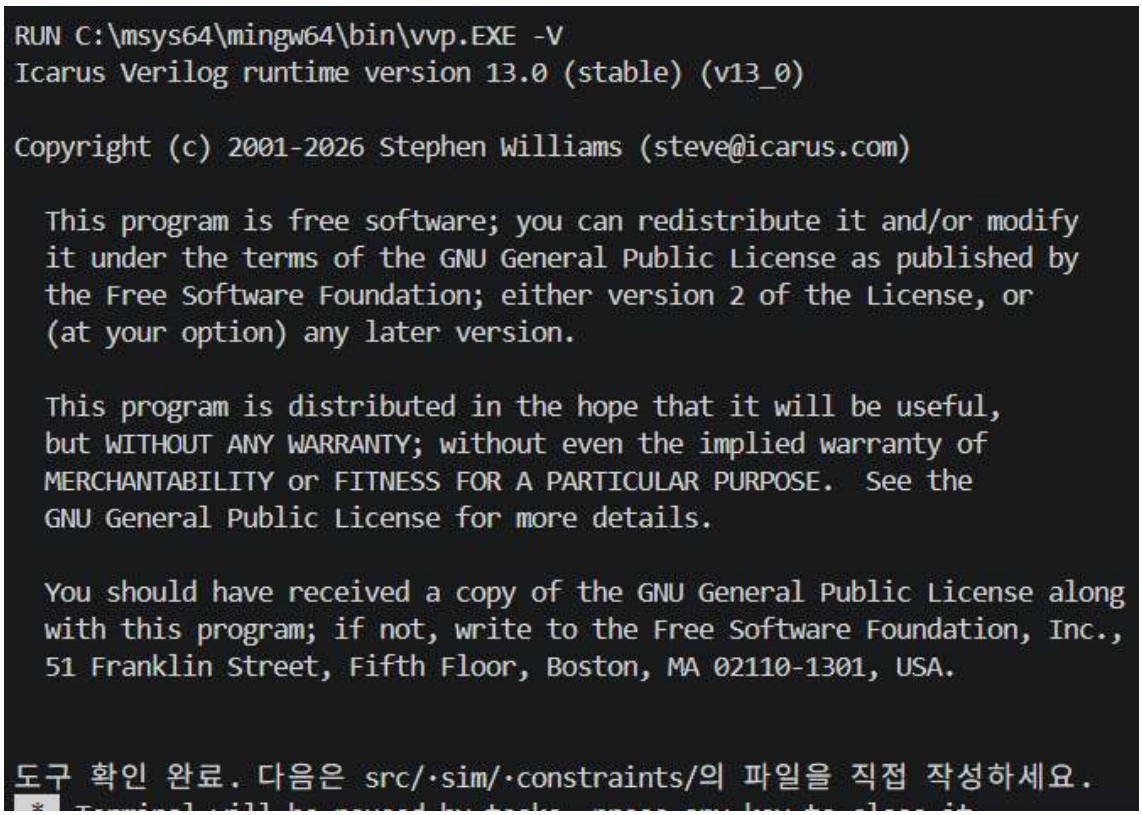
작성한 코드는 다음과 같다.

lab3_led_pwm.v

circuits > 01_led_pwm > src > lab3_led_pwm.v

```
1  `timescale 1ns/1ps
2  module lab3_led_pwm #(
3  parameter integer CLK_HZ = 50_000_000,
4  parameter integer PWM_HZ = 1_000,
5  parameter integer LEVELS = 10,
6  parameter integer DEBOUNCE_CYCLES = 1_000_000
7  ) (
8  input wire clk_50mhz,
9  input wire rst_p,
10 input wire button,
11 output wire [7:0] led
12 );
13 wire press;
14 wire pwm;
15 reg [3:0] level;
16
17 button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button (
18 .clk(clk_50mhz), .rst_p(rst_p), .button(button), .pulse(press)
19 );
20
21 always @(posedge clk_50mhz or posedge rst_p) begin
22 if (rst_p)
23 level <= 4'd0;
24 else if (press)
25 level <= (level == LEVELS) ? 4'd0 : level + 1'b1;
26 end
27
28 pwm_channel #(.PERIOD_CYCLES(CLK_HZ / PWM_HZ), .LEVELS(LEVELS)) u_pwm (
29 .clk(clk_50mhz), .rst_p(rst_p), .level(level), .pwm(pwm)
30 );
31
32 assign led = {8{pwm}};
33 endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.



check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

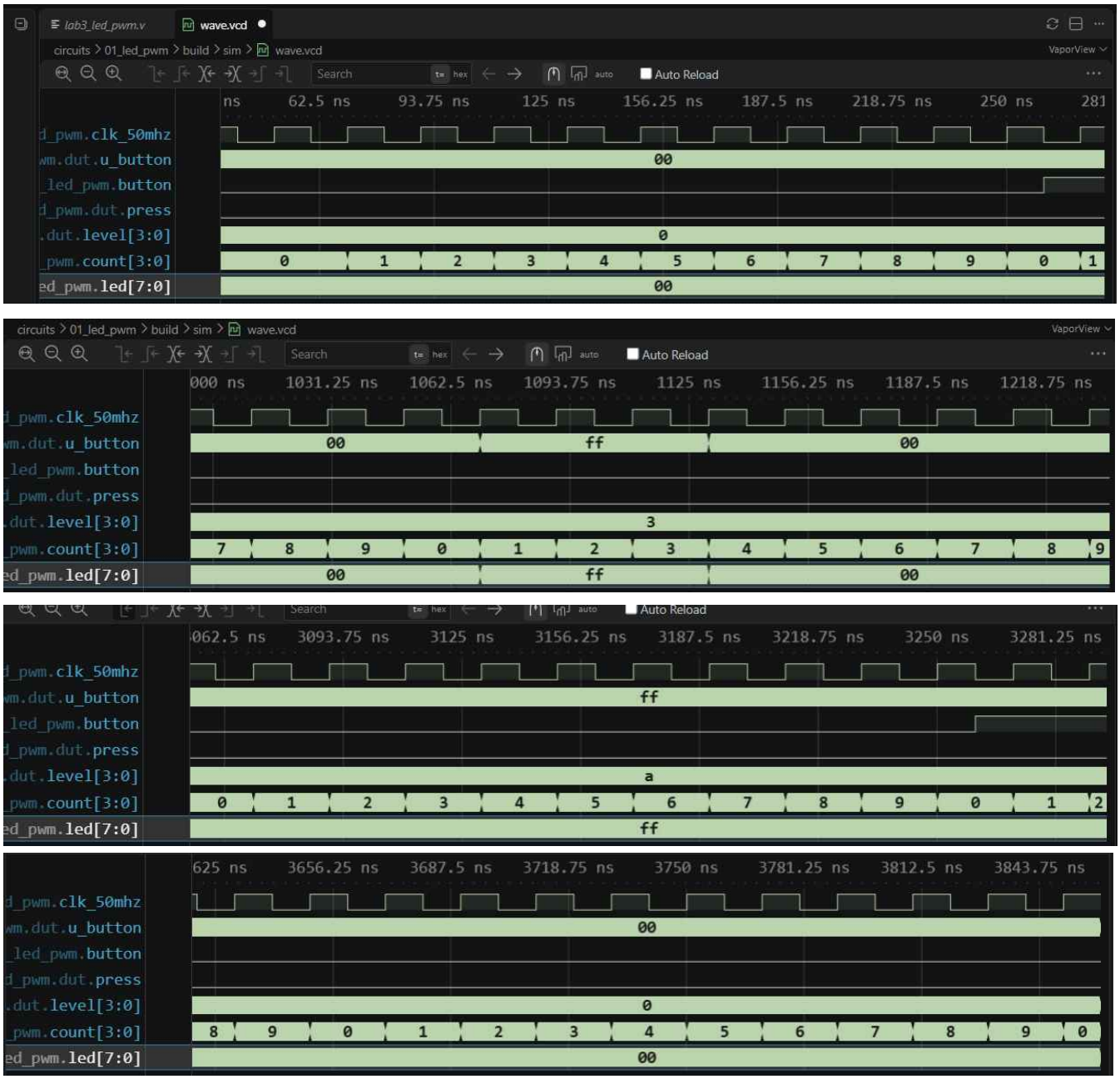


위 터미널 응답에서 알 수 있듯이 4개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.

이후 파형 출력을 통해 코드가 정상 작동하는지 확인하였다.

확인 시간 구간	검사 조건	level 예상값	LED 출력 예상값
71~251 ns	초기 밝기 0%	0	10회 검사 모두 LOW
1071~1251 ns	버튼 3회 입력 후 밝기 30%	3	처음 3회 HIGH, 이후 7회 LOW
3071~3251 ns	버튼 총 10회 입력 후 밝기 100%	10	10회 검사 모두 HIGH
3671~3851 ns	버튼을 한 번 더 누른 후 0% 복귀	0	10회 검사 모두 LOW

위는 이론에 따른 시뮬레이션 예상 결과이다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

정상 동작을 확인한 후, 테스트벤치에서 회로에 전달하는 LEVELS 파라미터를 10에서 5로 변경하였다. 출력 비교에 사용하는 기대값은 그대로 유지하였다.

변경 후에는 버튼을 한 번 누를 때마다 듀티비가 20%p씩 증가하므로, 세 번 누른 상태의 듀티비는 60%가 된다. 따라서 PWM 한 주기인 10클록 중 HIGH 구간이 기존 3클록에서 6클록으로 증가하였다. 1251 ns의 검사에서 기대값 3과 실제값 6이 일치하지 않아 “level=3 high=6” 오류가 발생하였다.

이는 문법 오류가 아니라 파라미터 변경에 따른 출력과 기존 기대값의 불일치로 발생한 자기검사 실패이다. 이후 LEVELS를 10으로 복구하고 다시 실행한 결과, “LAB3_LED_PWM_PASS checks=4”가 출력되어 정상 동작으로 복구되었음을 확인하였다.

6.2. RGB LED PWM

작성한 코드는 다음과 같다.

```
`timescale 1ns/1ps
module lab3_rgb_pwm #(
parameter integer CLK_HZ = 50_000_000,
parameter integer PWM_HZ = 1_000,
parameter integer LEVELS = 10,
parameter integer DEBOUNCE_CYCLES = 1_000_000
) (
input wire clk_50mhz,
input wire rst_p,
input wire button_r,
input wire button_g,
input wire button_b,
output wire [3:0] led_r,
output wire [3:0] led_g,
output wire [3:0] led_b
);
wire press_r, press_g, press_b;
wire pwm_r, pwm_g, pwm_b;
reg [3:0] level_r, level_g, level_b;

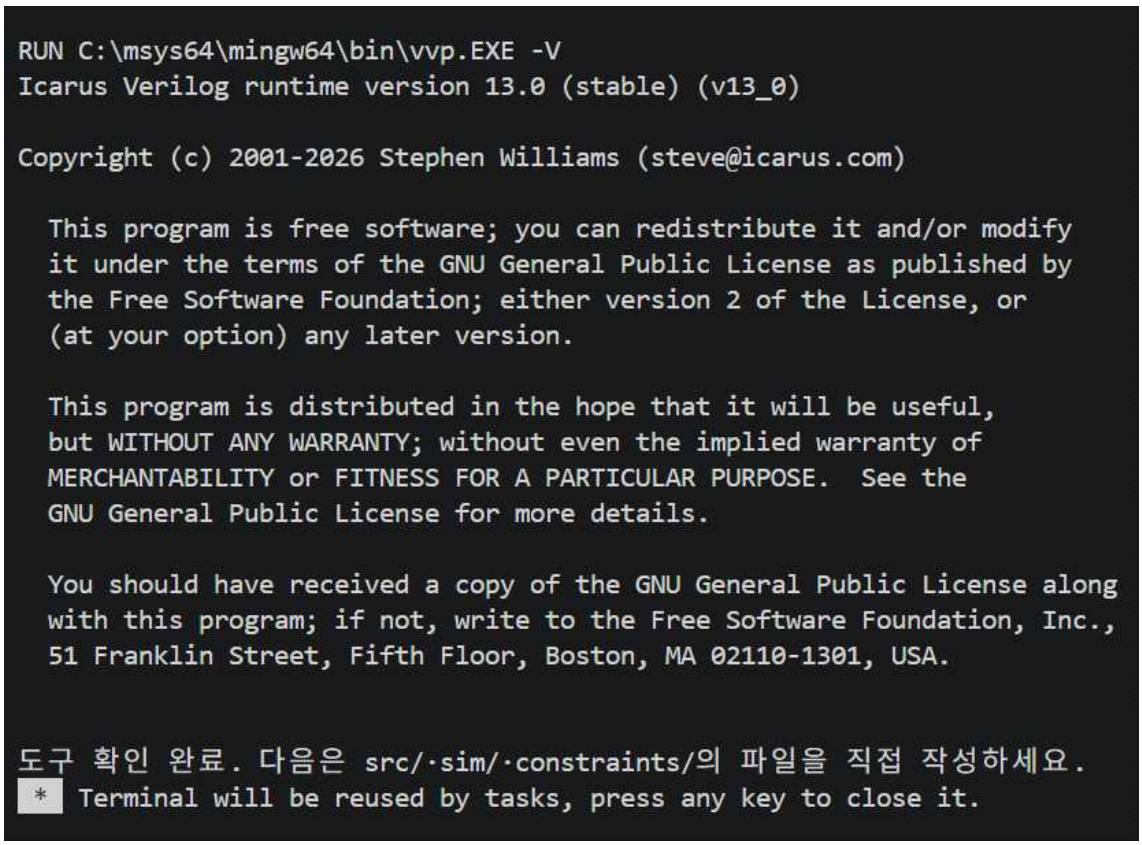
button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_r (
.clk(clk_50mhz), .rst_p(rst_p), .button(button_r), .pulse(press_r));
button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_g (
.clk(clk_50mhz), .rst_p(rst_p), .button(button_g), .pulse(press_g));
button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_b (
.clk(clk_50mhz), .rst_p(rst_p), .button(button_b), .pulse(press_b));

always @(posedge clk_50mhz or posedge rst_p) begin
if (rst_p) begin
level_r <= 0; level_g <= 0; level_b <= 0;
end else begin
if (press_r) level_r <= (level_r == LEVELS) ? 0 : level_r + 1'b1;
if (press_g) level_g <= (level_g == LEVELS) ? 0 : level_g + 1'b1;
if (press_b) level_b <= (level_b == LEVELS) ? 0 : level_b + 1'b1;
end
end

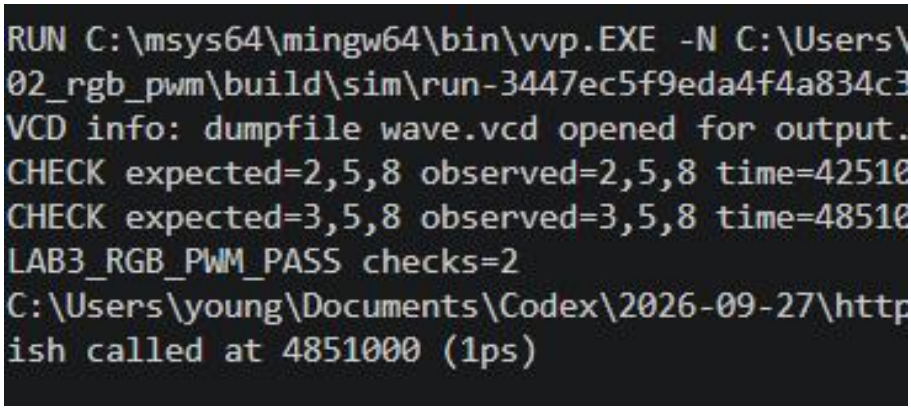
pwm_channel #(.PERIOD_CYCLES(CLK_HZ / PWM_HZ), .LEVELS(LEVELS)) u_pwm_r
(.clk(clk_50mhz), .rst_p(rst_p), .level(level_r), .pwm(pwm_r));
pwm_channel #(.PERIOD_CYCLES(CLK_HZ / PWM_HZ), .LEVELS(LEVELS)) u_pwm_g
(.clk(clk_50mhz), .rst_p(rst_p), .level(level_g), .pwm(pwm_g));
pwm_channel #(.PERIOD_CYCLES(CLK_HZ / PWM_HZ), .LEVELS(LEVELS)) u_pwm_b
(.clk(clk_50mhz), .rst_p(rst_p), .level(level_b), .pwm(pwm_b));

assign led_r = {4{pwm_r}};
assign led_g = {4{pwm_g}};
assign led_b = {4{pwm_b}};
endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.



check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

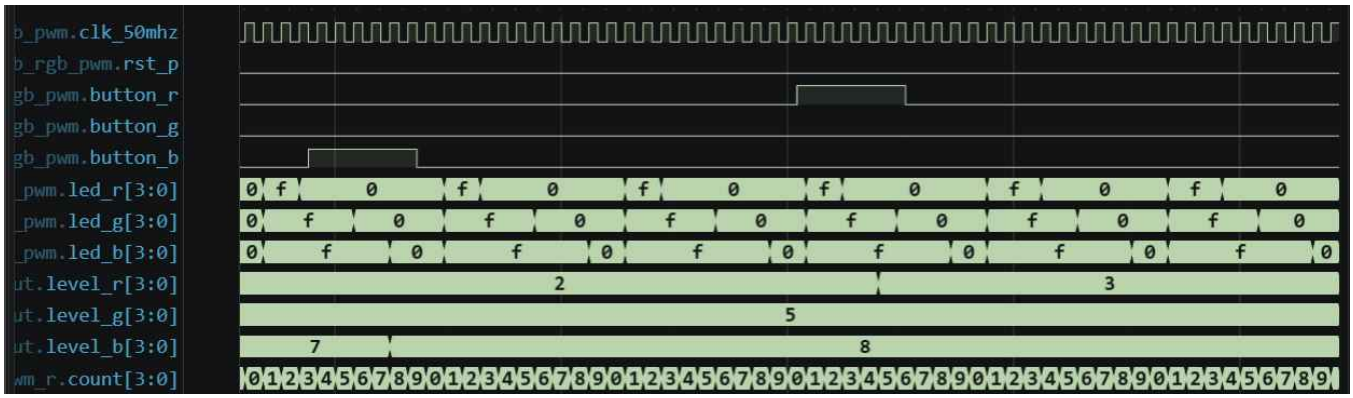


위 터미널 응답에서 알 수 있듯이 2개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

확인 시간 구간	검사 조건	밝기 단계 예상값	출력 예상값
4071~4251 ns	R 버튼 2회, G 버튼 5회, B 버튼 8회 입력 후	level_r=2, level_g=5, level_b=8	R: 처음 2회 HIGH, 이후 8회 LOW / G: 처음 5회 HIGH, 이후 5회 LOW / B: 처음 8회 HIGH, 이후 2회 LOW
4671~4851 ns	R 버튼만 1회 추가 입력 후	level_r=3, level_g=5, level_b=8	R: 처음 3회 HIGH, 이후 7회 LOW / G: 처음 5회 HIGH, 이후 5회 LOW / B: 처음 8회 HIGH, 이후 2회 LOW

위는 이론에 따른 시뮬레이션 예상 결과이다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

R 채널의 초기 밝기 단계를 0에서 1로 변경하고 테스트벤치의 입력과 기대값은 유지하였다. 이에 따라 R 버튼을 두 번 누른 후의 밝기 단계가 2가 아닌 3이 되어, 듀티비가 20%에서 30%로 증가하였다. G와 B는 기존의 50%, 80%를 유지하였다.

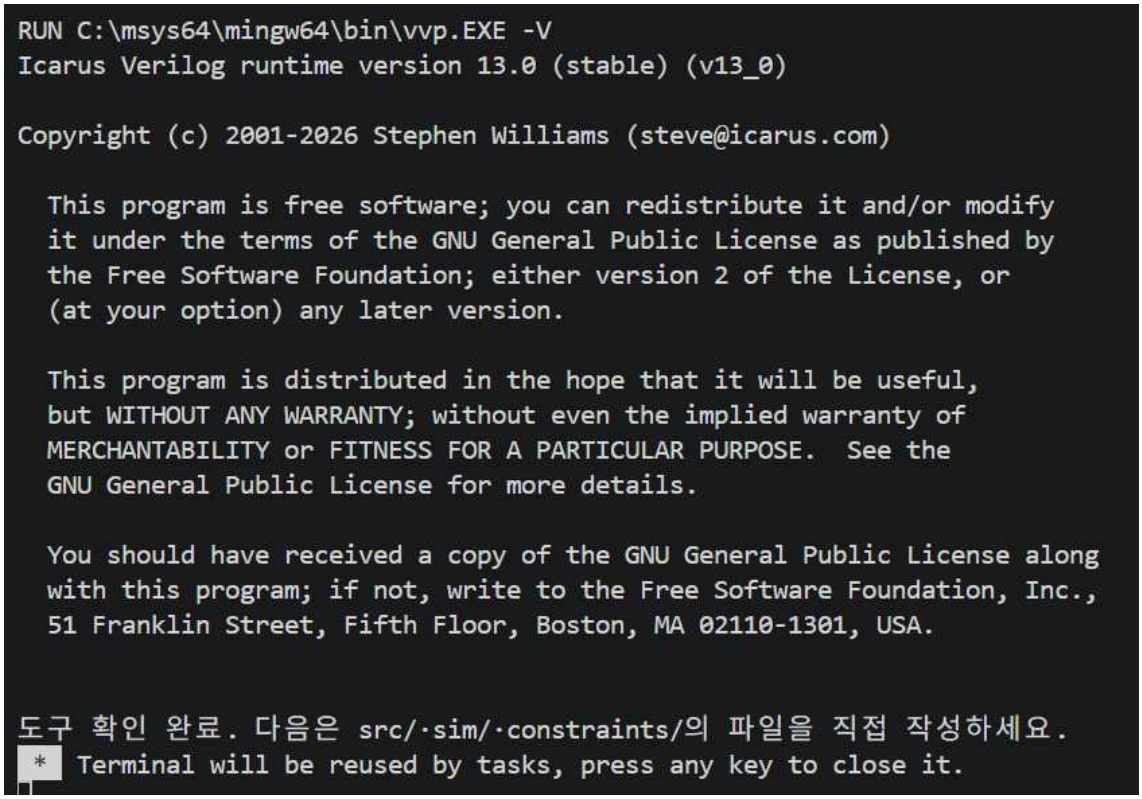
4251 ns의 검사에서 R의 HIGH 클록 수가 기대값 2와 달리 3으로 측정되어 “rgb high=3,5,8” 오류가 발생하였다. 이후 초기값을 0으로 복구한 결과, “LAB3_RGB_PWM_PASS checks=2”가 출력되어 정상 동작으로 복구되었음을 확인하였다.

6.3. Piezo

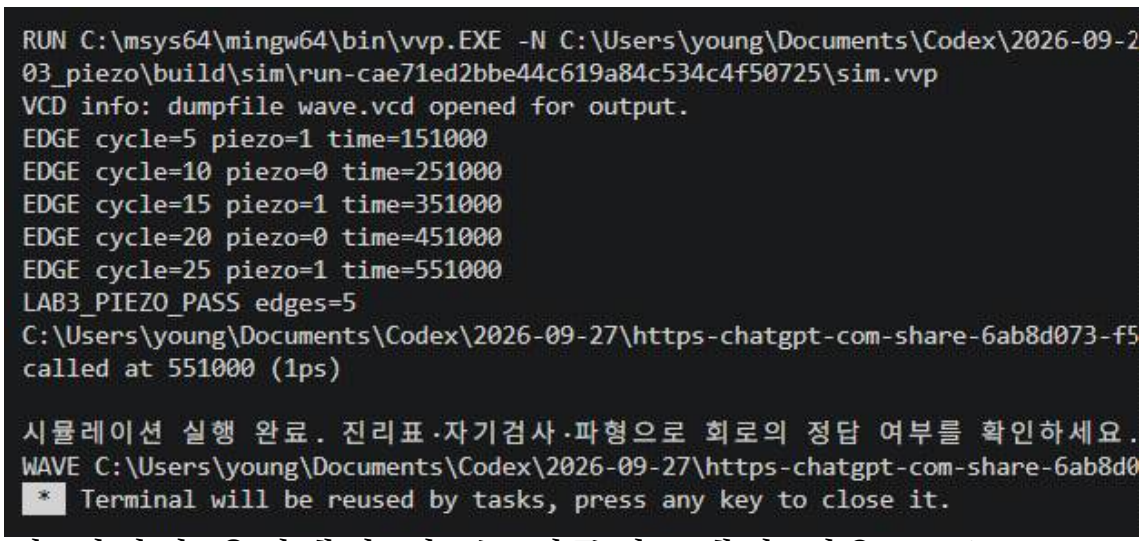
작성한 코드는 다음과 같다.

```
`timescale 1ns/1ps
module tb_piezo;
reg clk_50mhz=0,rst_p=1;
wire piezo;
reg previous_piezo=0;
integer cycles,last_cycle=-1,edges=0;
always #10 clk_50mhz=~clk_50mhz;
lab3_piezo #(.CLK_HZ(1000),.TONE_HZ(100)) dut(.clk_50mhz(clk_50mhz),.rst_p(rst_p),.piezo(piezo) );
initial begin
$dumpfile("wave.vcd");$dumpvars(0,tb_piezo);
repeat(3)@(negedge clk_50mhz);rst_p=0;
for(cycles=1;cycles<=25;cycles=cycles+1)begin
@(posedge clk_50mhz);#1;
if(piezo !== ((cycles / 5) % 2 == 1))
    $fatal(1,"cycle=%0d expected=%0d piezo=%b",cycles,(cycles / 5) % 2,piezo);
if(piezo !== previous_piezo)begin
if(last_cycle>=0 && cycles-last_cycle!=5)$fatal(1,"half period=%0d",cycles-last_cycle);
last_cycle=cycles;edges=edges+1;
$display("EDGE cycle=%0d piezo=%b time=%0t",cycles,piezo,$time);
end
previous_piezo=piezo;
end
if(edges!=5)$fatal(1,"too few edges");
$display("LAB3_PIEZO_PASS edges=%0d",edges); $finish;
end
initial begin □#5000;$fatal(1,"timeout");end
endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.



check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

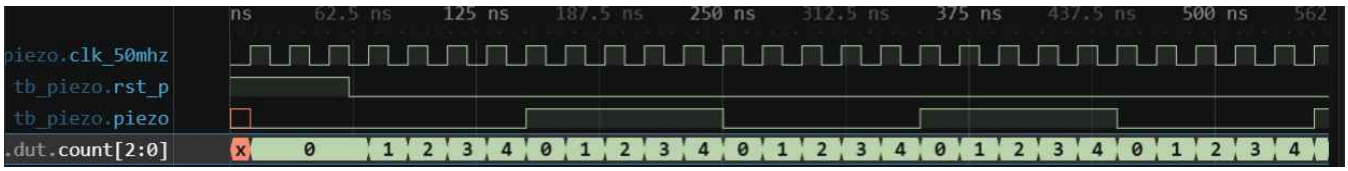


위 터미널 응답에서 알 수 있듯이 5개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.

이후 파형 출력을 통해 시간 구간별로 올바를 출력을 내놓는지 분석하였다.

확인 시간 구간	piezo 예상값	확인 사항
150 ns 이상~250 ns 미만	1	HIGH가 100 ns 동안 유지된다.
250 ns 이상~350 ns 미만	0	LOW가 100 ns 동안 유지된다.
350 ns 이상~450 ns 미만	1	동일한 HIGH 구간이 반복된다.
450 ns 이상~550 ns 미만	0	동일한 LOW 구간이 반복된다.
550 ns	0 → 1	다섯 번째 출력 반전이 발생한다.

위의 표의 결과가 시뮬레이션의 예상 결과값이다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

테스트벤치에서 회로에 전달하는 TONE_HZ를 100에서 50으로 변경하고, 출력 비교에 사용하는 기대값은 유지하였다. 반주기 클록 수는 CLK_HZ/(2×TONE_HZ)이므로, 변경 후에는 5클록에서 10클록으로 증가한다.

기존 설정에서는 150 ns에 출력이 HIGH로 반전되어야 하지만, 변경한 회로에서는 아직 반주기 계수가 끝나지 않아 LOW를 유지한다.
따라서 151 ns의 검사에서 “cycle=5 expected=1 piezo=0” 오류가 발생하였다. 시뮬레이션은 이 첫 불일치에서 종료되었다.

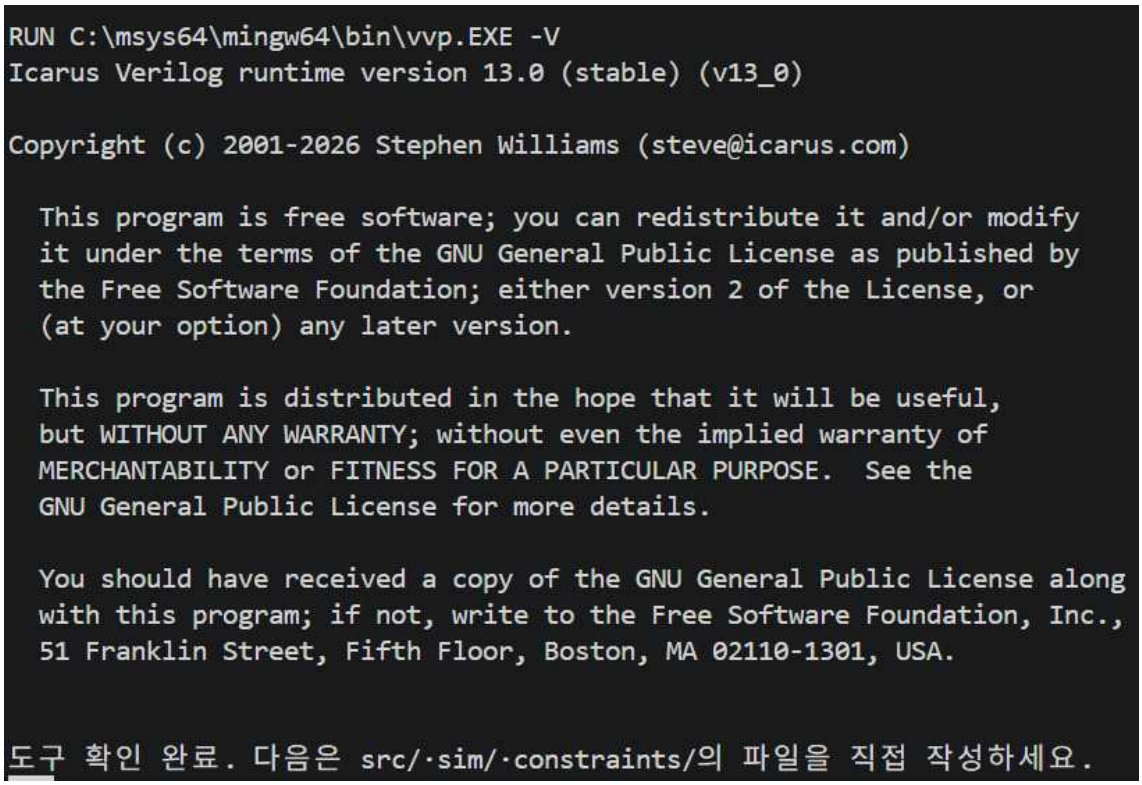
이후 TONE_HZ를 100으로 복구하고 다시 실행한 결과, 551 ns에서 “LAB3_PIEZO_PASS edges=5”가 출력되어 정상 동작으로 복구되었음을 확인하였다.

6.4. Stepper

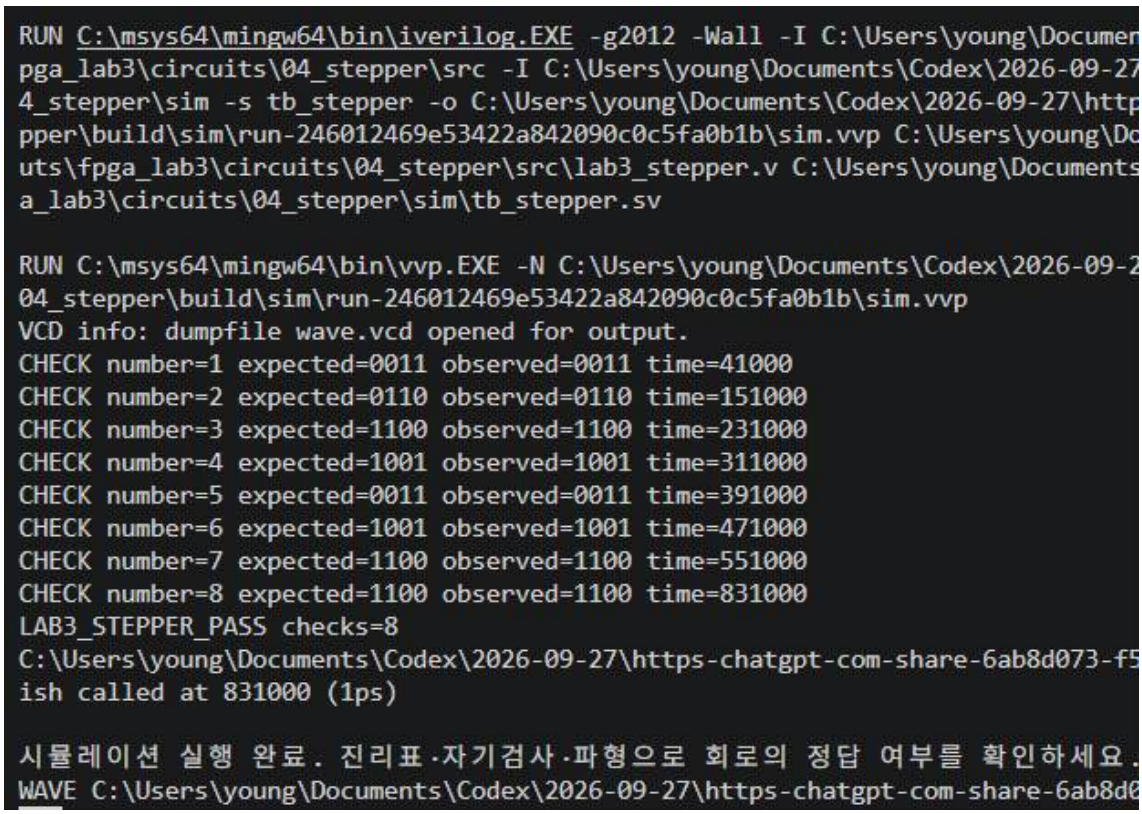
작성한 코드는 다음과 같다.

```
`timescale 1ns/1ps
module tb_stepper;
reg clk_50mhz=0,rst_p=1,enable=0,direction=0;
wire [3:0] stepmotor;
integer checks=0;
always #10 clk_50mhz=~clk_50mhz;
lab3_stepper #(.CLK_HZ(8),.STEP_HZ(2)) dut
(.clk_50mhz(clk_50mhz),.rst_p(rst_p),.enable(enable),.direction(direction),.stepmotor( stepmotor));
task check_value(input [3:0] value); begin #1;if(stepmotor!==value)$fatal(1,"got=%b expected=%b" ,stepmotor,valu
$display("CHECK number=%0d expected=%b observed=%b time=%0t",checks,value,stepmotor,$time);
end endtask
initial begin
$dumpfile("wave.vcd");$dumpvars(0,tb_stepper);
repeat(2)@(negedge clk_50mhz);rst_p=0;check_value(4'b0011);
enable=1;wait(dut.enable_sync);wait(dut.state==1);check_value(4'b0110);
wait(dut.state==2);check_value(4'b1100);
wait(dut.state==3);check_value(4'b1001);
wait(dut.state==0);check_value(4'b0011);
direction=1;wait(dut.direction_sync);wait(dut.state==3);check_value(4'b1001);
wait(dut.state==2);check_value(4'b1100);
enable=0;wait(!dut.enable_sync);repeat(12)@(posedge clk_50mhz);check_value(4'b1100);
$display("LAB3_STEPPER_PASS checks=%0d",checks); $finish;
end
initial begin □#5000;$fatal(1,"timeout");end
endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.



check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.



```
RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -I C:\Users\young\Documer
pga_lab3\circuits\04_stepper\src -I C:\Users\young\Documents\Codex\2026-09-27
4_stepper\sim -s tb_stepper -o C:\Users\young\Documents\Codex\2026-09-27\http
pper\build\sim\run-246012469e53422a842090c0c5fa0b1b\sim.vvp C:\Users\young\Do
uts\fpga_lab3\circuits\04_stepper\src\lab3_stepper.v C:\Users\young\Documents
a_lab3\circuits\04_stepper\sim\tb_stepper.sv

RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\Codex\2026-09-2
04_stepper\build\sim\run-246012469e53422a842090c0c5fa0b1b\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
CHECK number=1 expected=0011 observed=0011 time=41000
CHECK number=2 expected=0110 observed=0110 time=151000
CHECK number=3 expected=1100 observed=1100 time=231000
CHECK number=4 expected=1001 observed=1001 time=311000
CHECK number=5 expected=0011 observed=0011 time=391000
CHECK number=6 expected=1001 observed=1001 time=471000
CHECK number=7 expected=1100 observed=1100 time=551000
CHECK number=8 expected=1100 observed=1100 time=831000
LAB3_STEPPER_PASS checks=8
C:\Users\young\Documents\Codex\2026-09-27\https-chatgpt-com-share-6ab8d073-f5
ish called at 831000 (1ps)

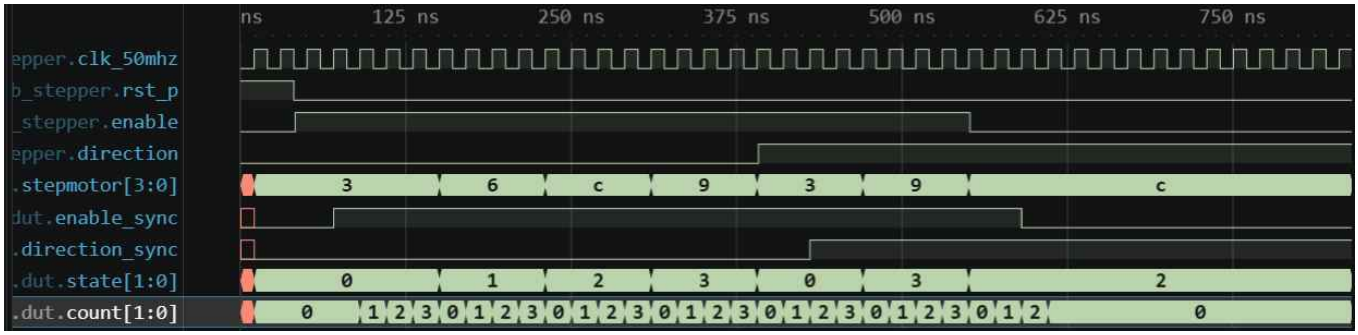
시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\young\Documents\Codex\2026-09-27\https-chatgpt-com-share-6ab8d0
```

위 터미널 응답에서 알 수 있듯이 8개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

확인 구간	파형에서 확인할 내용
150~390 ns	정방향으로 0110 → 1100 → 1001 → 0011, 80 ns마다 변경
470~550 ns	역방향으로 1001 → 1100 변경
590~831 ns	enable_sync=0, state=2, stepmotor=1100 유지

위는 이론에 따른 시뮬레이션 예상 결과이다.



테스트벤치에서 회로에 전달하는 STEP_HZ를 2에서 1로 변경하였다. 단계 전환에 필요한 클록 수는 CLK_HZ/STEP_HZ이므로, 변경 후에는 4클록에서 8클록으로 증가한다.

시뮬레이션 결과, 정방향 출력 패턴은 기존과 동일한 순서로 나타났지만 단계 사이의 간격은 80 ns에서 160 ns로 증가하였다. 예를 들어 수정 후에는 231 ns, 391 ns, 551 ns, 711 ns의 검사에서 각각 0110, 1100, 1001, 0011이 확인되었다.
테스트벤치는 정해진 상태에 도달한 후 출력 패턴을 비교하므로, 전환 속도가 느려져도 패턴이 올바르면 PASS가 발생한다.

따라서 이번 수정에서는 검사 실패보다 상태 전환 간격과 종료 시각의 변화를 확인하였다. 이후 STEP_HZ를 2로 복구하여 기존 간격과 종료 시각으로 돌아왔음을 확인하였다.

6.5. MMSS Clock

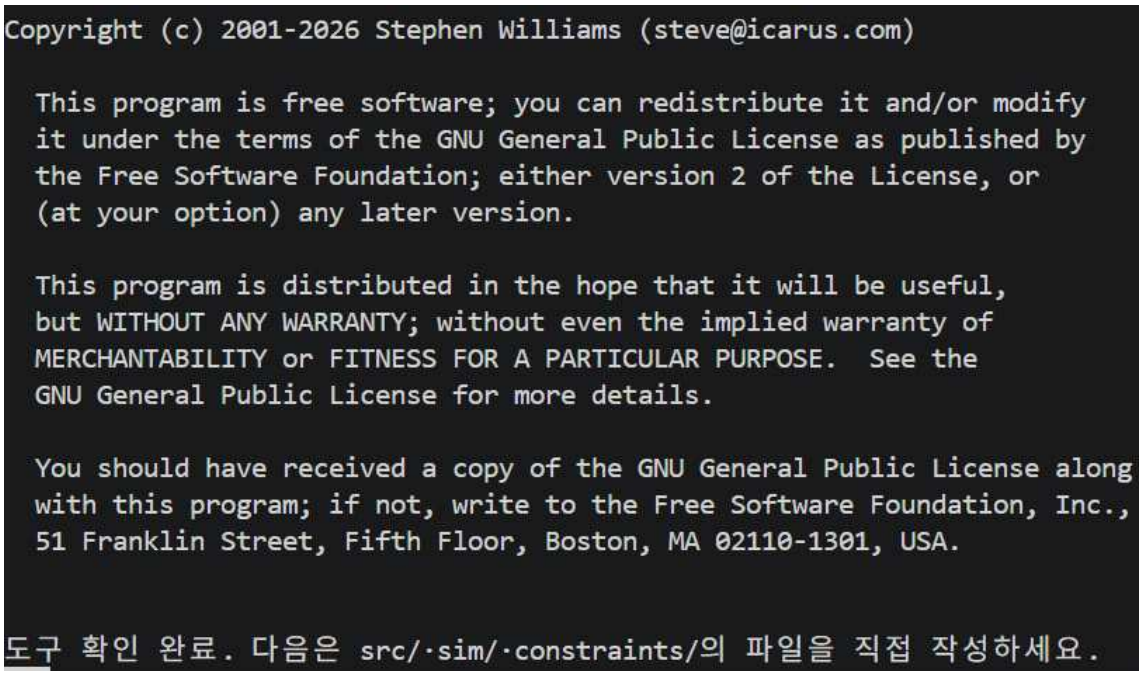
작성한 코드는 다음과 같다.

tb_mmss_counter.sv

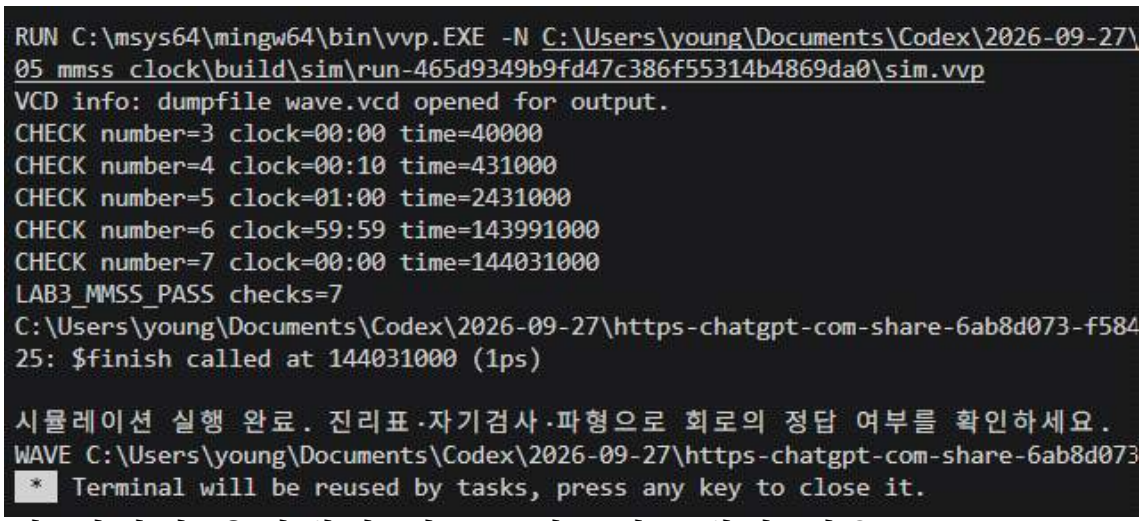
circuits > 05_mmss_clock > sim > tb_mmss_counter.sv

```
`timescale 1ns/1ps
module tb_mmss_counter;
reg clk_50mhz=0,rst_p=1;
wire [3:0] mt,mo,st,so;
reg [3:0] decode_digit;
wire [7:0] decoded_segments;
integer checks=0;
always #10 clk_50mhz=~clk_50mhz;
mmss_counter #(.CLK_HZ(2)) dut(.clk(clk_50mhz),.rst_p(rst_p),.minute_tens(mt),.minute_ones(mo) ,.second_tens(st)
sevenseg_decode decoder(.digit(decode_digit),.segments(decoded_segments));
task advance(input integer seconds);begin repeat (seconds*2)@(posedge clk_50mhz);#1;end endtask
task check_time(input integer emt,input integer emo,input integer est,input integer eso);
begin if (mt!==emt||mo!==emo||st!==est||so!==eso)$fatal(1,"time=%0d%0d:%0d%0d",mt,mo,st,so); checks++;
$display("CHECK number=%0d clock=%0d%0d:%0d%0d time=%0t",checks,mt,mo,st,so,$time);
end endtask
initial begin
$dumpfile("wave.vcd");$dumpvars(0,tb_mmss_counter);
decode_digit=0;#1;if(decoded_segments!==8'b1111_1100)$fatal(1,"digit 0 decode");checks++;
decode_digit=9;#1;if(decoded_segments!==8'b1111_0110)$fatal(1,"digit 9 decode");checks++;
repeat(2)@(negedge clk_50mhz);rst_p=0;check_time(0,0,0,0);
advance(10);check_time(0,0,1,0);
advance(50);check_time(0,1,0,0);
advance(3539);check_time(5,9,5,9);
advance(1);check_time(0,0,0,0);
$display("LAB3_MMSS_PASS checks=%0d",checks); $finish;
end
initial begin #200000;$fatal(1,"timeout");end
endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.



check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

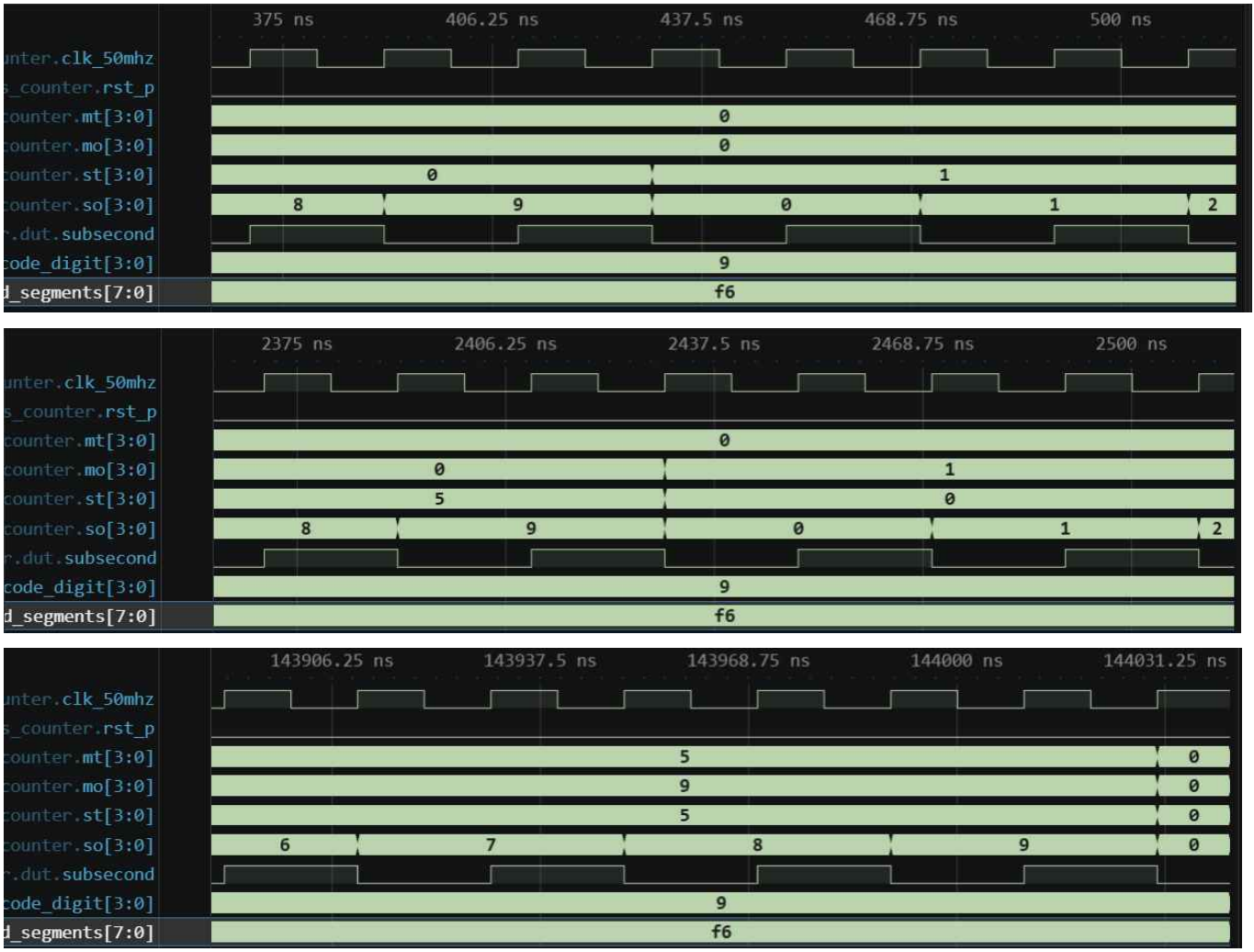


위 터미널 응답에서 알 수 있듯이 7개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.
이론에 따르면,

확인 시각	mt	mo	st	so	예상 시간 및 의미
391 ns	0	0	0	9	00:09, 초 자리올림 직전
431 ns	0	0	1	0	00:10, 초의 십의 자리로 자리올림
2391 ns	0	0	5	9	00:59, 분 자리올림 직전
2431 ns	0	1	0	0	01:00, 분 증가 및 초 초기화
143991 ns	5	9	5	9	59:59, 전체 순환 직전
144031 ns	0	0	0	0	00:00으로 복귀

과 같은 출력을 내놓아야 한다.



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을
실행해보았다.

테스트벤치에서 시간 카운터에 전달하는 CLK_HZ를 2에서 4로 변경하였다. 입력 클록의 주기는 20 ns로 유지하고, 테스트벤치의 대기 클록 수와 기대값은 변경하지 않았다.

변경 후에는 시계의 시간이 1초 증가하는 데 필요한 클록 수가 두 배가 되므로, 같은 시뮬레이션 시간 동안 계수되는 시간은 절반이 된다. 이에 따라 431 ns에서 기존 기대값인 00:10 대신 00:05가 출력되어 자기검사가 실패하였다. 수정 실행은 이 첫 불일치에서 종료되었다.

이후 CLK_HZ를 2로 복구한 결과, 144031 ns에서 “LAB3_MMSS_PASS checks=7”이 출력되어 자리올림과 전체 순환 동작이 정상적으로 복구되었음을 확인하였다.

6.6. Character LCD

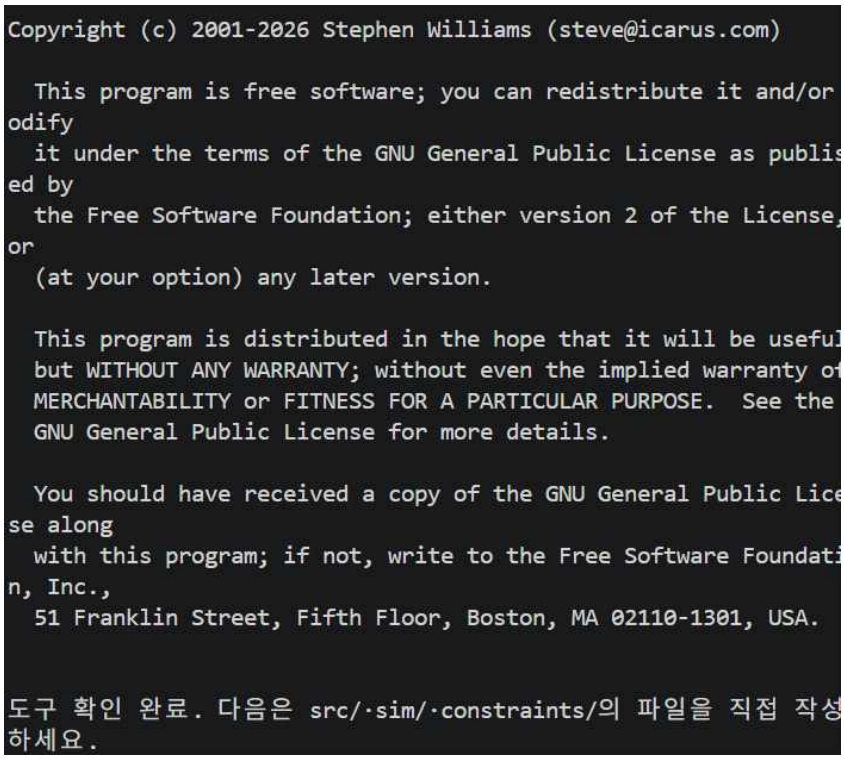
작성한 코드는 다음과 같다.

wave.vcd ● tb_character_lcd.sv ×

circuits > 06_character_lcd > sim > tb_character_lcd.sv

```
1  `timescale 1ns/1ps
2  module tb_character_lcd;
3  reg clk_50mhz=0,rst_p=1;
4  wire lcd_e,lcd_rs,lcd_rw;
5  wire [7:0] lcd_data;
6  reg [7:0] expected_data[0:39];
7  reg expected_rs[0:39];
8  integer count=0,i;
9  always #10 clk_50mhz=~clk_50mhz;
10 lab3_character_lcd #(.TICK_CYCLES(2),.POWER_TICKS(3),.NORMAL_WAIT_TICKS(1),.CLEAR_WAIT_TICKS(2) ) dut
11 (.clk_50mhz(clk_50mhz),.rst_p(rst_p),.lcd_e(lcd_e),.lcd_rs(lcd_rs),.lcd_rw(lcd_rw),.lcd_data( lcd_data));
12
13 initial begin
14 expected_data[0]=8'h38;expected_data[1]=8'h38;expected_data[2]=8'h38;
15 expected_data[3]=8'h0c;expected_data[4]=8'h06;expected_data[5]=8'h01;expected_data[6]=8'h80;
16 expected_data[7]="F";expected_data[8]="P";expected_data[9]="G";expected_data[10]="A";
17 expected_data[11]=" ";expected_data[12]="L";expected_data[13]="A";expected_data[14]="B"; expected_data[15]="3";
18 for(i=16;i<=22;i=i+1)expected_data[i]=" ";
19 expected_data[23]=8'hc0;
20 expected_data[24]="L";expected_data[25]="C";expected_data[26]="D";expected_data[27]=" ";
21 expected_data[28]="C";expected_data[29]="O";expected_data[30]="N";expected_data[31]="T";
22 expected_data[32]="R";expected_data[33]="O";expected_data[34]="L";expected_data[35]="L";
23 expected_data[36]="E";expected_data[37]="R";expected_data[38]=" ";expected_data[39]=" ";
24 for(i=0;i<40;i=i+1)expected_rs[i]=(i>=7 && i!=23);
25 expected_rs[38]=1;expected_rs[39]=1;
26 end
27 always @(negedge lcd_e) if(!rst_p && count<40) begin
28 if(lcd_rw!==0||lcd_rs!==expected_rs[count]||lcd_data!==expected_data[count])
29 $fatal(1,"index=%0d rs=%b data=%h",count,lcd_rs,lcd_data);
30 $display("BYTE index=%0d rs=%b data=%h time=%0t",count,lcd_rs,lcd_data,$time);
31 count=count+1;
32 if(count==40)begin $display ("LAB3_LCD_PASS bytes=%0d",count); $finish;end
33 end
34 initial begin
35 $dumpfile("wave.vcd");$dumpvars(0,tb_character_lcd);
36 repeat(3)@(negedge clk_50mhz);rst_p=0;
37 end
38 initial begin #50000;$fatal(1,"timeout count=%0d",count); end
39 endmodule
```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.



check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

```
RUN C:\msys64\mingw64\bin\vvp.EXE -N C:\Users\young\Documents\Codex\2026-09-27
build\sim\run-3cceb5371bc04269ace405742bccebf6\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
BYTE index=0 rs=0 data=38 time=310000
BYTE index=1 rs=0 data=38 time=510000
BYTE index=2 rs=0 data=38 time=710000
BYTE index=3 rs=0 data=0c time=910000
BYTE index=4 rs=0 data=06 time=1110000
BYTE index=5 rs=0 data=01 time=1310000
BYTE index=6 rs=0 data=80 time=1550000
BYTE index=7 rs=1 data=46 time=1750000
BYTE index=8 rs=1 data=50 time=1950000
BYTE index=9 rs=1 data=47 time=2150000
BYTE index=10 rs=1 data=41 time=2350000
BYTE index=11 rs=1 data=20 time=2550000
BYTE index=12 rs=1 data=4c time=2750000
BYTE index=13 rs=1 data=41 time=2950000
BYTE index=14 rs=1 data=42 time=3150000
BYTE index=15 rs=1 data=33 time=3350000
BYTE index=16 rs=1 data=20 time=3550000
BYTE index=17 rs=1 data=20 time=3750000
BYTE index=18 rs=1 data=20 time=3950000
BYTE index=19 rs=1 data=20 time=4150000
BYTE index=20 rs=1 data=20 time=4350000
BYTE index=21 rs=1 data=20 time=4550000
BYTE index=22 rs=1 data=20 time=4750000
BYTE index=23 rs=0 data=c0 time=4950000
BYTE index=24 rs=1 data=4c time=5150000
BYTE index=25 rs=1 data=43 time=5350000
BYTE index=26 rs=1 data=44 time=5550000
BYTE index=27 rs=1 data=20 time=5750000
BYTE index=28 rs=1 data=43 time=5950000
BYTE index=29 rs=1 data=4f time=6150000
BYTE index=30 rs=1 data=4e time=6350000
BYTE index=31 rs=1 data=54 time=6550000
BYTE index=32 rs=1 data=52 time=6750000
BYTE index=33 rs=1 data=4f time=6950000
BYTE index=34 rs=1 data=4c time=7150000
BYTE index=35 rs=1 data=4c time=7350000
BYTE index=36 rs=1 data=45 time=7550000
BYTE index=37 rs=1 data=52 time=7750000
BYTE index=38 rs=1 data=20 time=7950000
BYTE index=39 rs=1 data=20 time=8150000
LAB3_LCD_PASS bytes=40
C:\Users\young\Documents\Codex\2026-09-27\https-chatgpt-com-share-6ab8d073-f58
alled at 8150000 (1ps)

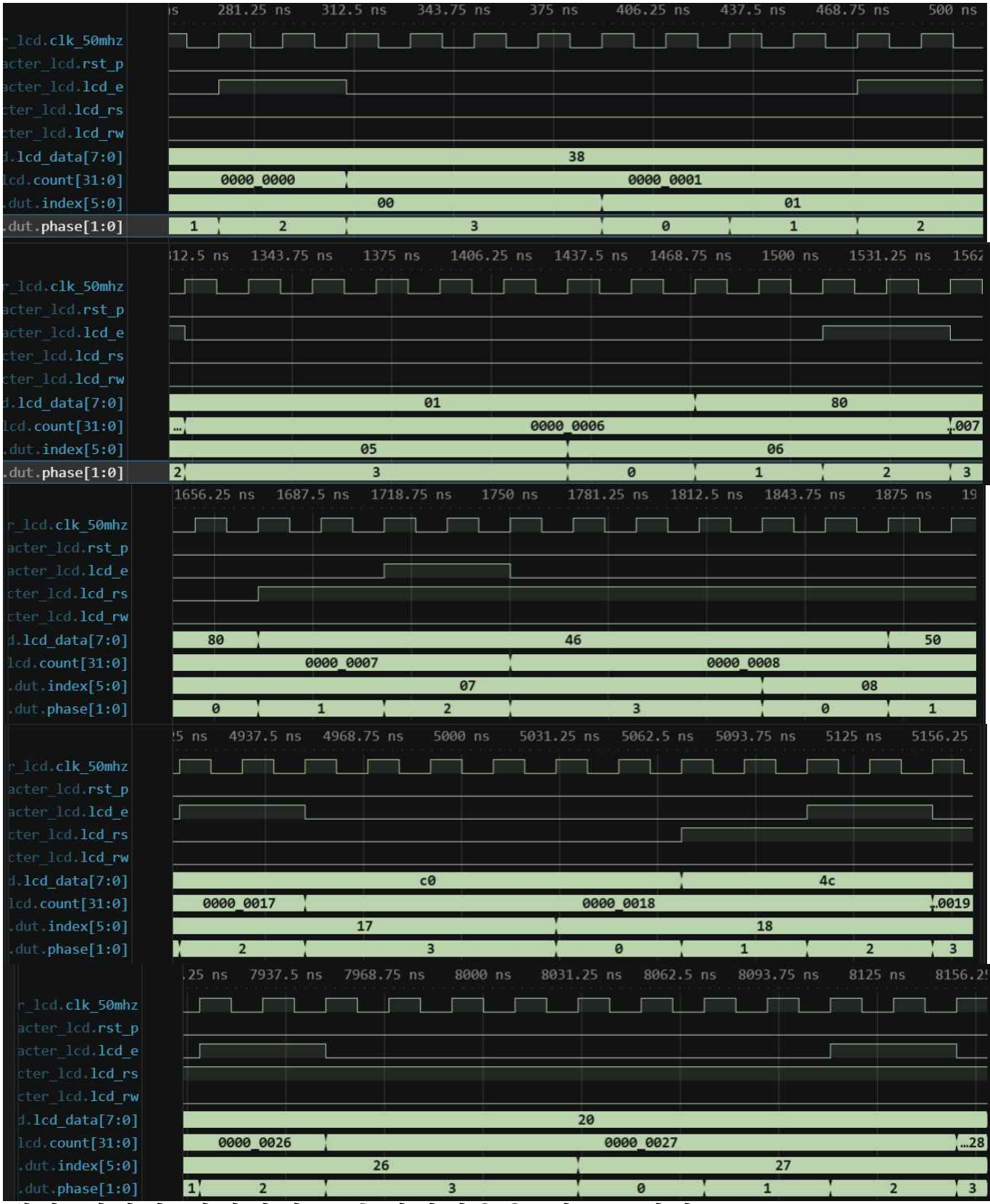
시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정답 여부를 확인하세요.
```

위 터미널 응답에서 알 수 있듯이 40개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.

이후 파형 출력을 통해 시간 구간별로 올바른 출력을 내놓는지 분석하였다.

이론에 따르면 첫 번째 상태 순환은 다음과 같아야 한다.

확인 시각	lcd_rs	lcd_data(16진수)	예상 동작
310 ns	0	38	LCD 기능 설정 명령 전송
1310 ns	0	01	화면 지우기 명령 전송
1550 ns	0	80	첫째 줄 시작 주소 설정
1750 ns	1	46	첫째 줄의 첫 문자 'F' 전송
4950 ns	0	C0	둘째 줄 시작 주소 설정
5150 ns	1	4C	둘째 줄의 첫 문자 'L' 전송
8150 ns	1	20	마지막 공백 전송 및 40바이트 검사 완료



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

둘째 줄 문자열의 E를 X로 변경하고 테스트벤치의 기대값은 그대로 유지하였다. E의 ASCII 코드는 16진수 45이고, X의 ASCII 코드는 58이다.

수정 후 7550 ns의 Enable 하강 에지에서 lcd_rs=1, lcd_data=58이 출력되었다. 테스트벤치는 index 36에서 45가 출력될 것으로 예상하므로 실제 출력과 일치하지 않아 오류가 발생하였다. 이 결과를 통해 문자열 한 글자의 변경이 LCD 데이터 버스의 ASCII 값에 직접 반영되는 것을 확인하였다.

이후 해당 문자를 다시 E로 복구하였다. 7550 ns에서 lcd_data=45가 출력되었고, 8150 ns에서 LAB3_LCD_PASS bytes=40이 나타나 전체 초기화 명령과 문자열 전송이 정상적으로 복구되었음을 확인하였다.

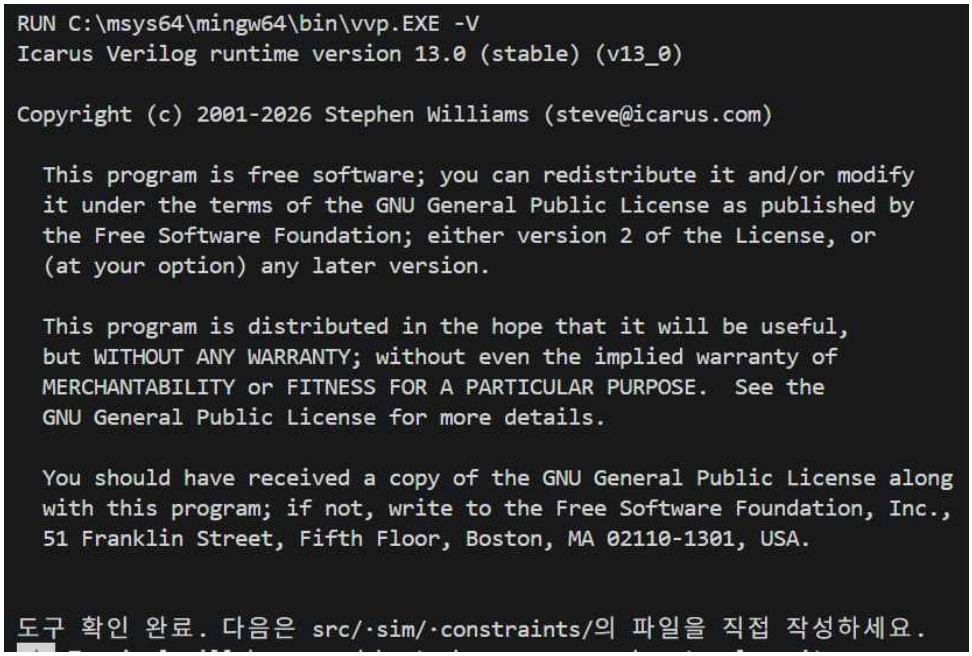
6.7. UART echo

작성한 코드는 다음과 같다.

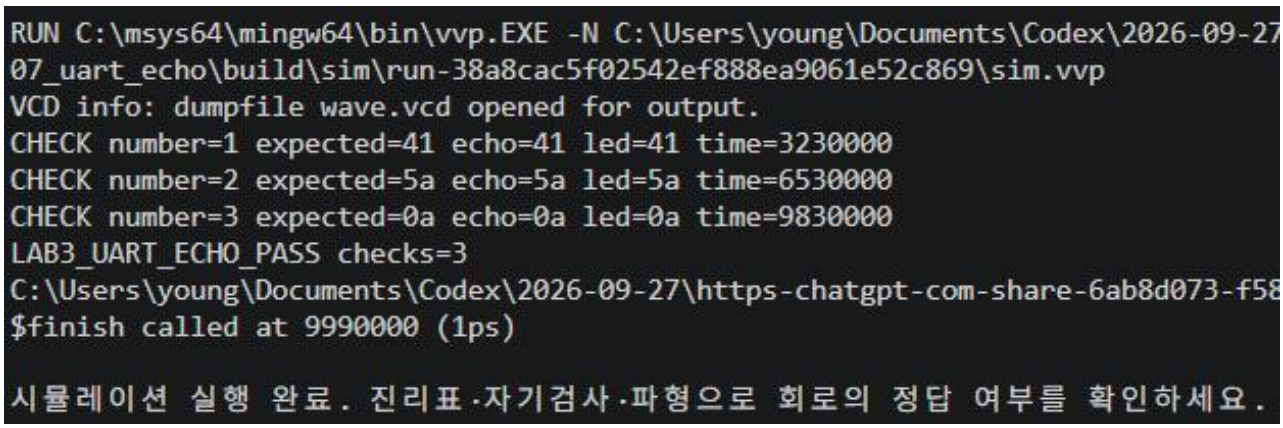
```
`timescale 1ns/1ps
module tb_uart_echo;
localparam integer DIV=8;
reg clk_50mhz=0,rst_p=1,uart_rxd=1;
wire uart_txd;
wire [7:0] led;
integer checks=0,i;
reg [7:0] observed;
always #10 clk_50mhz=~clk_50mhz;
lab3_uart_echo #(.CLK_HZ(800),.BAUD(100)) dut
(.clk_50mhz(clk_50mhz),.rst_p(rst_p),.uart_rxd(uart_rxd),.uart_txd(uart_txd),.led(led));
task send_byte(input [7:0] value);
begin
@(negedge clk_50mhz); uart_rxd=0;repeat(DIV)@(negedge clk_50mhz);
for(i=0;i<8;i=i+1)begin uart_rxd=value[i];repeat(DIV)@(negedge clk_50mhz);end
uart_rxd=1;repeat(DIV)@(negedge clk_50mhz);
end endtask
task receive_byte(input [7:0] expected);
begin
observed=0;@(negedge uart_txd);repeat(DIV+DIV/2)@(posedge clk_50mhz);
for(i=0;i<8;i=i+1)begin observed[i]=uart_txd;repeat(DIV)@(posedge clk_50mhz);end
if(observed!==expected)$fatal(1,"echo=%h expected=%h",observed,expected);
if(led!==expected)$fatal(1,"led=%h expected=%h",led,expected);
checks=checks+1;
$display("CHECK number=%0d expected=%h echo=%h led=%h time=%0t",checks,expected,observed,led,$time);
repeat(DIV)@(posedge clk_50mhz);
end endtask
initial begin
$dumpfile("wave.vcd");$dumpvars(0,tb_uart_echo);
repeat(4)@(negedge clk_50mhz);rst_p=0;
fork send_byte(8'h41); receive_byte(8'h41); join
fork send_byte(8'h5a); receive_byte(8'h5a); join
fork send_byte(8'h0a); receive_byte(8'h0a); join
if(dut.rx_framing_error)$fatal(1,"unexpected framing error");
$display("LAB3_UART_ECHO_PASS checks=%0d",checks); $finish;
end
initial begin #50000;$fatal(1,"timeout");end
endmodule

```

이 코드를 check tools를 실행하였다. 그 결과는 아래 사진으로 첨부하였다.



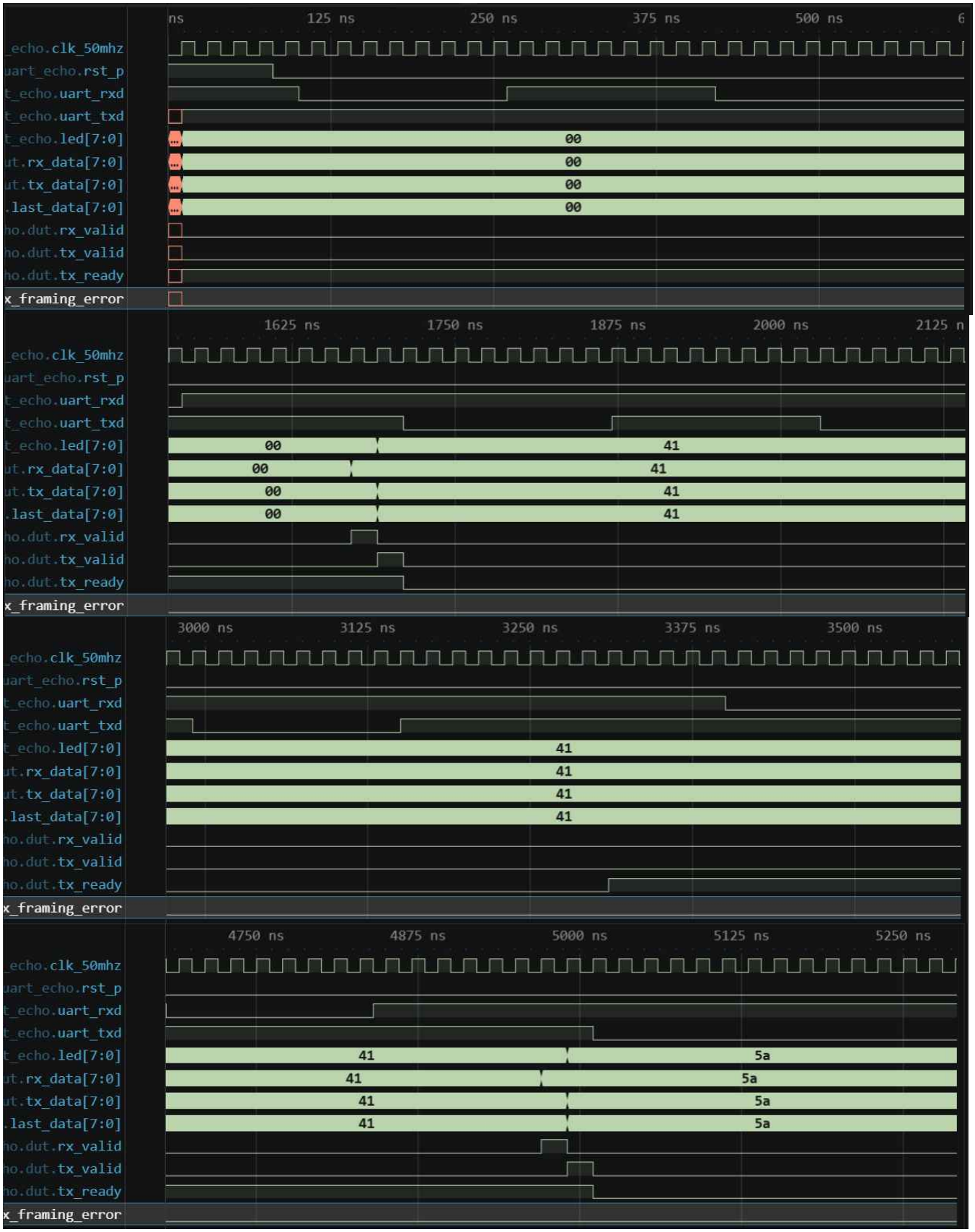
check tools를 실행한 결과 오류가 발생하지 않았으므로 다음으로 simulate를 실행하였다.

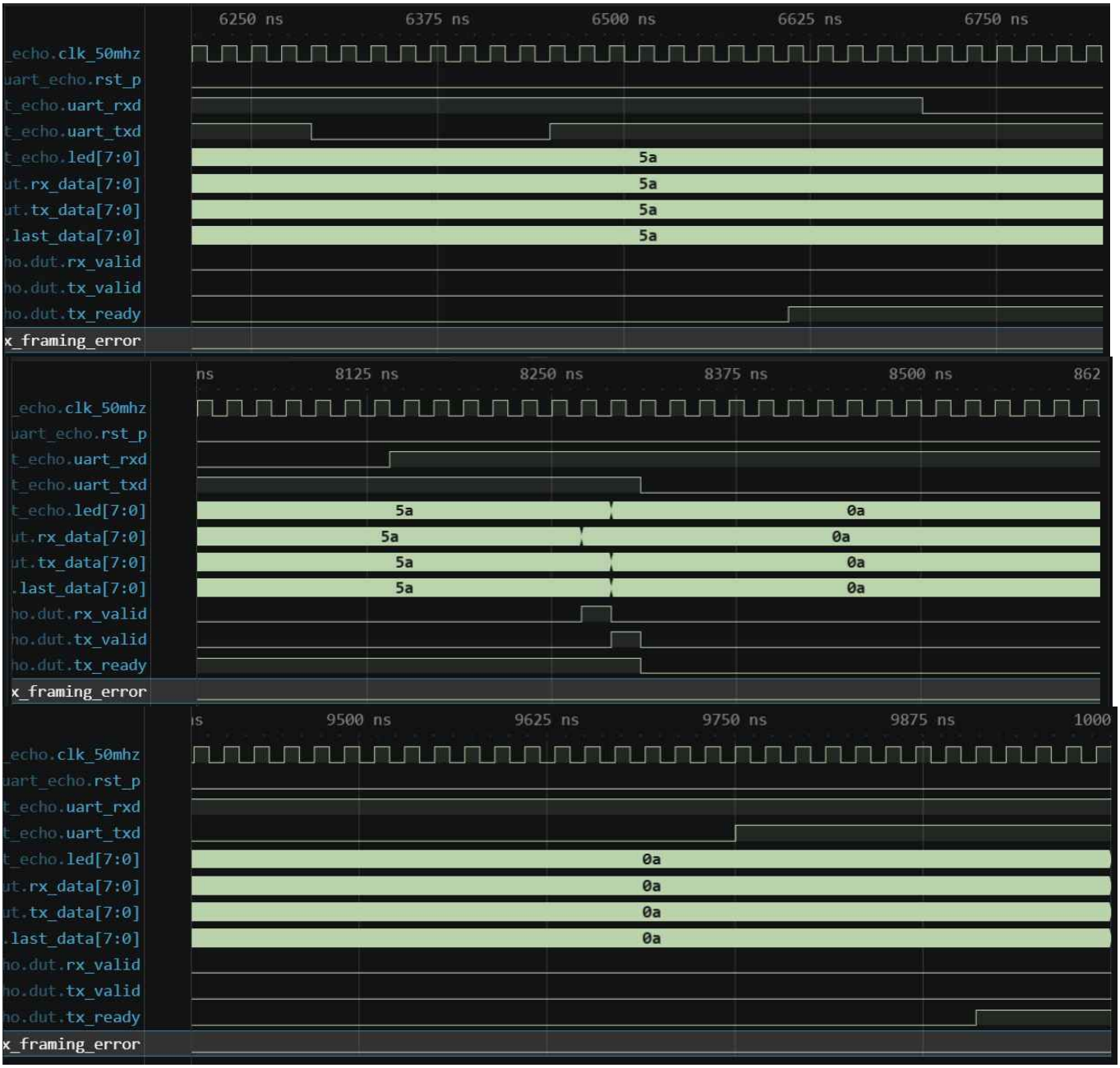


위 터미널 응답에서 알 수 있듯이 3개의 경우 모두 올바르게 출력되었다는 것을 알 수 있다.

이론에 따르면 다음과 같은 출력을 얻어야 한다.

확인 시각 또는 구간	예상값 및 동작
100 ns	uart_rxd가 LOW로 변하며 0x41 수신의 시작 비트가 입력된다.
1670 ns	0x41의 수신이 완료되어 rx_valid가 한 클록 동안 HIGH가 된다.
1690 ns	led와 last_data가 0x41로 변경되고 tx_valid가 HIGH가 된다.
1710 ns	uart_txd가 LOW로 변하며 0x41 에코의 시작 비트가 출력된다.
3230 ns	에코 데이터와 LED가 모두 0x41인지 첫 번째 검사를 완료한다.
4990 ns	두 번째 수신값 0x5A가 led와 last_data에 반영된다.
6530 ns	0x5A 에코와 LED의 두 번째 검사를 완료한다.
8290 ns	세 번째 수신값 0x0A가 led와 last_data에 반영된다.
9830 ns	0x0A 에코와 LED의 세 번째 검사를 완료한다.
9990 ns	LAB3_UART_ECHO_PASS checks=3을 출력하고 종료한다.


ns
125 ns
250 ns
375 ns
500 ns
_echo.clk_50mhz
uart_echo.rst_p
t_echo.uart_rxd
t_echo.uart_txd
t_echo.led[7:0]
ut.rx_data[7:0]
ut.tx_data[7:0]
.last_data[7:0]
no.dut.rx_valid
no.dut.tx_valid
no.dut.tx_ready
x_framing_error
00
1625 ns
1750 ns
1875 ns
2000 ns
2125 n
41
3000 ns
3125 ns
3250 ns
3375 ns
3500 ns
4750 ns
4875 ns
5000 ns
5125 ns
5250 ns
5a



예상 결과와 마찬가지로 출력되었음을 알 수 있다.

이번에는 일부러 오류를 발생시켜 위의 과정을 실행해보았다.

매뉴얼에 따라 DUT와 테스트 모델의 통신 속도를 함께 두 배로 변경했을 때 비트 시간은 160 ns에서 80 ns로 감소하였다.

송수신 속도가 서로 일치하므로 0x41, 0x5A, 0x0A는 모두 정상적으로 에코되었으며, 전체 종료 시간은 9990 ns에서 5190 ns로 단축되었다.

오류 검출을 확인하기 위해 DUT만 BAUD=200으로 설정하고 테스트 모델은 기존 DIV=8로 유지하였다. 두 장치의 데이터 샘플링 시점이 일치하지 않아 첫 번째 바이트의 에코가 기대값 0x41이 아닌 0xFA로 해석되었다. 이에 따라 3250 ns에서 “echo=fa expected=41” 오류가 발생하였다.

이후 DUT를 BAUD=100으로 복구하여 테스트 모델과 비트 시간을 다시 일치시켰다. 복구 후 0x41, 0x5A, 0x0A가 모두 정상적으로 에코되었고 “LAB3_UART_ECHO_PASS checks=3”을 확인하였다.

7. 포트 및 핀 번호 입력

7.1. Led PWM

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
# Combo II-DLD S75 / MAIN CLOCK F = 50 MHz.
set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
set_property PACKAGE_PIN K4 [get_ports rst_p]
set_property PACKAGE_PIN N8 [get_ports button]
set_property PACKAGE_PIN N5 [get_ports {led[0]}]
set_property PACKAGE_PIN M1 [get_ports {led[1]}]
set_property PACKAGE_PIN M3 [get_ports {led[2]}]
set_property PACKAGE_PIN M7 [get_ports {led[3]}]
set_property PACKAGE_PIN N7 [get_ports {led[4]}]
set_property PACKAGE_PIN M2 [get_ports {led[5]}]
set_property PACKAGE_PIN M4 [get_ports {led[6]}]
set_property PACKAGE_PIN L4 [get_ports {led[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports *]
create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
set_false_path -from [get_ports {rst_p button}]
```

7.2. RGB PWM

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
# Combo II-DLD S75 / MAIN CLOCK F = 50 MHz.
set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
set_property PACKAGE_PIN K4 [get_ports rst_p]
set_property PACKAGE_PIN N8 [get_ports button_r]
set_property PACKAGE_PIN N4 [get_ports button_g]
set_property PACKAGE_PIN N1 [get_ports button_b]
set_property PACKAGE_PIN T2 [get_ports {led_r[3]}]
set_property PACKAGE_PIN U1 [get_ports {led_r[2]}]
set_property PACKAGE_PIN P2 [get_ports {led_r[1]}]
set_property PACKAGE_PIN R3 [get_ports {led_r[0]}]
set_property PACKAGE_PIN U5 [get_ports {led_g[3]}]
set_property PACKAGE_PIN V1 [get_ports {led_g[2]}]
set_property PACKAGE_PIN R7 [get_ports {led_g[1]}]
set_property PACKAGE_PIN T6 [get_ports {led_g[0]}]
set_property PACKAGE_PIN U3 [get_ports {led_b[3]}]
set_property PACKAGE_PIN W2 [get_ports {led_b[2]}]
set_property PACKAGE_PIN R5 [get_ports {led_b[1]}]
set_property PACKAGE_PIN T3 [get_ports {led_b[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports *]
create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
set_false_path -from [get_ports {rst_p button_r button_g button_b}]
```

7.3. Piezo

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
# Combo II-DLD S75 / MAIN CLOCK F = 50 MHz.
set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
set_property PACKAGE_PIN K4 [get_ports rst_p]
set_property PACKAGE_PIN Y21 [get_ports piezo]
set_property IOSTANDARD LVCMOS33 [get_ports *]
create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
set_false_path -from [get_ports rst_p]
```

7.4. Stepper

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
# Combo II-DLD S75 / MAIN CLOCK F = 50 MHz.
set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
set_property PACKAGE_PIN K4 [get_ports rst_p]
set_property PACKAGE_PIN N8 [get_ports enable]
set_property PACKAGE_PIN N4 [get_ports direction]
set_property PACKAGE_PIN Y20 [get_ports {stepmotor[3]}]
set_property PACKAGE_PIN Y22 [get_ports {stepmotor[2]}]
set_property PACKAGE_PIN AA20 [get_ports {stepmotor[1]}]
set_property PACKAGE_PIN AA21 [get_ports {stepmotor[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports *]
create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
set_false_path -from [get_ports {rst_p enable direction}]
```

7.5. MMSS Clock

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
# Combo II-DLD S75 / MAIN CLOCK F = 50 MHz.
set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
set_property PACKAGE_PIN K4 [get_ports rst_p]
set_property PACKAGE_PIN H2 [get_ports {seg_data[0]}]
set_property PACKAGE_PIN J7 [get_ports {seg_data[1]}]
set_property PACKAGE_PIN J3 [get_ports {seg_data[2]}]
set_property PACKAGE_PIN J1 [get_ports {seg_data[3]}]
set_property PACKAGE_PIN E4 [get_ports {seg_data[4]}]
set_property PACKAGE_PIN E2 [get_ports {seg_data[5]}]
set_property PACKAGE_PIN F5 [get_ports {seg_data[6]}]
set_property PACKAGE_PIN F1 [get_ports {seg_data[7]}]
set_property PACKAGE_PIN K5 [get_ports {seg_com[0]}]
set_property PACKAGE_PIN K3 [get_ports {seg_com[1]}]
set_property PACKAGE_PIN K1 [get_ports {seg_com[2]}]
set_property PACKAGE_PIN L6 [get_ports {seg_com[3]}]
set_property PACKAGE_PIN G3 [get_ports {seg_com[4]}]
set_property PACKAGE_PIN G1 [get_ports {seg_com[5]}]
set_property PACKAGE_PIN H6 [get_ports {seg_com[6]}]
set_property PACKAGE_PIN H4 [get_ports {seg_com[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports *]
create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
set_false_path -from [get_ports rst_p]

```

7.6. Character LCD

아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
# Combo II-DLD S75 / MAIN CLOCK F = 50 MHz.
set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
set_property PACKAGE_PIN K4 [get_ports rst_p]
set_property PACKAGE_PIN A6 [get_ports lcd_e]
set_property PACKAGE_PIN G6 [get_ports lcd_rs]
set_property PACKAGE_PIN D6 [get_ports lcd_rw]
set_property PACKAGE_PIN A4 [get_ports {lcd_data[0]}]
set_property PACKAGE_PIN B2 [get_ports {lcd_data[1]}]
set_property PACKAGE_PIN C3 [get_ports {lcd_data[2]}]
set_property PACKAGE_PIN D4 [get_ports {lcd_data[3]}]
set_property PACKAGE_PIN A2 [get_ports {lcd_data[4]}]
set_property PACKAGE_PIN C5 [get_ports {lcd_data[5]}]
set_property PACKAGE_PIN C1 [get_ports {lcd_data[6]}]
set_property PACKAGE_PIN D1 [get_ports {lcd_data[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports *]
create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
set_false_path -from [get_ports rst_p]

```

7.7. UART ECHO

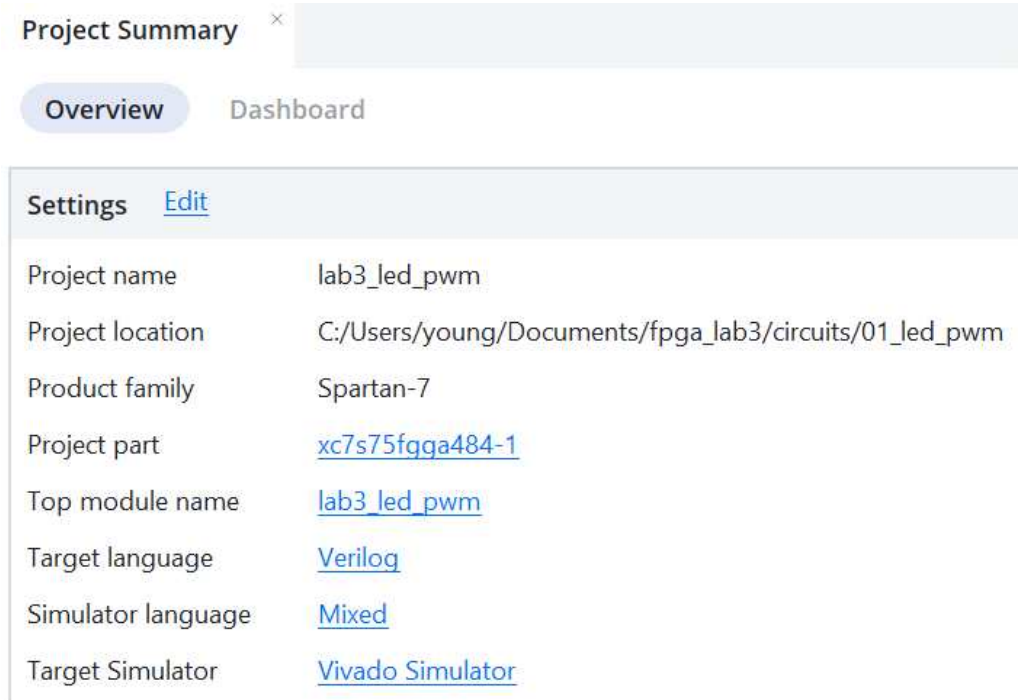
아래와 같이 대상 부품 및 핀 제약을 설정했다.

```
# Combo II-DLD S75 / MAIN CLOCK F = 50 MHz / USB UART 9600 8N1.
set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
set_property PACKAGE_PIN K4 [get_ports rst_p]
set_property PACKAGE_PIN C6 [get_ports uart_rxd]
set_property PACKAGE_PIN F6 [get_ports uart_txd]
set_property PACKAGE_PIN N5 [get_ports {led[0]}]
set_property PACKAGE_PIN M1 [get_ports {led[1]}]
set_property PACKAGE_PIN M3 [get_ports {led[2]}]
set_property PACKAGE_PIN M7 [get_ports {led[3]}]
set_property PACKAGE_PIN N7 [get_ports {led[4]}]
set_property PACKAGE_PIN M2 [get_ports {led[5]}]
set_property PACKAGE_PIN M4 [get_ports {led[6]}]
set_property PACKAGE_PIN L4 [get_ports {led[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports *]
create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
set_false_path -from [get_ports {rst_p uart_rxd}]
```

8. vivado 내의 실험 과정 및 결과

8.1. LED PWM

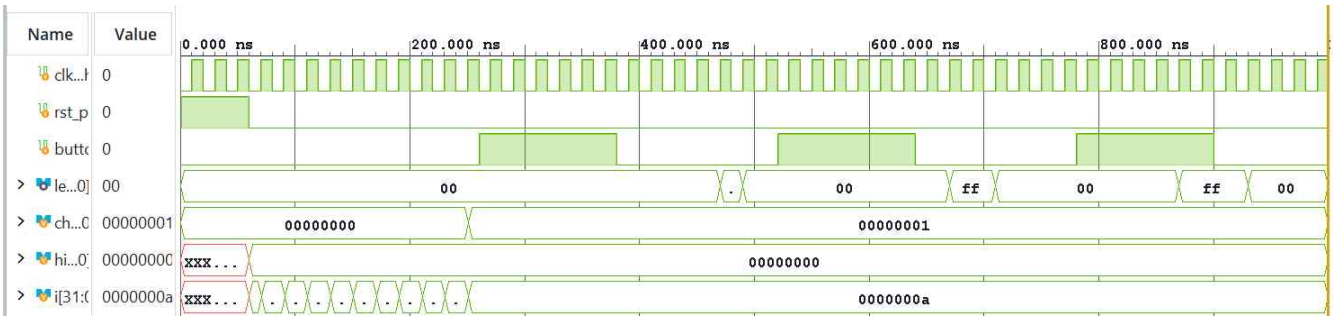
- Vivado 내의 settings는 다음과 같다.



첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서
작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.

이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의
파형이 일치하였기에 vscode에서의 파형이 올바른
파형이라고 할 수 있다.

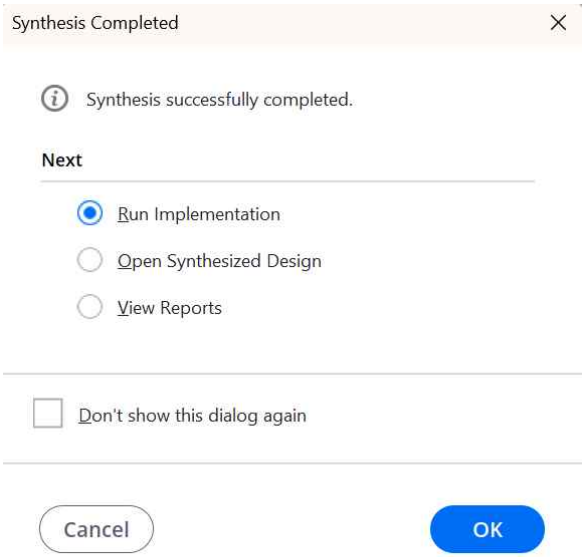
이를 vivado파형과 비교해보면, 정확히 같은 파형의
결과를 얻은 것을 알 수 있다.
즉, 문제 없이 알맞은 파형을 얻었다.

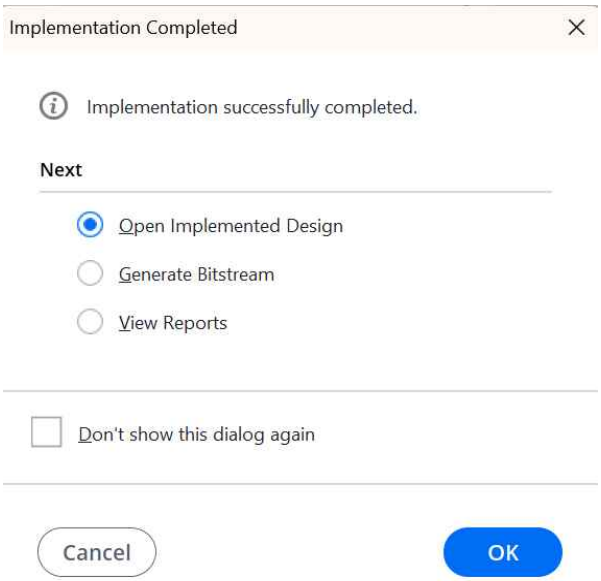
위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와
같은 Tcl Console의 결과값을 얻었다.

```
# run 1000ns
CHECK expected_high=0 observed_high=0 time=251000
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_led_pwm_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:12 . Memory (MB): peak = 1752.586 ; gain = 4.477
```

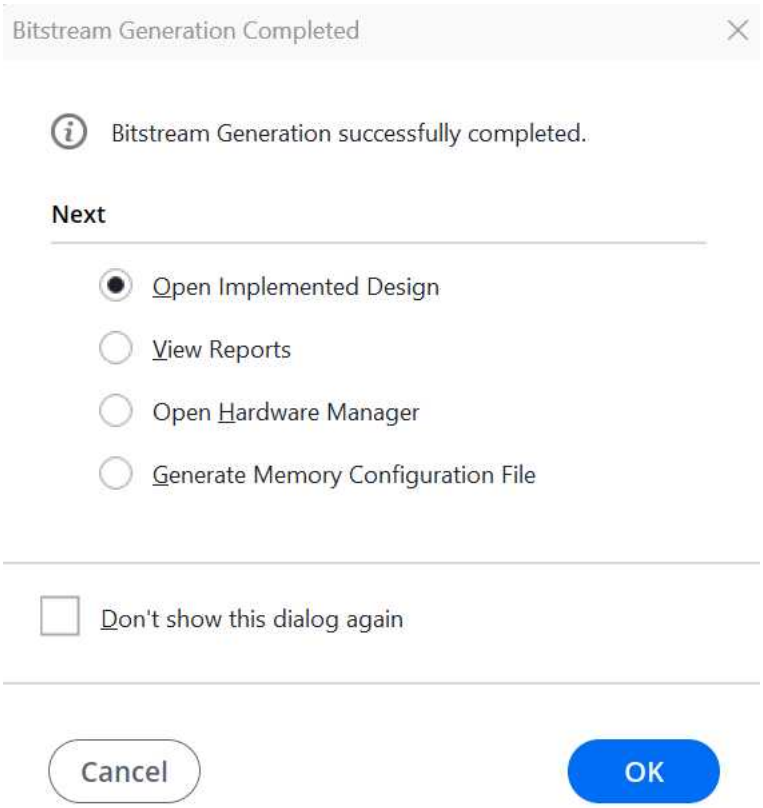
오류 없이 정상적으로 출력되었음을 확인할 수 있다.

이후 Run synthesis와 Run Implementation을 진행하였다.





두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.

Tcl Console | Messages | Log | Reports | Design Runs

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
synth_1	constrs_1	synth_design Complete!												92	45
impl_1	constrs_1	write_bitstream Complete!	8.971	0.000	0.160	0.000		0.000	0.106	0	8 Warn			92	52

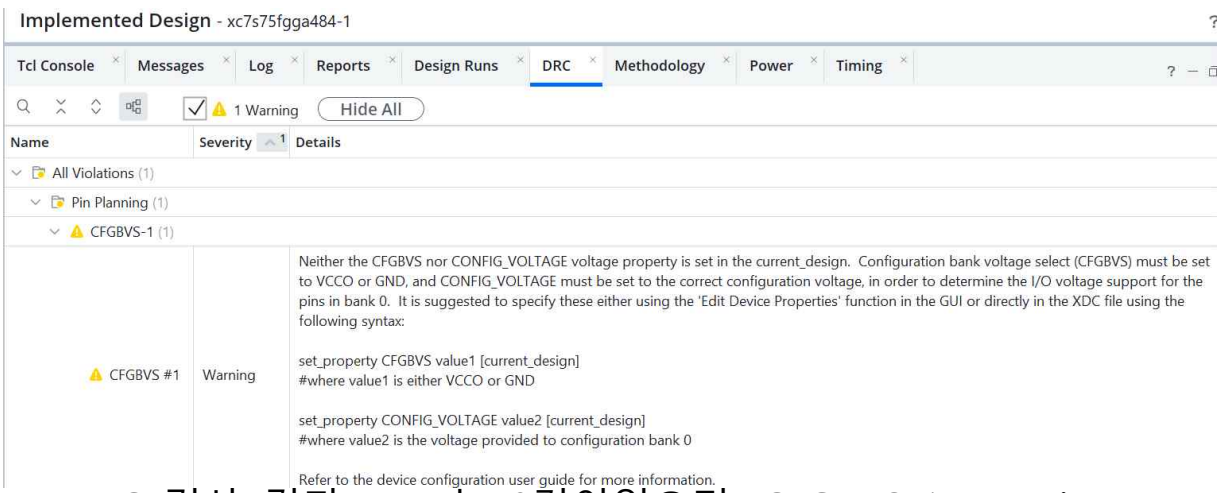
사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:
7ECAF94BFDD3CDF6DDBB890CBA0D89714A941C393711E02D588E6A248A1F38DC

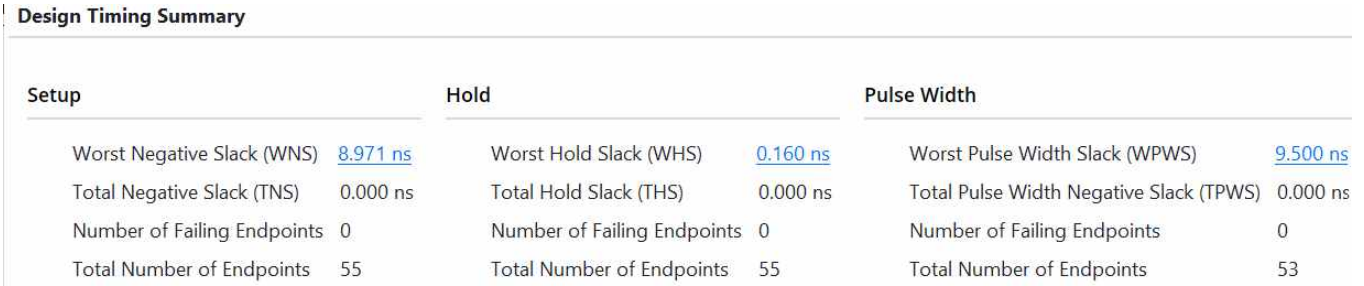
- Bit 경로

Bitstream:
C:\Users\young\Documents\fpga_lab3\circuits\01_led_pwm\lab3_led_pwm.runs\impl_1\lab3_led_pwm.bit



- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.
  해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

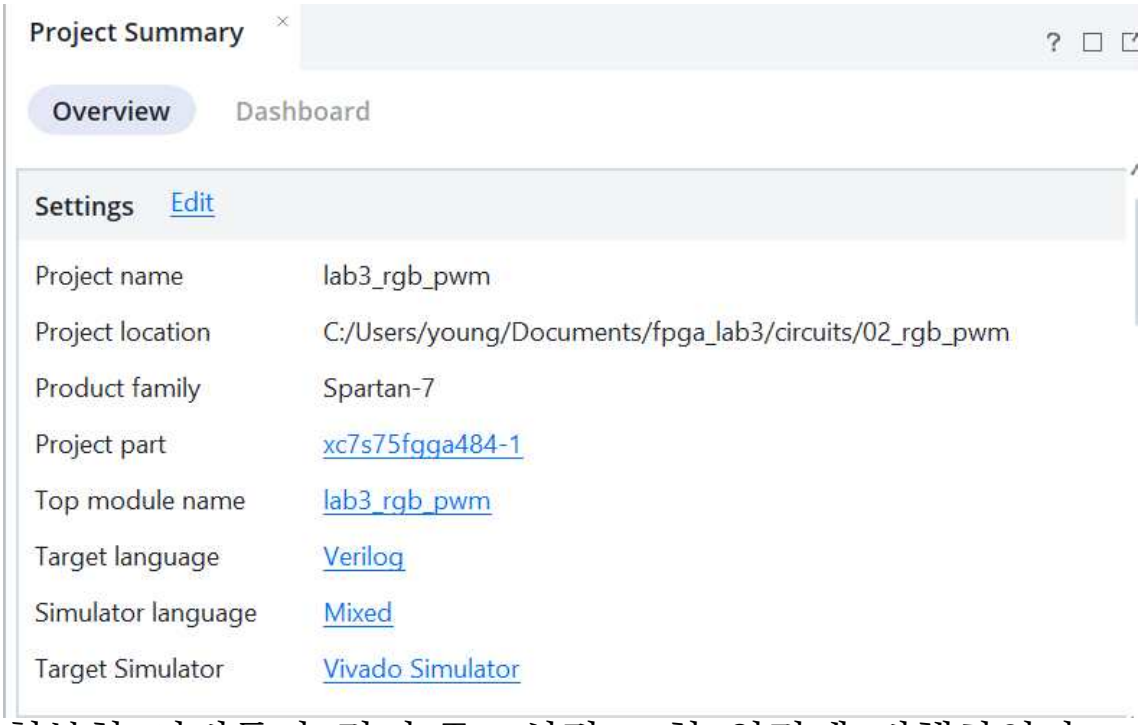
- Timing 결과 검증



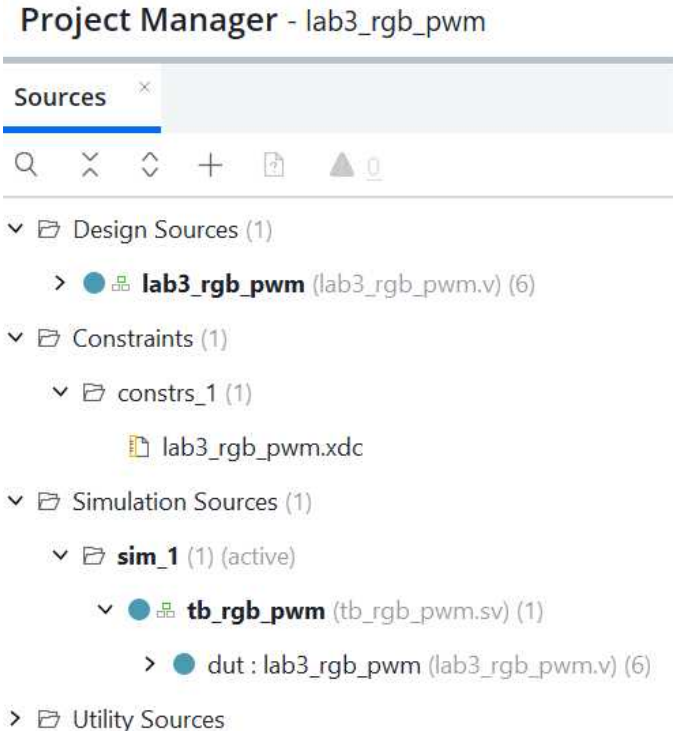
50MHz, 20ns 클록 제약을 적용하여 구현 후 타이밍을 검증하였다. Setup 분석 결과 WNS는 +8.971ns, TNS는 0.000ns였으며 실패 Endpoint는 0개였다. Hold 분석에서는 WHS가 +0.160ns, THS가 0.000ns였으며 실패 Endpoint는 0개였다. Pulse Width 분석 또한 WPWS +9.500ns, TPWS 0.000ns로 확인되었다. 모든 Slack이 0 이상이고 타이밍 위반 경로가 없으므로 본 회로는 주어진 50MHz 클록 조건에서 타이밍 요구사항을 만족한다.

8.2. RGB PWM

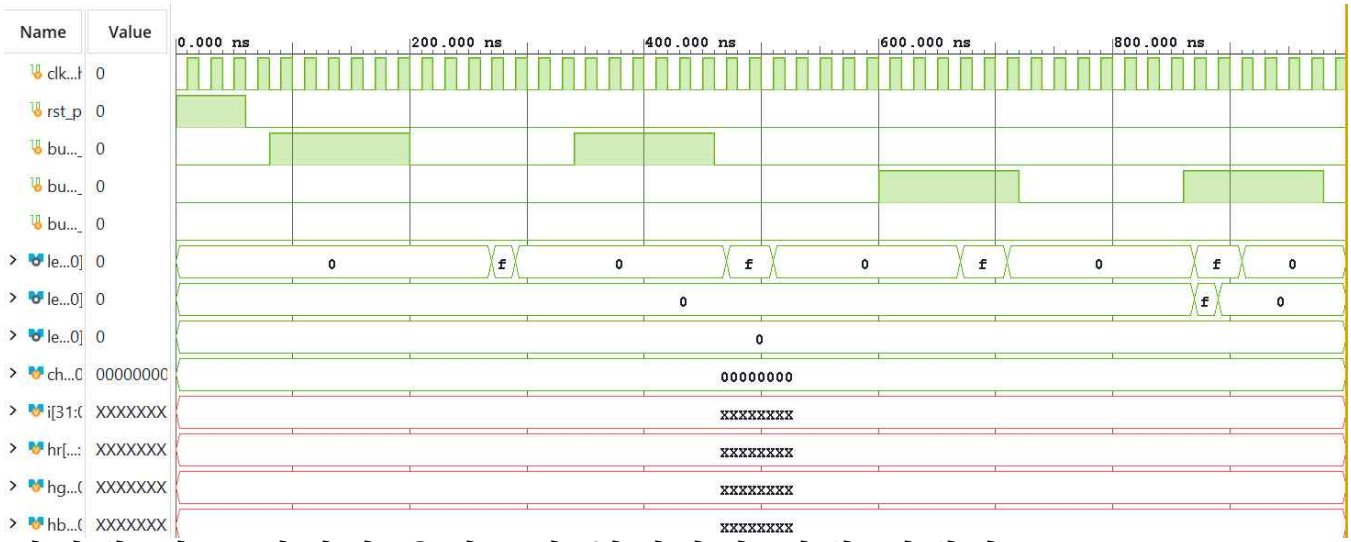
- Vivado 내의 settings는 다음과 같다.



첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서 작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.

이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의 파형이 일치하였기에 vscode에서의 파형이 올바른 파형이라고 할 수 있다.

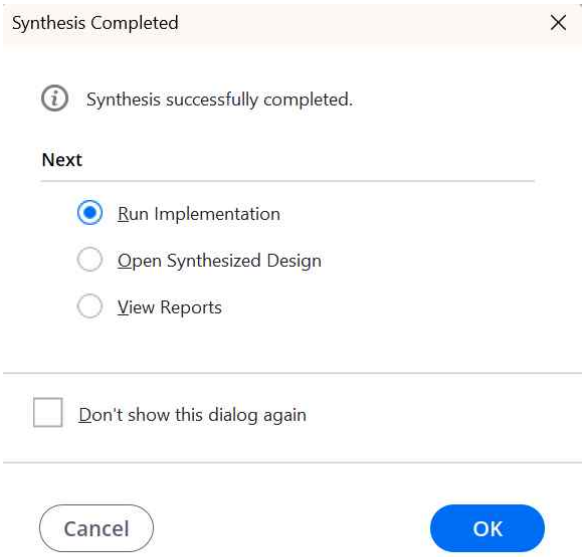
이를 vivado파형과 비교해보면, 정확히 같은 파형의 결과를 얻은 것을 알 수 있다.
즉, 문제 없이 알맞은 파형을 얻었다.

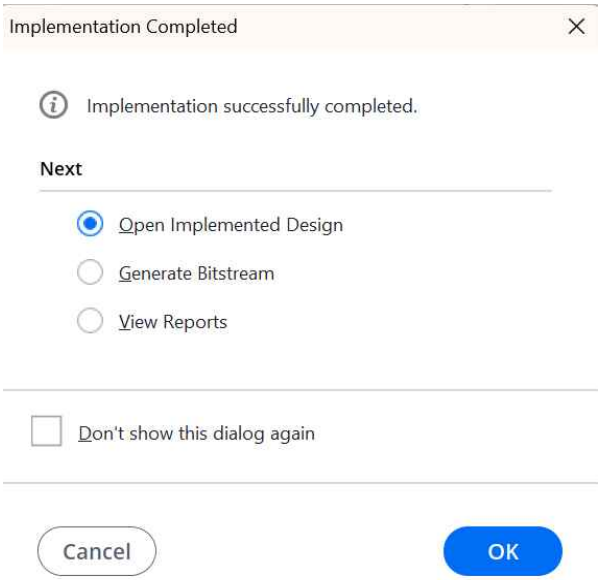
위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

```
source tb_rgb_pwm.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wa
#   }
# }
# run 1000ns
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_rgb_pwm_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:03 ; elapsed = 00:00:06 . Memory (MB): peak = 2932.051 ; gain = 0.000
```

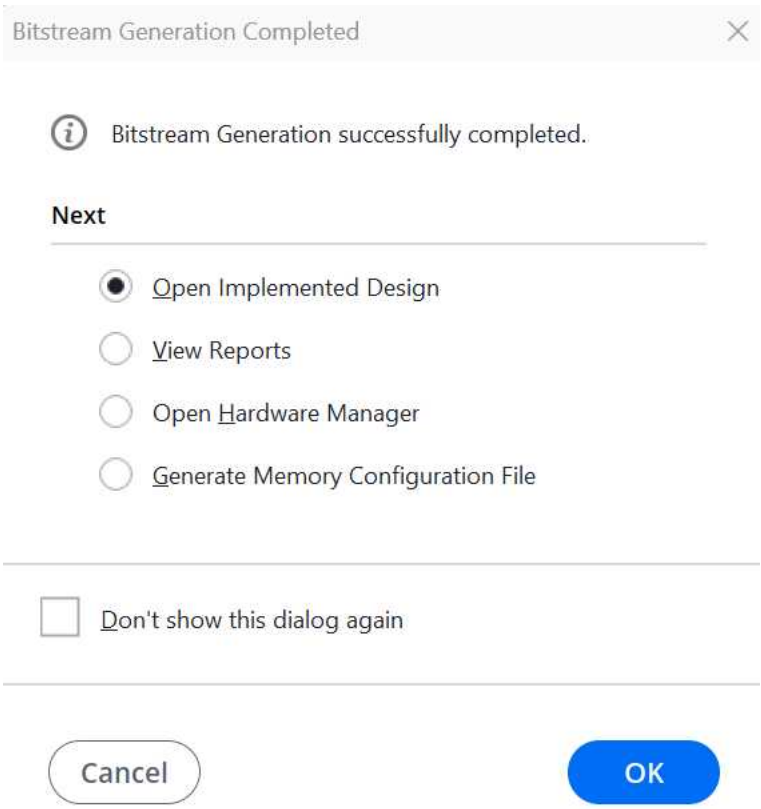
오류 발생 없이 정상적으로 출력을 얻었음을 확인할 수 있다.

이후 Run synthesis와 Run Implementation을 진행하였다.

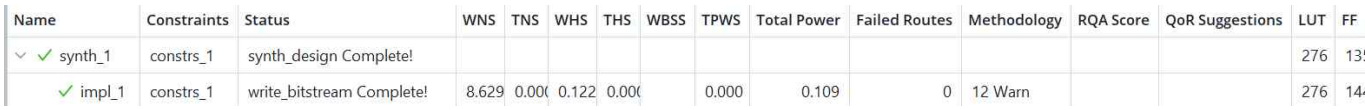




두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.



Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												276	13
✓ impl_1	constrs_1	write_bitstream Complete!	8.629	0.00(	0.122	0.00(		0.000	0.109	0	12 Warn			276	14

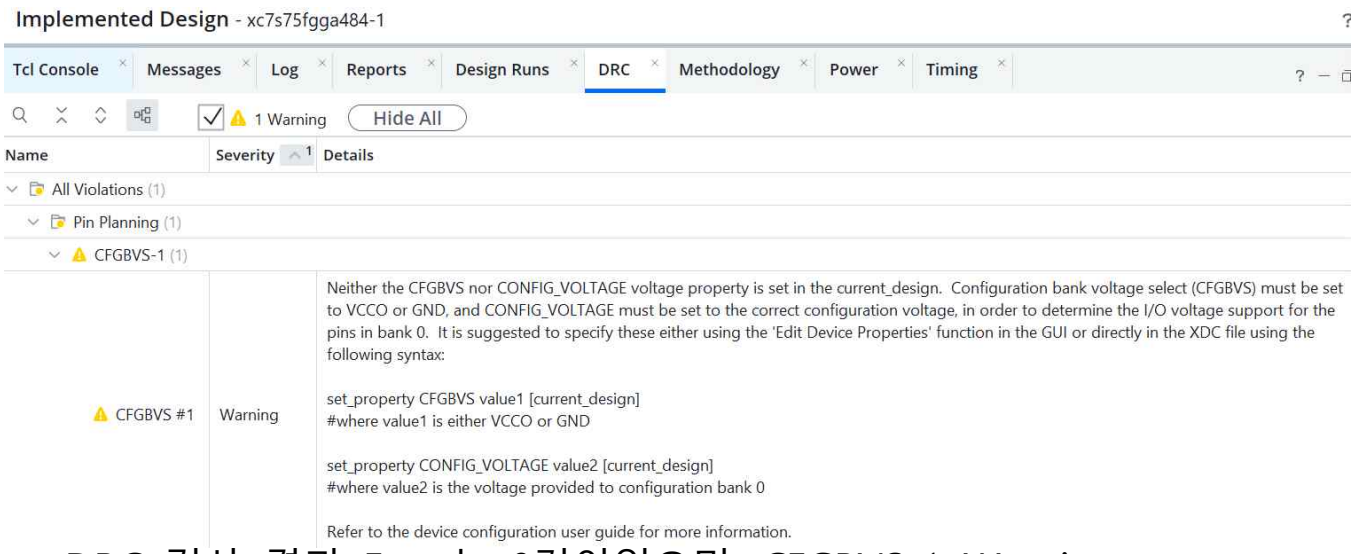
사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:
844EC8FB07304A7107258B32D9190BC698F5C29B8E882C46F2DDABC52CD1C0C6

- Bit 경로

Bitstream:
C:₩Users₩young₩Documents₩fpga_lab3₩circuits₩02_rgb_pwm₩lab3_rgb_pwm.runs₩impl_1₩lab3_rgb_pwm.bit



- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.
  해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

- Timing 결과 검증

Design Timing Summary

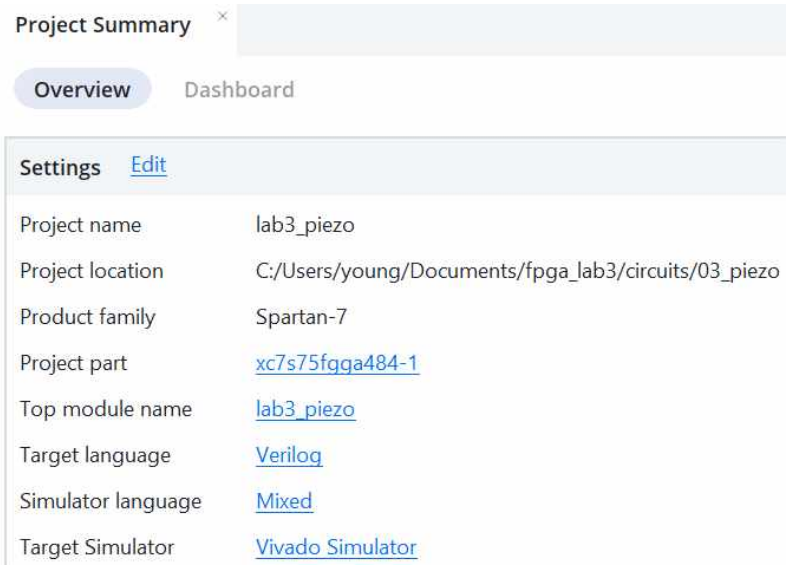
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	8.629 ns	Worst Hold Slack (WHS)	0.122 ns	Worst Pulse Width Slack (WPWS)	9.500 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	153	Total Number of Endpoints	153	Total Number of Endpoints	145

All user specified timing constraints are met.

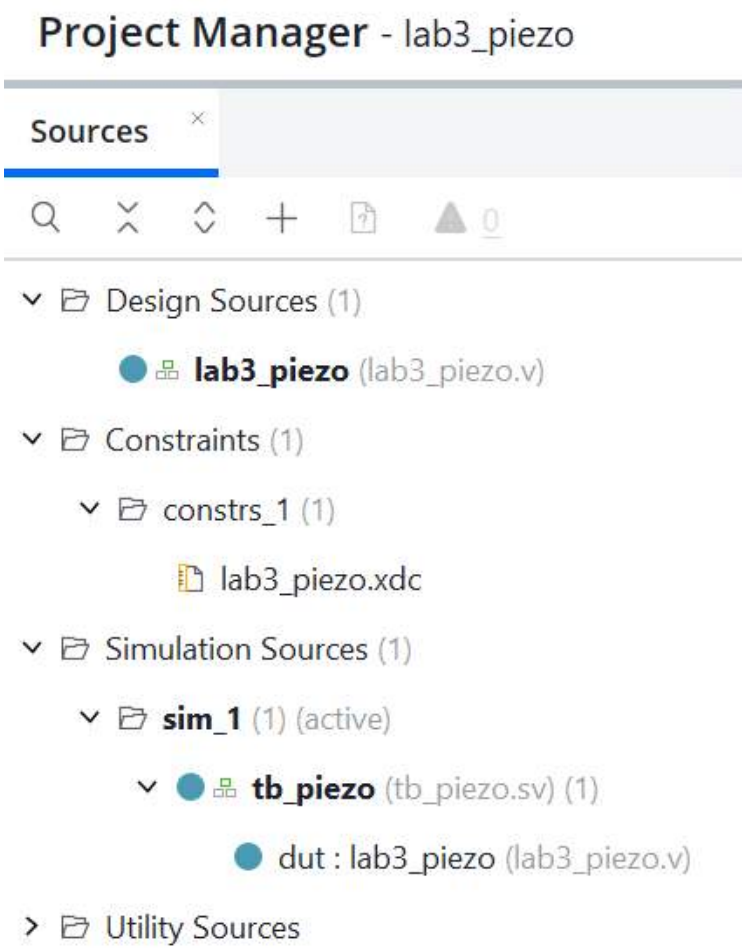
50MHz, 20ns 클록 제약을 적용하여 RGB PWM 회로의 구현 후 타이밍을 검증하였다. Setup 분석 결과 WNS는 +8.629ns, TNS는 0.000ns였으며 153개 Endpoint 중 실패한 Endpoint는 없었다. Hold 분석 결과 WHS는 +0.122ns, THS는 0.000ns였으며 Hold 위반 역시 발생하지 않았다. Pulse Width 분석에서는 WPWS가 +9.500ns, TPWS가 0.000ns로 확인되었다. 모든 사용자 지정 타이밍 제약이 충족되었으므로 본 회로는 50MHz 동작 조건에서 타이밍 요구사항을 만족한다.

8.3. Piezo

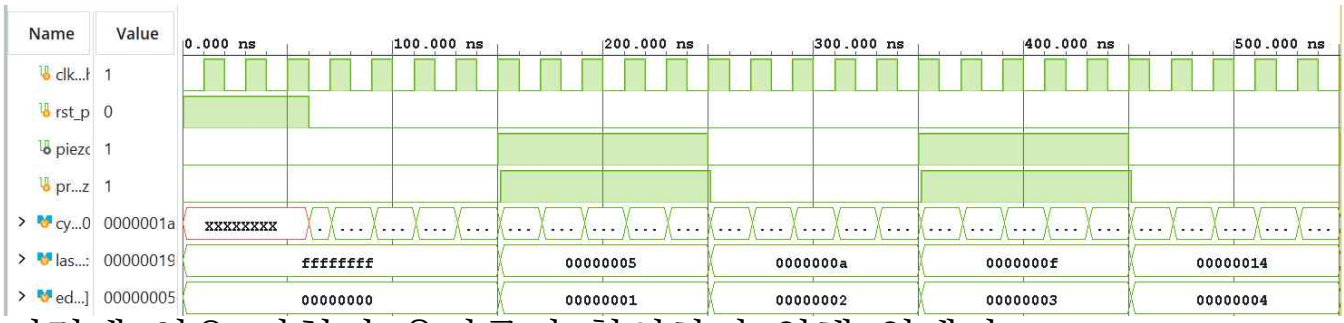
- Vivado 내의 settings는 다음과 같다.



첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서 작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.

이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의 파형이 일치하였기에 vscode에서의 파형이 올바른 파형이라고 할 수 있다.

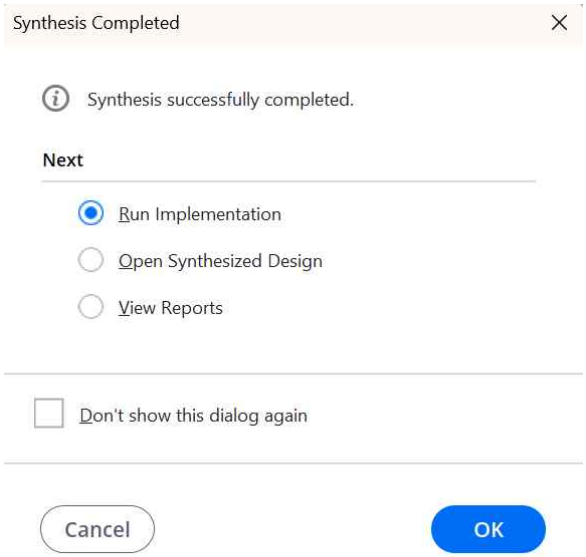
이를 vivado파형과 비교해보면, 정확히 같은 파형의 결과를 얻은 것을 알 수 있다.
즉, 문제 없이 알맞은 파형을 얻었다.

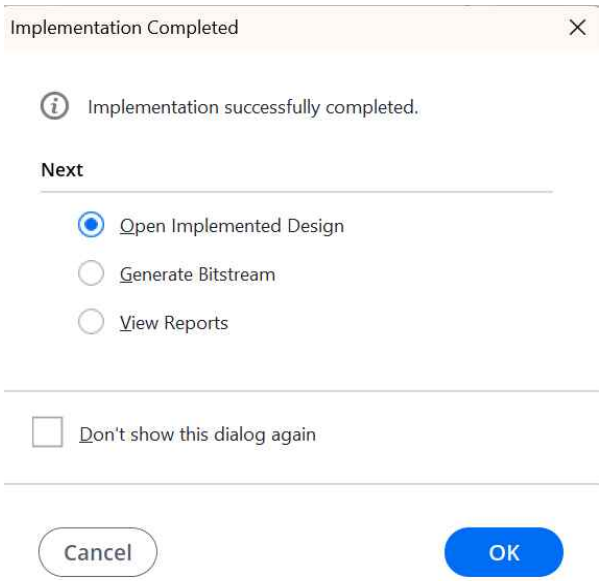
위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

```
source tb_piezo.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wa
#   }
# }
# run 1000ns
EDGE cycle=5 piezo=1 time=151000
EDGE cycle=10 piezo=0 time=251000
EDGE cycle=15 piezo=1 time=351000
EDGE cycle=20 piezo=0 time=451000
EDGE cycle=25 piezo=1 time=551000
LAB3_PIEZO_PASS edges=5
$finish called at time : 551 ns : File "C:/Users/young/Documents/fpga_lab3/circuits/03_piezo/sim/tb_piezo.sv" Line 24
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_piezo_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:03 ; elapsed = 00:00:06 . Memory (MB): peak = 3109.355 ; gain = 0.000
```

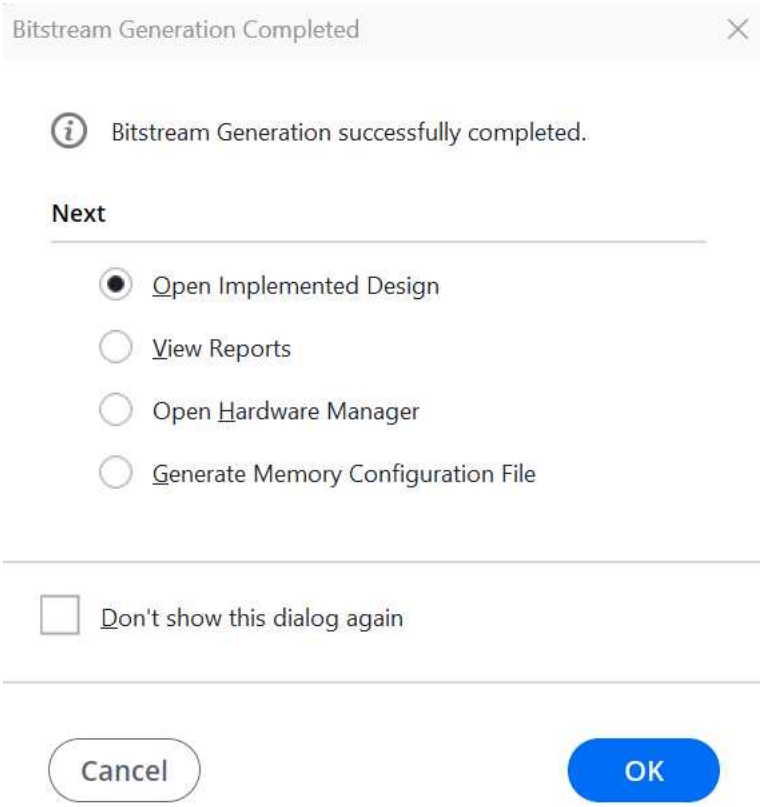
그 결과 정상적으로 출력을 내놓았음을 알 수 있다.

이후 Run synthesis와 Run Implementation을 진행하였다.

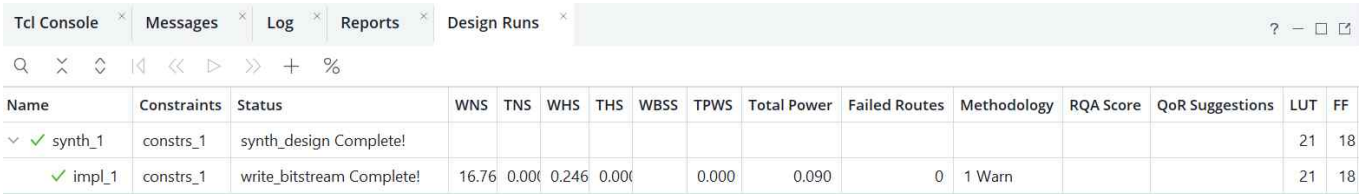




두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.



Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
synth_1	constrs_1	synth_design Complete!												21	18
impl_1	constrs_1	write_bitstream Complete!	16.76	0.000	0.246	0.000		0.000	0.090	0	1 Warn			21	18

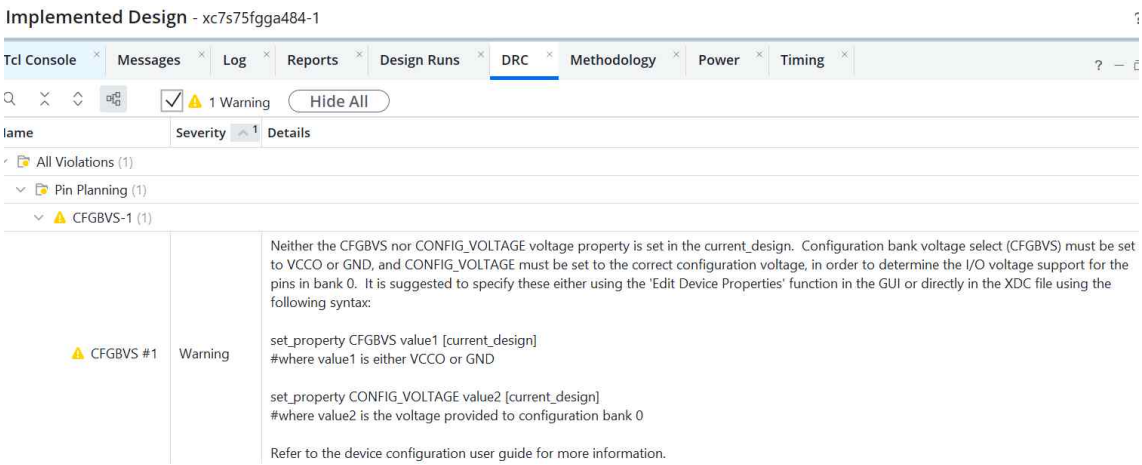
사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:
9A68C191717BFC72718903EDCF6D01B50C22F9D308DD234B506EDA44892B4300

- Bit 경로

Bitstream:
C:₩Users₩young₩Documents₩fpga_lab3₩circuits₩03_piezo₩lab3_piezo.runs₩impl_1₩lab3_piezo.bit



- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.
  해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

- Timing 결과 검증

Design Timing Summary

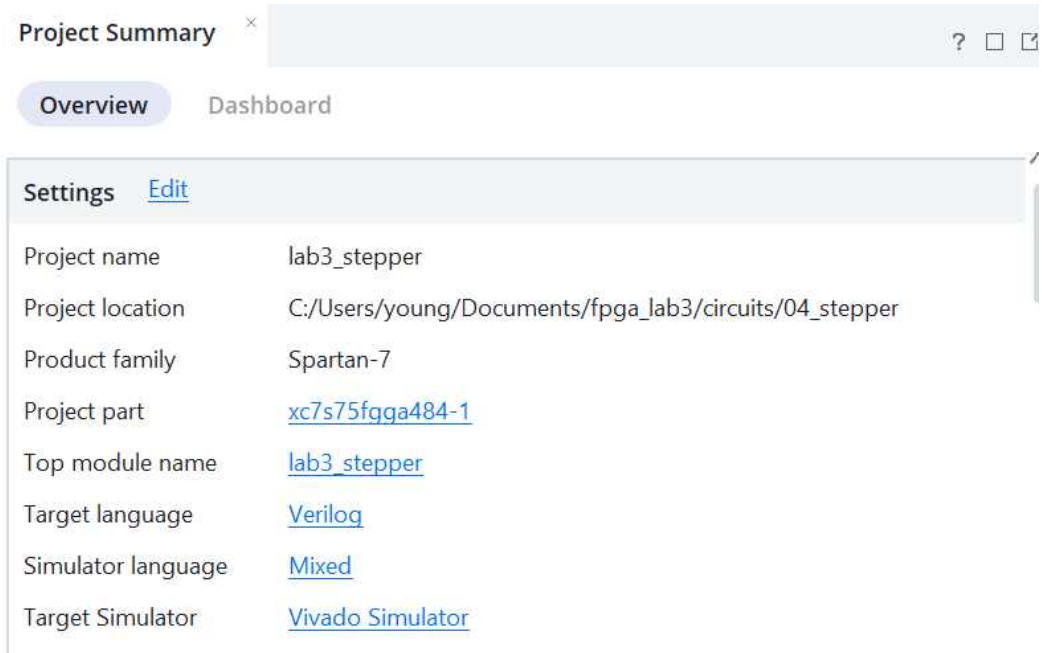
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	16.761 ns	Worst Hold Slack (WHS)	0.246 ns	Worst Pulse Width Slack (WPWS)	9.500 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	18	Total Number of Endpoints	18	Total Number of Endpoints	19

All user specified timing constraints are met.

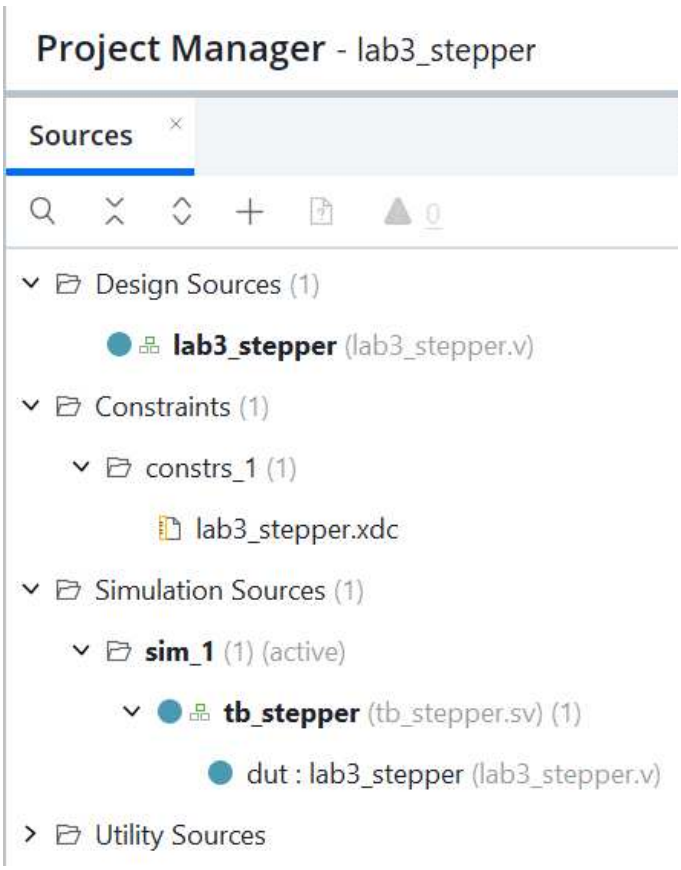
50MHz, 20ns 클록 제약을 적용하여 피에조 회로의 구현 후 타이밍을 검증하였다. Setup 분석 결과 WNS는 +16.761ns, TNS는 0.000ns였으며 18개 Endpoint 중 실패한 Endpoint는 없었다. Hold 분석 결과 WHS는 +0.246ns, THS는 0.000ns였으며 Hold 위반 역시 발생하지 않았다. Pulse Width 분석에서는 WPWS가 +9.500ns, TPWS가 0.000ns로 확인되었다. 모든 사용자 지정 타이밍 제약이 충족되었으므로 본 회로는 50MHz 클록 조건에서 안정적으로 동작할 수 있다.

8.4. Stepper

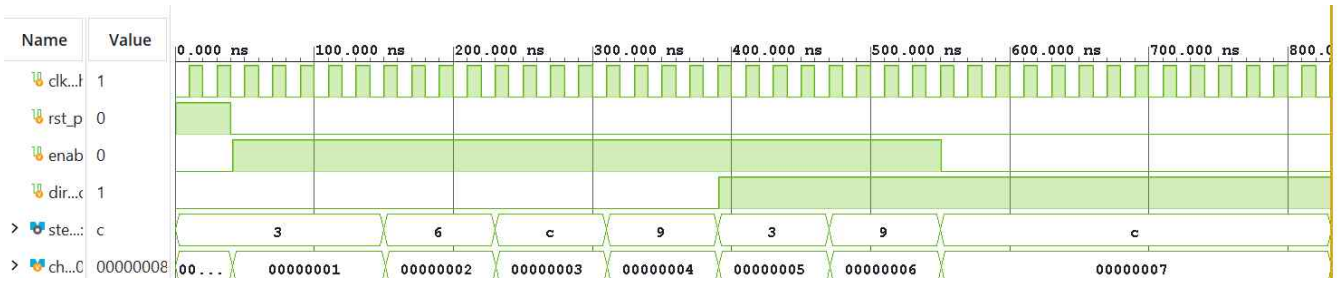
- Vivado 내의 settings는 다음과 같다.



첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서
작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.

이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의
파형이 일치하였기에 vscode에서의 파형이 올바른
파형이라고 할 수 있다.

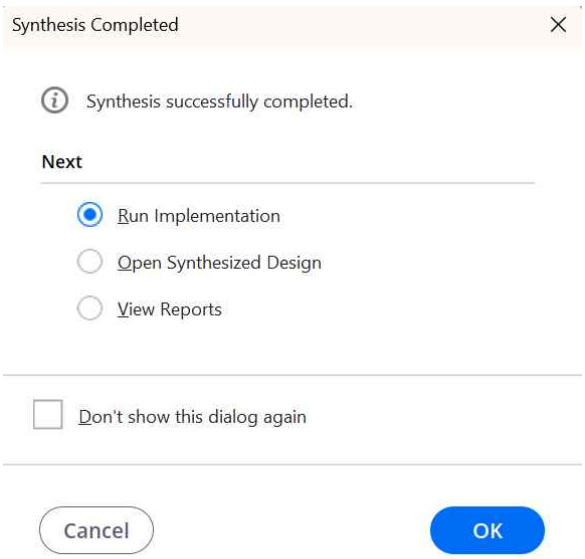
이를 vivado파형과 비교해보면, 정확히 같은 파형의
결과를 얻은 것을 알 수 있다.
즉, 문제 없이 알맞은 파형을 얻었다.

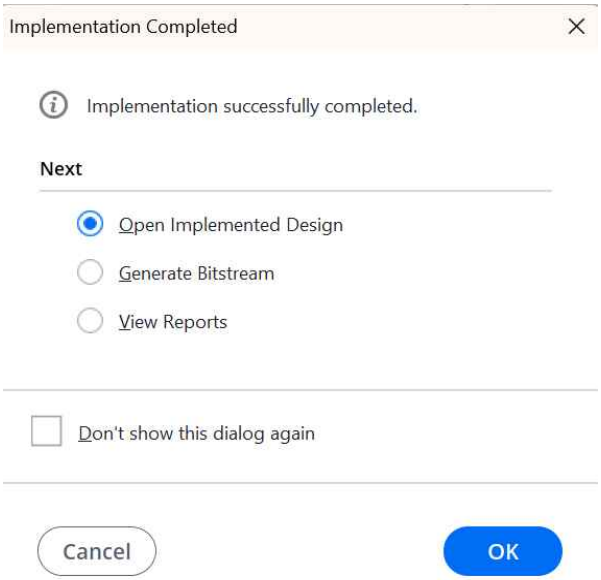
위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

```
source tb_stepper.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wav
#   }
# }
# run 1000ns
CHECK number=1 expected=0011 observed=0011 time=41000
CHECK number=2 expected=0110 observed=0110 time=151000
CHECK number=3 expected=1100 observed=1100 time=231000
CHECK number=4 expected=1001 observed=1001 time=311000
CHECK number=5 expected=0011 observed=0011 time=391000
CHECK number=6 expected=1001 observed=1001 time=471000
CHECK number=7 expected=1100 observed=1100 time=551000
CHECK number=8 expected=1100 observed=1100 time=831000
LAB3_STEPPER_PASS checks=8
$finish called at time : 831 ns : File "C:/Users/young/Documents/fpga_lab3/circuits/04_stepper/sim/tb_stepper.sv" Line 22
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_stepper_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:01 ; elapsed = 00:00:06 . Memory (MB): peak = 3924.676 ; gain = 0.000
```

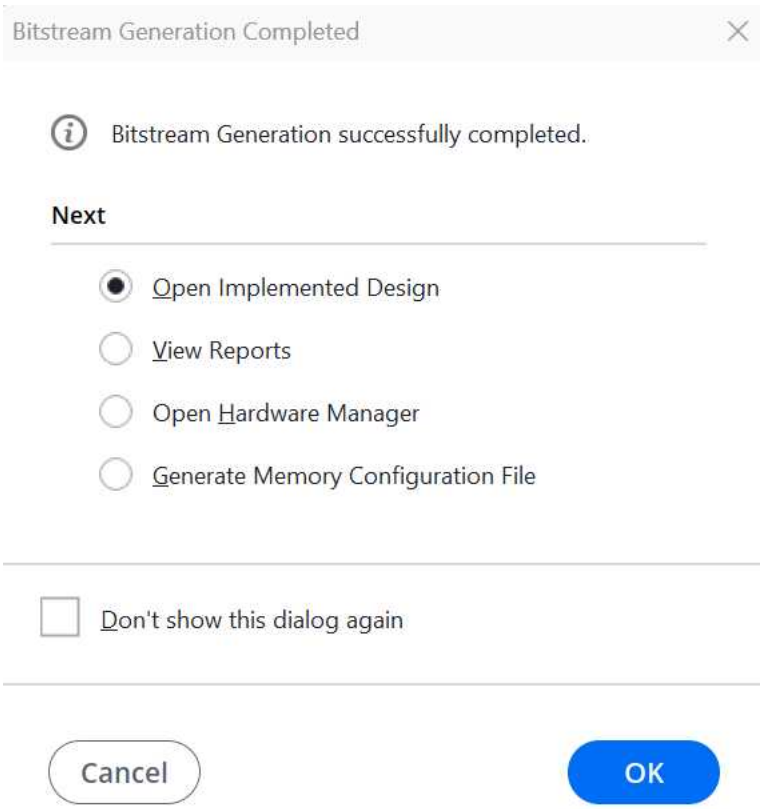
이번에도 마찬가지로 오류 없이 시뮬레이션이 진행되었음을 알 수 있다.

이후 Run synthesis와 Run Implementation을 진행하였다.

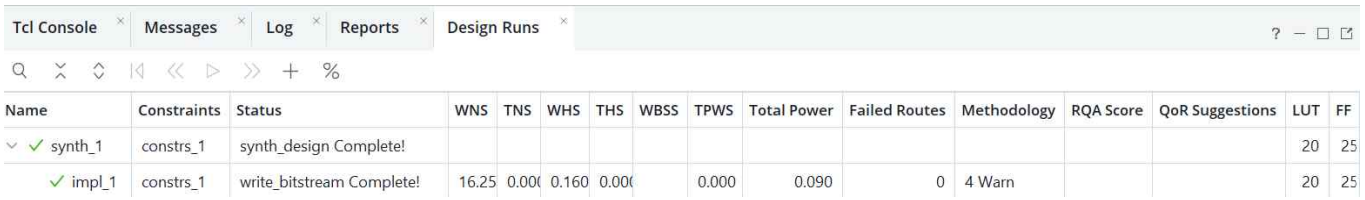




두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.



Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												20	25
✓ impl_1	constrs_1	write_bitstream Complete!	16.25	0.00(	0.160	0.00(		0.000	0.090	0	4 Warn			20	25

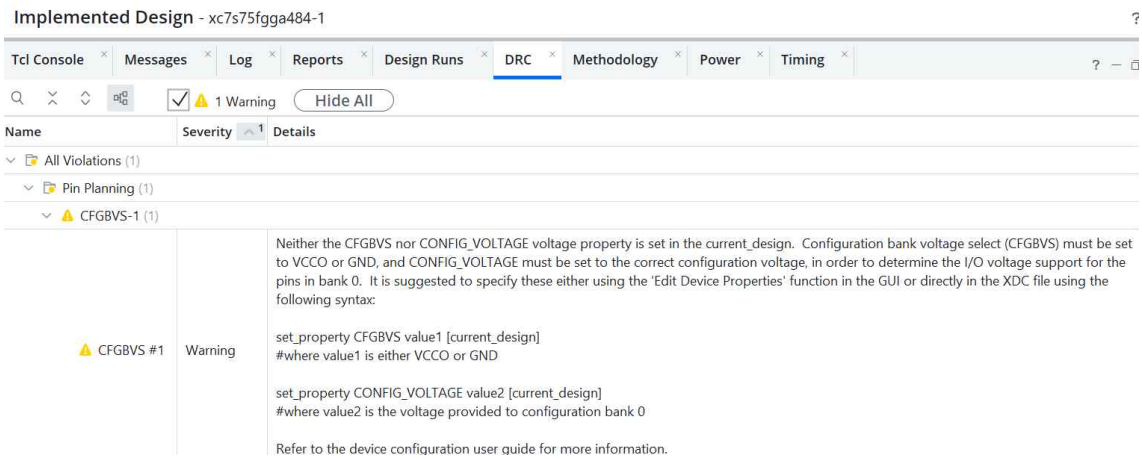
사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:
C54C3A7EB0CDFDACBB03AB84DA420A07DABB3AC7ADF057CB625B9DEE1E6175B0

- Bit 경로

C:\Users\young\Documents\fpga_lab3\circuits\04_stepper\lab3_stepper.runs\impl_1\lab3_stepper.bit



- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.
  해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

- Timing 결과 검증

Design Timing Summary

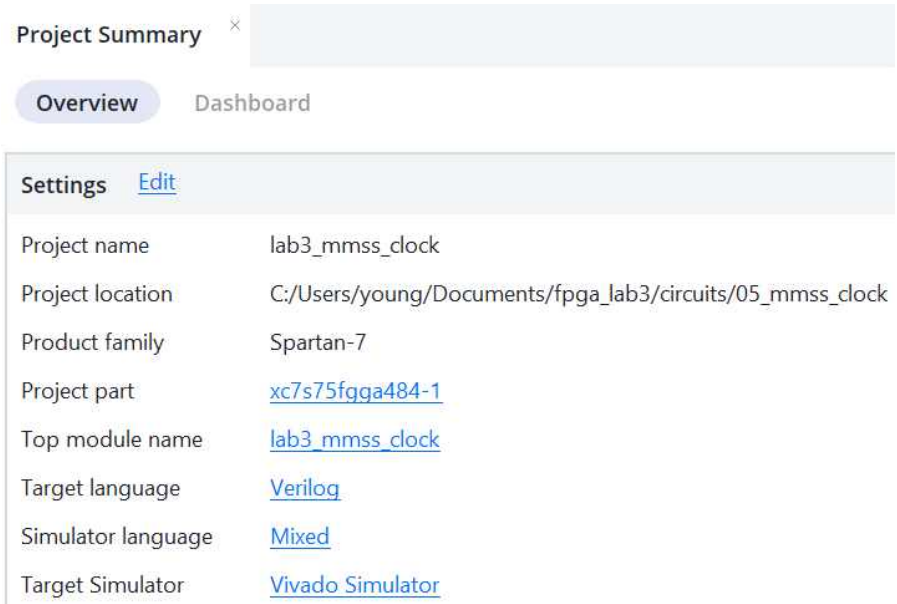
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	16.252 ns	Worst Hold Slack (WHS)	0.160 ns	Worst Pulse Width Slack (WPWS)	9.500 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	23	Total Number of Endpoints	23	Total Number of Endpoints	26

All user specified timing constraints are met.

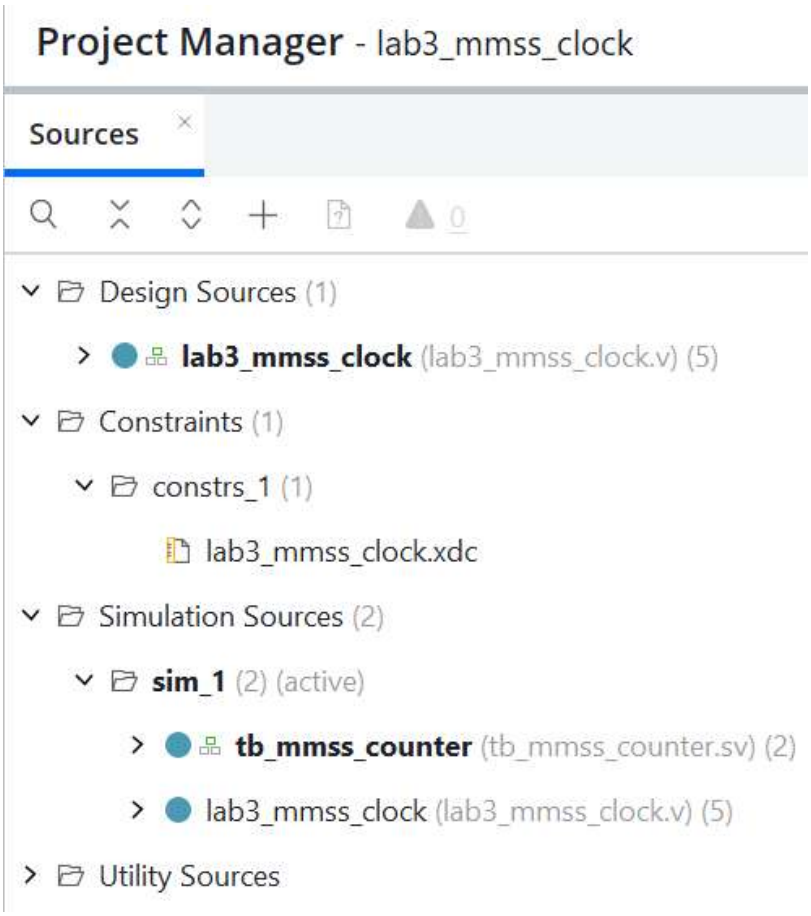
50MHz, 20ns 클록 제약을 적용하여 스텝모터 제어 회로의 구현 후 타이밍을 검증하였다. Setup 분석 결과 WNS는 +16.252ns, TNS는 0.000ns였으며 23개 Endpoint 중 실패한 Endpoint는 없었다. Hold 분석 결과 WHS는 +0.160ns, THS는 0.000ns였으며 Hold 위반도 발생하지 않았다. Pulse Width 분석에서는 WPWS가 +9.500ns, TPWS가 0.000ns로 확인되었다. 모든 사용자 지정 타이밍 제약을 충족하므로 본 회로는 50MHz 클록 조건에서 정상적으로 동작할 수 있다.

8.5. MMSS Clock

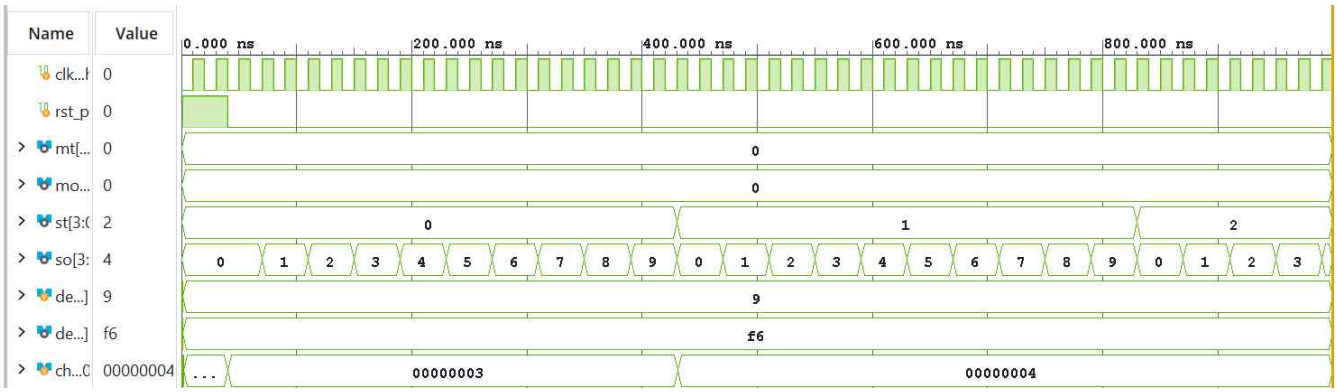
- Vivado 내의 settings는 다음과 같다.



첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서
작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.

이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의
파형이 일치하였기에 vscode에서의 파형이 올바른
파형이라고 할 수 있다.

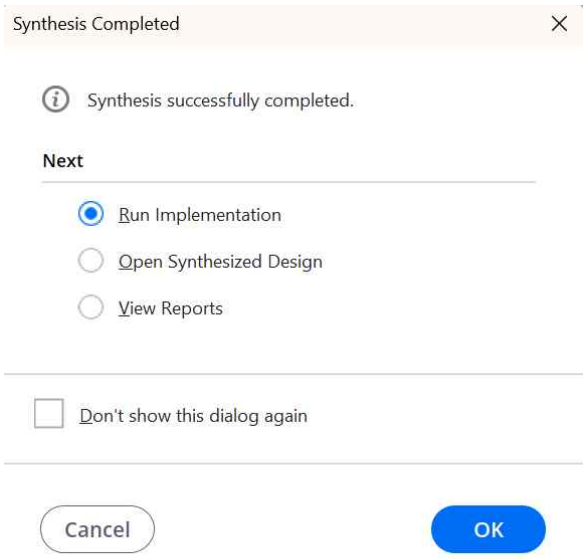
이를 vivado파형과 비교해보면, 정확히 같은 파형의
결과를 얻은 것을 알 수 있다.

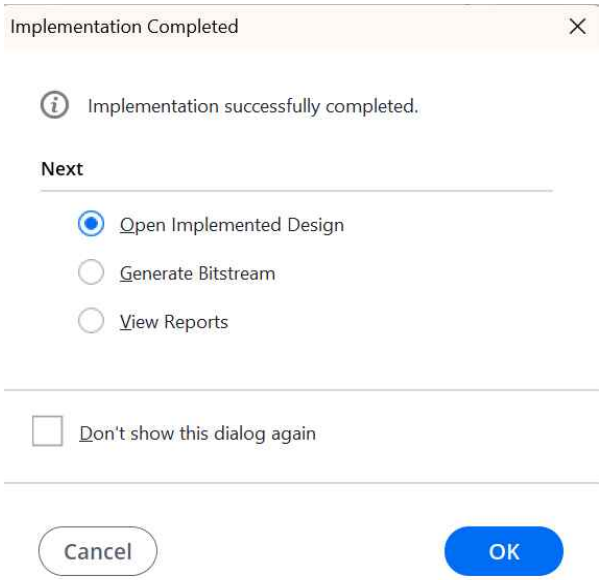
즉, 문제 없이 알맞은 파형을 얻었다.
위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와
같은 Tcl Console의 결과값을 얻었다.

```
source tb_mmss_counter.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wav
#   }
# }
# run 1000ns
CHECK number=3 clock=00:00 time=40000
CHECK number=4 clock=00:10 time=431000
xsim: Time (s): cpu = 00:00:06 ; elapsed = 00:00:06 . Memory (MB): peak = 3924.676 ; gain = 0.000
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_mmss_counter_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:06 ; elapsed = 00:00:09 . Memory (MB): peak = 3924.676 ; gain = 0.000
```

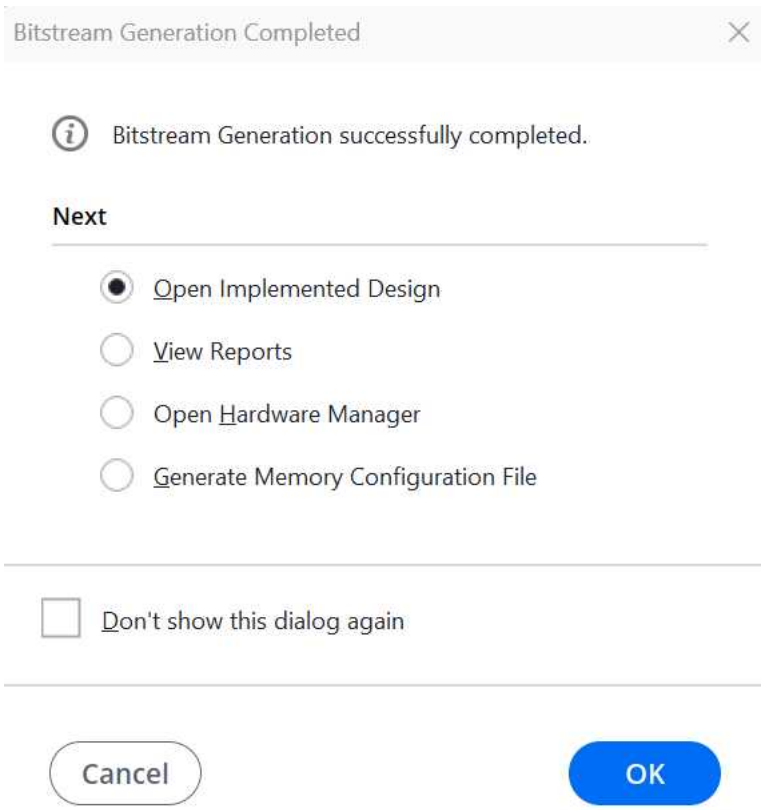
마찬가지로 오류없이 알맞은 결과를 얻었다.

이후 Run synthesis와 Run Implementation을 진행하였다.

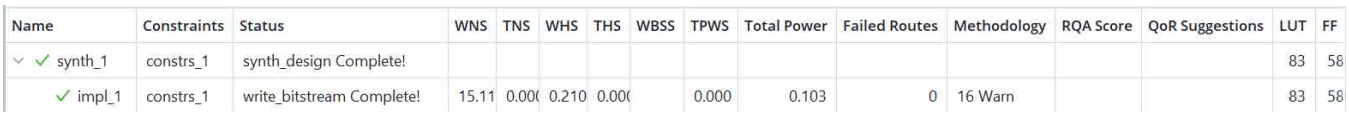




두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.



Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												83	58
✓ impl_1	constrs_1	write_bitstream Complete!	15.11	0.000	0.210	0.000		0.000	0.103	0	16 Warn			83	58

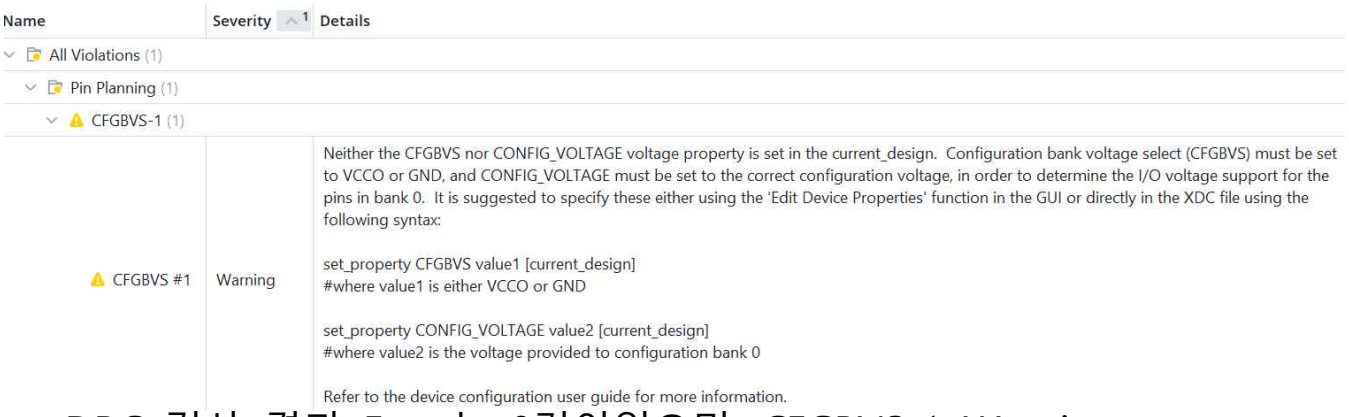
사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:
0BC3B9F3034C8ECDAC543BC04A4CFFC363A48325A2B796182C647AC3792483EF

- Bit 경로

C:\Users\young\Documents\fpga_lab3\circuits\05_mmss_clock\lab3_mmss_clock.runs\impl_1\lab3_mmss_clock.bit



Name	Severity	Details
All Violations (1)		
Pin Planning (1)		
CFGBVS-1 (1)		
CFGBVS #1	Warning	Neither the CFGBVS nor CONFIG_VOLTAGE voltage property is set in the current_design. Configuration bank voltage select (CFGBVS) must be set to VCCO or GND, and CONFIG_VOLTAGE must be set to the correct configuration voltage, in order to determine the I/O voltage support for the pins in bank 0. It is suggested to specify these either using the 'Edit Device Properties' function in the GUI or directly in the XDC file using the following syntax: set_property CFGBVS value1 [current_design] #where value1 is either VCCO or GND set_property CONFIG_VOLTAGE value2 [current_design] #where value2 is the voltage provided to configuration bank 0 Refer to the device configuration user guide for more information.

- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.
  해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

- Timing 결과 검증

Design Timing Summary

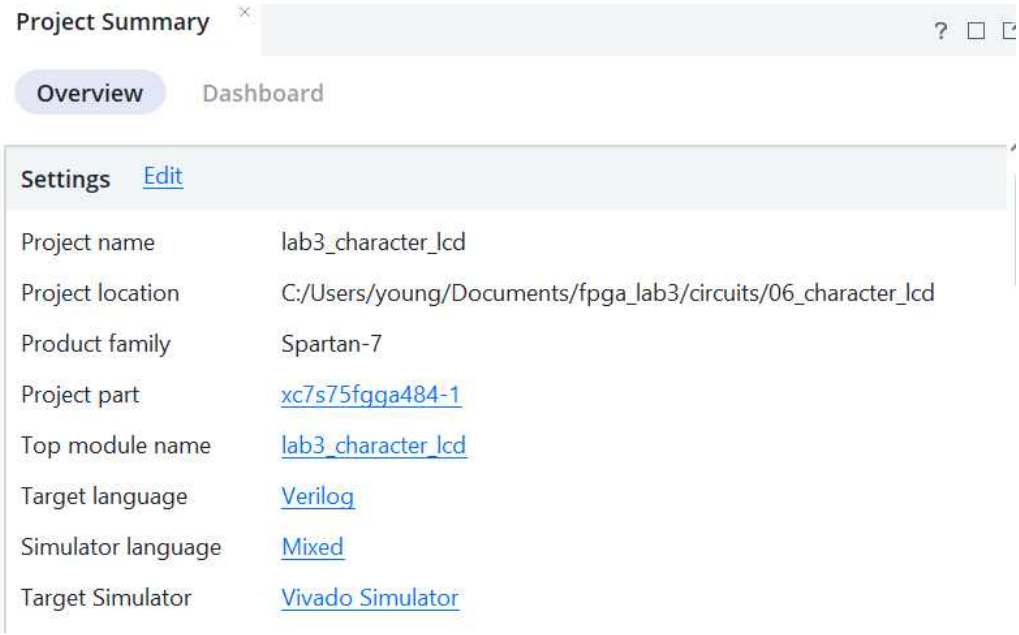
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	15.118 ns	Worst Hold Slack (WHS)	0.210 ns	Worst Pulse Width Slack (WPWS)	9.500 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	74	Total Number of Endpoints	74	Total Number of Endpoints	59

All user specified timing constraints are met.

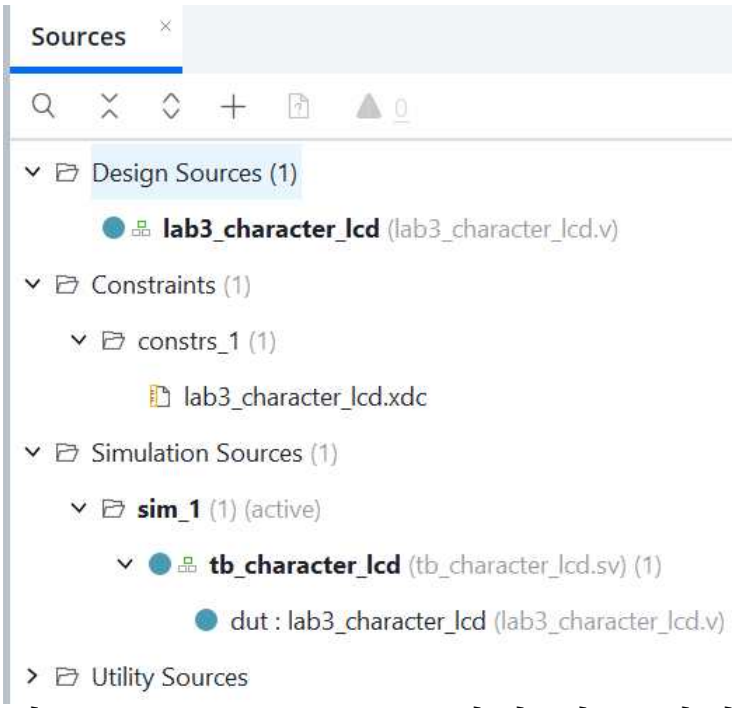
50MHz, 20ns 클록 제약을 적용하여 MM:SS 시계 회로의 구현 후 타이밍을 검증하였다. Setup 분석 결과 WNS는 +15.118ns, TNS는 0.000ns였으며 74개 Endpoint 중 실패한 Endpoint는 없었다. Hold 분석 결과 WHS는 +0.210ns, THS는 0.000ns였으며 Hold 위반도 발생하지 않았다. Pulse Width 분석에서는 WPWS가 +9.500ns, TPWS가 0.000ns로 확인되었다. 모든 사용자 지정 타이밍 제약이 충족되었으므로 본 회로는 50MHz 클록 조건에서 정상적으로 동작할 수 있다.

8.6. Character LCD

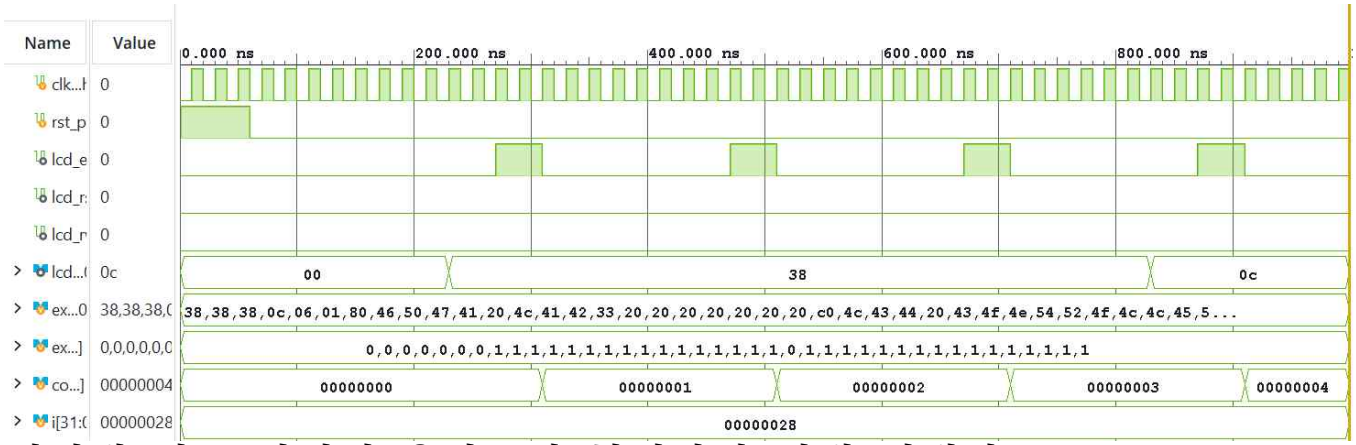
- Vivado 내의 settings는 다음과 같다.



첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서 작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.

이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의 파형이 일치하였기에 vscode에서의 파형이 올바른 파형이라고 할 수 있다.
이를 vivado파형과 비교해보면, 정확히 같은 파형의 결과를 얻은 것을 알 수 있다.

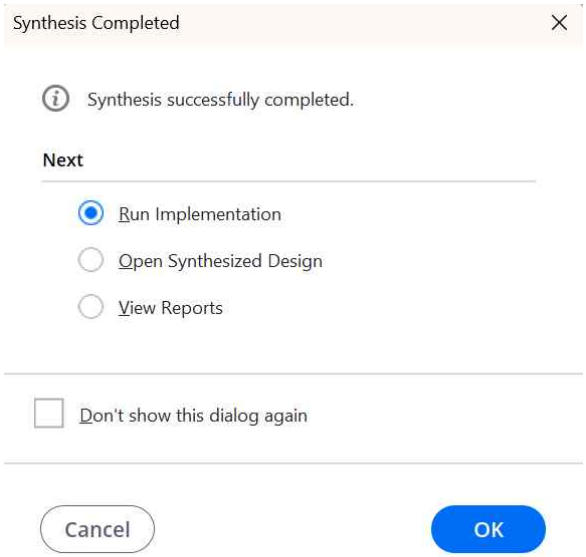
즉, 문제 없이 알맞은 파형을 얻었다.

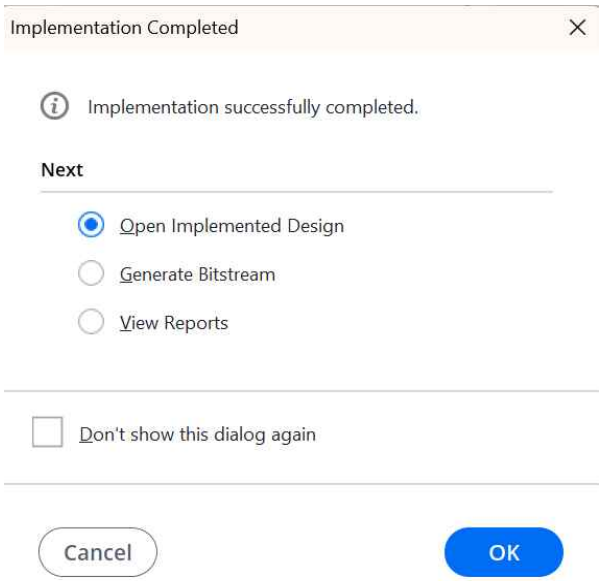
위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

```
source tb_character_lcd.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wav
#   }
# }
# run 1000ns
BYTE index=0 rs=0 data=38 time=310000
BYTE index=1 rs=0 data=38 time=510000
BYTE index=2 rs=0 data=38 time=710000
BYTE index=3 rs=0 data=0c time=910000
xsim: Time (s): cpu = 00:00:05 ; elapsed = 00:00:06 . Memory (MB): peak = 3924.676 ; gain = 0.000
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_character_lcd_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:05 ; elapsed = 00:00:09 . Memory (MB): peak = 3924.676 ; gain = 0.000
```

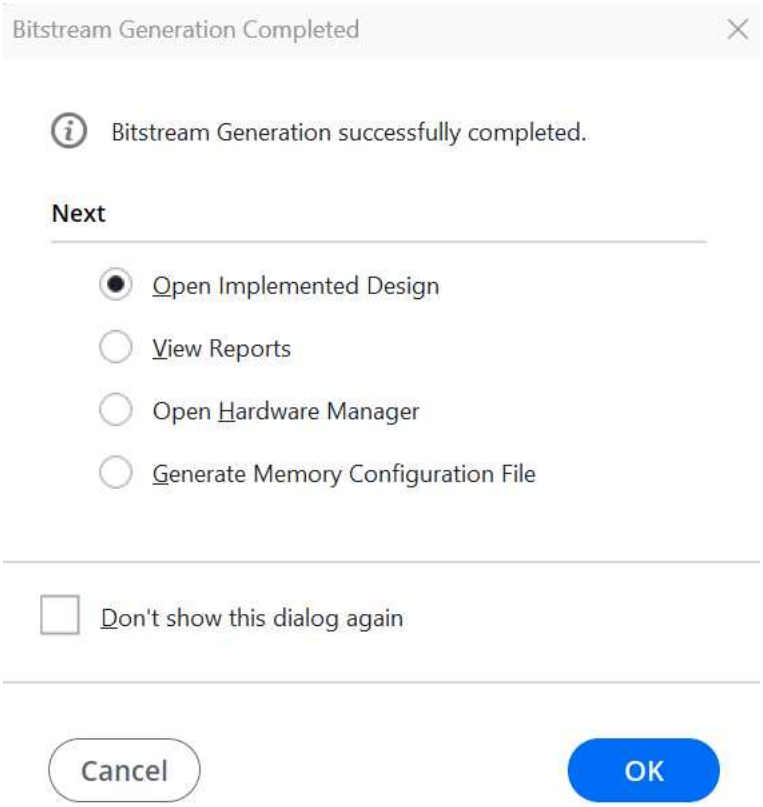
그 결과, 정상적으로 오류 없이 출력되었음을 확인할 수 있다.

이후 Run synthesis와 Run Implementation을 진행하였다.





두 결과 모두 성공하였고, 이후 generate bitstream을 진행하였다.



최종적으로 아래와 같은 결과를 오류 없이 얻었다.

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												112	93
✓ impl_1	constrs_1	write_bitstream Complete!	14.22	0.000	0.167	0.000		0.000	0.103	0	10 Warn			111	93

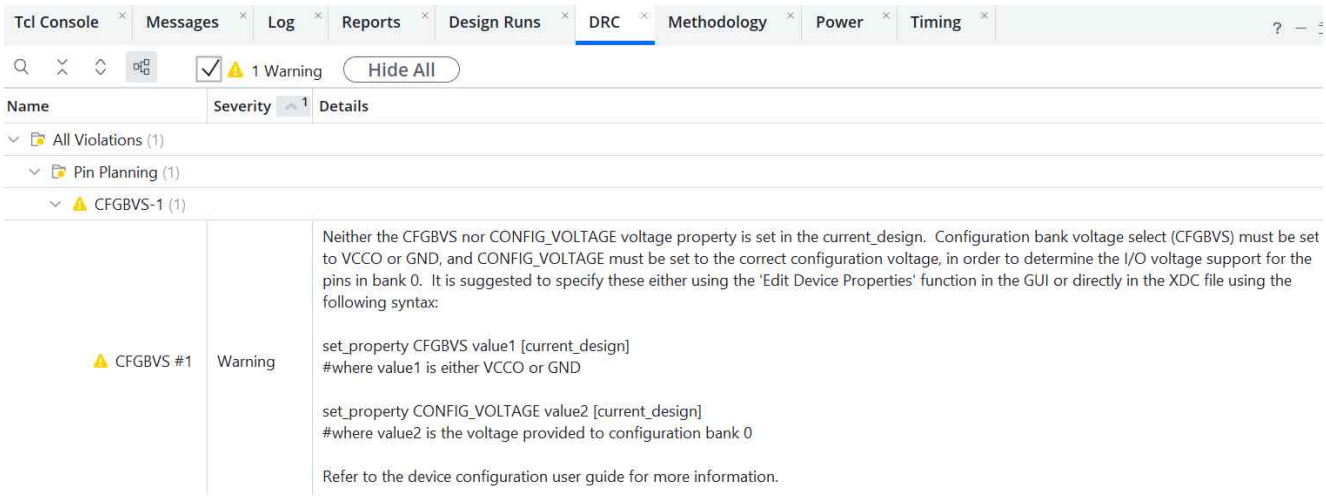
사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:
6C7BEC0E200F161422EB43A3095531CB4A3BB726A53B11AD9BA4E2F1DBF89573

- Bit 경로

C:\Users\young\Documents\fpga_lab3\circuits\06_character_lcd\lab3_character_lcd.runs\impl_1\lab3_character_lcd.bit



- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.
  해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

- Timing 결과 검증

Design Timing Summary

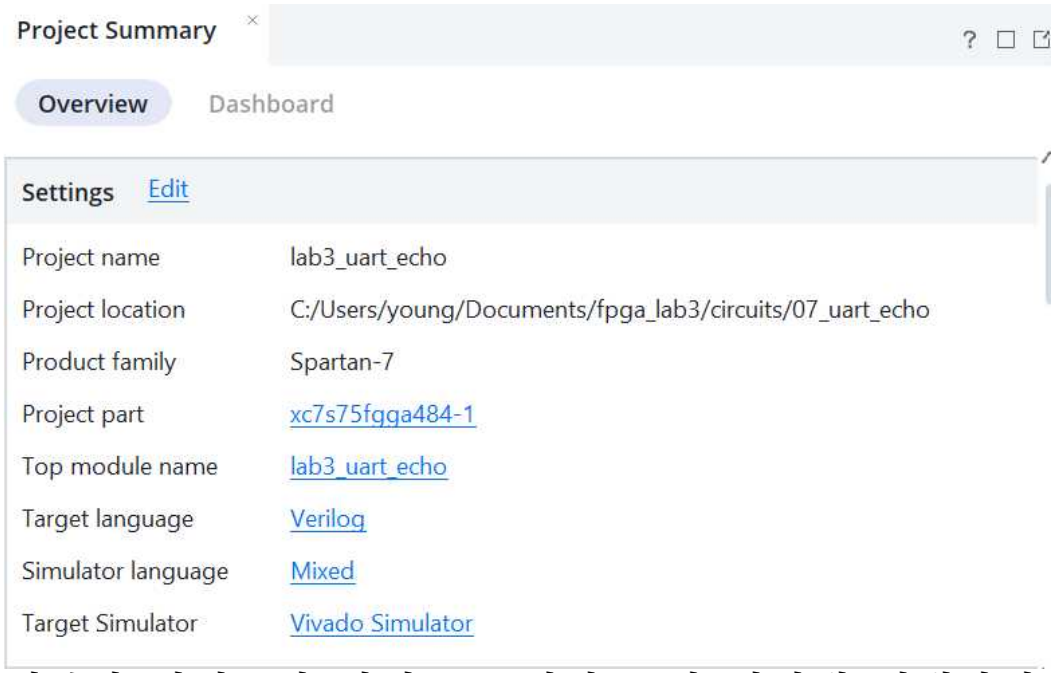
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS)	14.220 ns	Worst Hold Slack (WHS)	0.167 ns	Worst Pulse Width Slack (WPWS)	9.500 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns	Total Pulse Width Negative Slack (TPWS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	173	Total Number of Endpoints	173	Total Number of Endpoints	94

All user specified timing constraints are met.

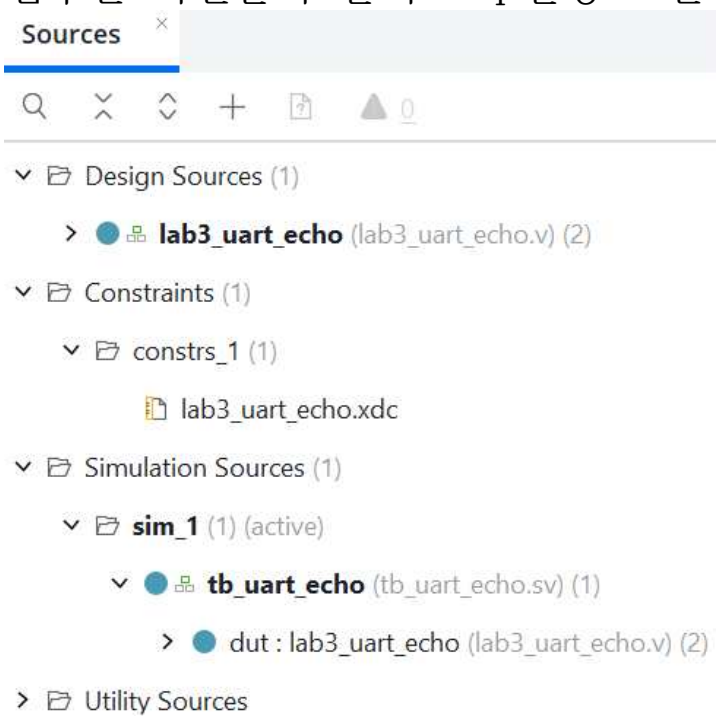
50MHz, 20ns 클록 제약을 적용하여 문자 LCD 제어 회로의 구현 후 타이밍을 검증하였다. Setup 분석 결과 WNS는 +14.220ns, TNS는 0.000ns였으며 173개 Endpoint 중 실패한 Endpoint는 없었다. Hold 분석 결과 WHS는 +0.167ns, THS는 0.000ns였으며 Hold 위반도 발생하지 않았다. Pulse Width 분석에서는 WPWS가 +9.500ns, TPWS가 0.000ns로 확인되었다. 모든 사용자 지정 타이밍 제약이 충족되었으므로 본 회로는 50MHz 클록 조건에서 정상적으로 동작할 수 있다.

8.7. UART ECHO

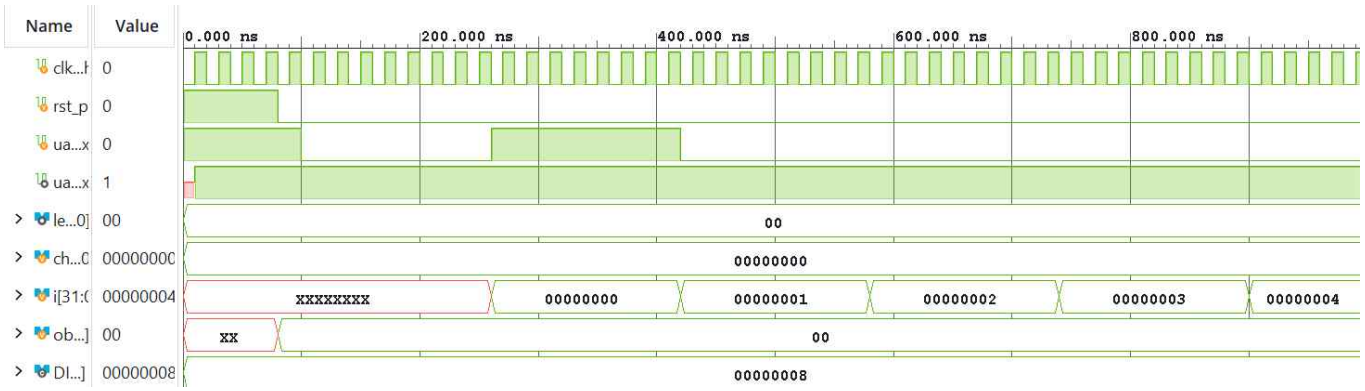
- Vivado 내의 settings는 다음과 같다.



첨부한 사진들과 같이 Top설정 또한 알맞게 진행하였다.



이를 Run simulation 하여 얻은 파형이다.



이렇게 얻은 파형이 올바른지 확인하기 위해 위에서
작성한 vscode 내에서의 시뮬레이션 결과와 비교하였다.

이론에 부합한 결과는 위에서 설명하였고, 그와 vscode의
파형이 일치하였기에 vscode에서의 파형이 올바른
파형이라고 할 수 있다.
이를 vivado파형과 비교해보면, 정확히 같은 파형의
결과를 얻은 것을 알 수 있다.

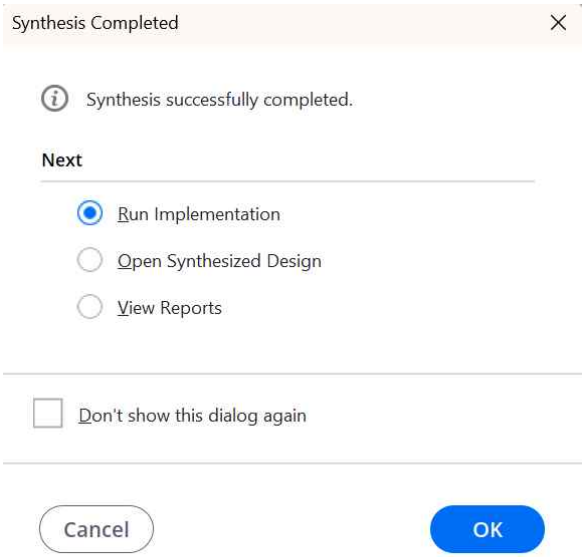
즉, 문제 없이 알맞은 파형을 얻었다.

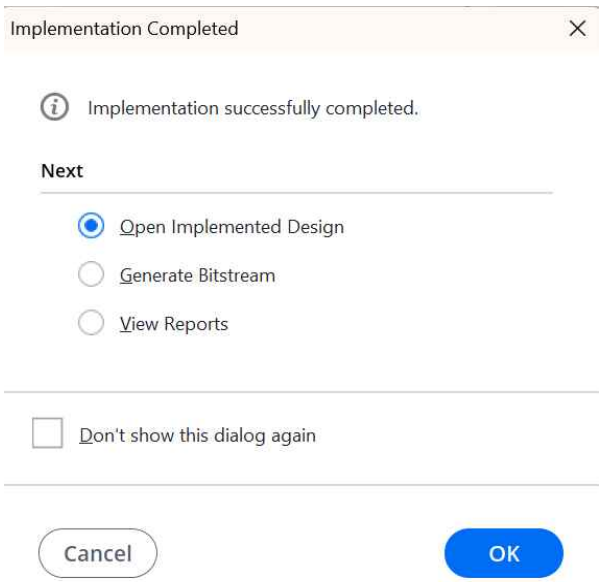
위의 파형을 얻기 위해 시뮬레이션을 행한 결과 아래와 같은 Tcl Console의 결과값을 얻었다.

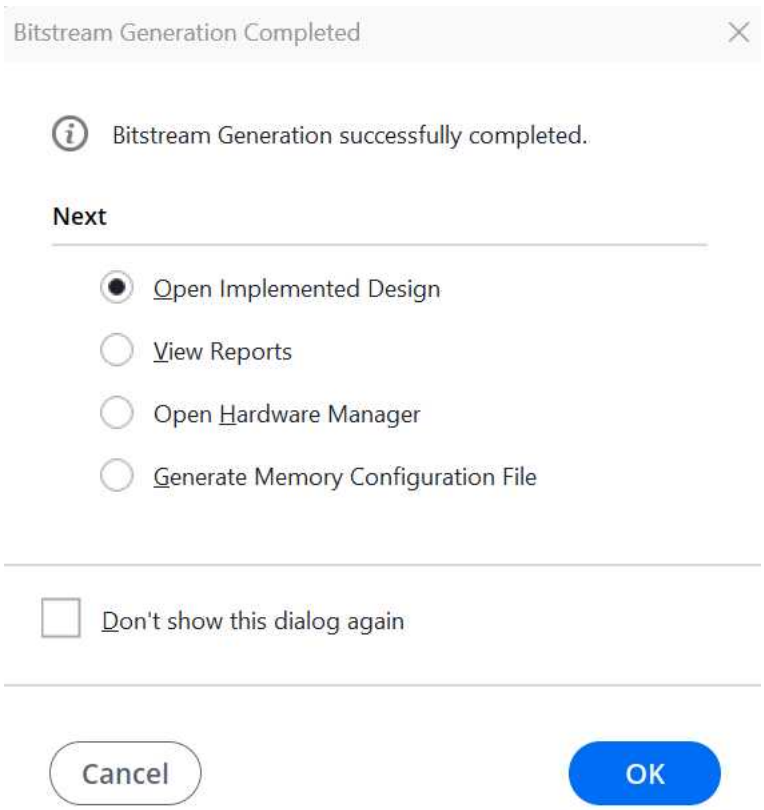
```
source tb_uart_echo.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wav
#   }
# }
# run 1000ns
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_uart_echo_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:06 . Memory (MB): peak = 4015.895 ; gain = 0.000
```

그 결과, 정상적으로 출력을 오류 없이 얻었다.

이후 Run synthesis와 Run Implementation을 진행하였다.




Implementation Completed
Implementation successfully completed.
Next
Open Implemented Design
Generate Bitstream
View Reports
Don't show this dialog again
Cancel
OK


Bitstream Generation Completed
Bitstream Generation successfully completed.
Next
Open Implemented Design
View Reports
Open Hardware Manager
Generate Memory Configuration File
Don't show this dialog again
Cancel
OK

최종적으로 아래와 같은 결과를 오류 없이 얻었다.

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												136	118
✓ impl_1	constrs_1	write_bitstream Complete!	14.61	0.000	0.131	0.000		0.000	0.091	0	9 Warn			134	118

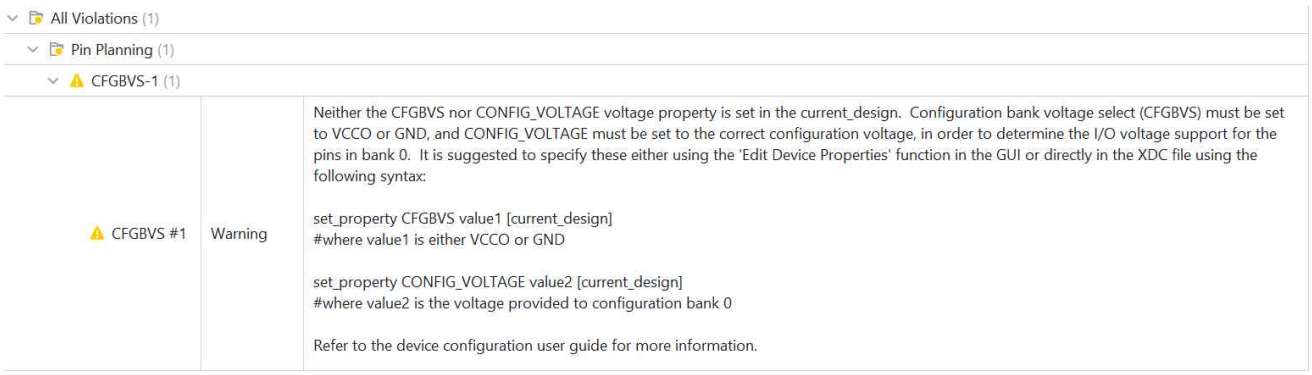
사용한 커밋은 이후 차례 <증빙 및 소스 commit 링크>에 주소로 기록하였다.

- Bit 해시:

SHA-256:
CA204B4AAC6ED1820C89A56735007F45E8497FD14B789453E16B5DDF1E434477

- Bit 경로

C:\Users\young\Documents\fpga_lab3\circuits\07_uart_echo\lab3_uart_echo.runs\impl_1\lab3_uart_echo.bit



All Violations (1)		
Pin Planning (1)		
CFGBVS-1 (1)		
CFGBVS #1	Warning	Neither the CFGBVS nor CONFIG_VOLTAGE voltage property is set in the current_design. Configuration bank voltage select (CFGBVS) must be set to VCCO or GND, and CONFIG_VOLTAGE must be set to the correct configuration voltage, in order to determine the I/O voltage support for the pins in bank 0. It is suggested to specify these either using the 'Edit Device Properties' function in the GUI or directly in the XDC file using the following syntax: set_property CFGBVS value1 [current_design] #where value1 is either VCCO or GND set_property CONFIG_VOLTAGE value2 [current_design] #where value2 is the voltage provided to configuration bank 0 Refer to the device configuration user guide for more information.

- DRC 검사 결과 Error는 0건이었으며, CFGBVS-1 Warning 1건이 발생하였다.
  해당 경고는 CFGBVS 및 CONFIG_VOLTAGE 속성이 설정되지 않아 발생한 것으로, 실제 보드 구성 전압과 대조가 필요한 항목이다.

- Timing 결과 검증



50MHz, 20ns 클록 제약을 적용하여 UART 에코 회로의 구현 후 타이밍을 검증하였다. Setup 분석 결과 WNS는 +14.617ns, TNS는 0.000ns였으며 239개 Endpoint 중 실패한 Endpoint는 없었다. Hold 분석 결과 WHS는 +0.131ns, THS는 0.000ns였으며 Hold 위반도 발생하지 않았다. Pulse Width 분석에서는 WPWS가 +9.500ns, TPWS가 0.000ns로 확인되었다. 모든 사용자 지정 타이밍 제약이 충족되었으므로 본 회로는 50MHz 클록 조건에서 정상적으로 동작할 수 있다.

9. 실험 결과와 예상 결과 간의 비교

9.1. LED PWM

- 2번 버튼을 반복해서 누르면 LED1부터 LED8까지 8개의 LED 밝기가 동시에 변했다.
- 버튼을 누를수록 LED가 단계적으로 밝아지는 모습을 확인할 수 있었다.
- 최대 밝기 이후 다시 LED가 꺼지는 상태로 돌아가 PWM 밝기 단계가 순환함을 확인했다.
- 모든 LED가 같은 밝기로 동작하므로 하나의 PWM 신호가 8개 LED에 공통으로 적용된 것을 확인할 수 있었다.
- 결과적으로 LED PWM 밝기 조절과 단계 순환 동작이 정상적으로 구현되었다.

보드 시연 영상:

fpga_lab3/evidence/01_led_pwm/board/videos/01_led_pwm_demo.mp4 at main · dalkomys777/fpga_lab3

9.2. RGB PWM

- 2·3·4번 버튼을 조작하면서 Full Color LED의 각 색상 성분이 변하는 모습을 확인했다.
- 빨간색과 초록색이 각각 출력되는 장면이 분명하게 나타났다.
- 여러 색상 성분이 함께 켜지면서 흰색, 자홍색 계열의 혼합색이 출력되는 것도 확인했다.
- 동일한 색상에서도 버튼 조작에 따라 밝기가 달라졌으며, 최대 단계 이후 해당 색상 성분이 꺼지는 순환 동작이 나타났다.
- 결과적으로 R·G·B 성분을 독립적으로 조절하고 이들을 조합하여 다양한 색을 출력하는 RGB PWM 기능이 정상적으로 동작했다.

보드 시연 영상:

fpga_lab3/evidence/02_rgb_pwm/board/videos/02_rgb_pwm_demo.mp4 at main · dalkomys777/fpga_lab3

9.3. PIEZO

- 피에조 활성화 스위치가 ON인 상태에서 피에조 소리가 출력되었다.
- 영상의 오디오를 분석한 결과, 주로 약 294.5 Hz의 음이 측정되어 설계한 약 294 Hz의 음과 일치했다.
- 영상 약 1.0~2.5초 및 3.75초 이후 구간에서 음이 발생했으며, 중간의 버튼 조작 구간에서는 음이 잠시 정지했다가 다시 출력되었다.
- 따라서 리셋 상태에서는 피에조 출력이 정지하고, 리셋 해제 후 다시 음이 발생하는 동작을 확인할 수 있었다.
- 결과적으로 목표 주파수의 피에조 출력과 리셋에 의한 출력 정지가 정상적으로 구현되었다.

보드 시연 영상:

fpga_lab3/evidence/03_piezo/board/videos/03_piezo_demo.mp4 at main · dalkomys777/fpga_lab3

9.4. Stepper

- 2번 버튼을 누르는 동안 스테핑 모터의 색상 원판 위치가 변해 실제 회전이 발생하는 것을 확인했다.
- 모터 아래의 구동 상태 LED가 켜지며 모터 코일에 구동 신호가 출력되는 모습도 확인했다.
- 버튼을 누르지 않는 구간에서는 원판이 정지하여 Enable 신호에 따른 구동 및 정지 동작을 확인할 수 있었다.
- 결과적으로 모터의 회전과 정지 기능을 영상을 통해 확인할 수 있었고, 정상적으로 코드가 기능하였음을 판단할 수 있다.

보드 시연 영상:

fpga_lab3/evidence/04_stepper/board/videos/04_stepper_demo.mp4 at main · dalkomys777/fpga_lab3

9.5. MMSS Clock

- 7-Segment 표시가 약 00:02 → 00:08 → 00:18 순서로 증가하여 1초 단위 계수가 이루어지는 것을 확인했다.
- 영상 후반에는 00:59가 표시되었으며, 이후 01:01까지 증가했다.
- 따라서 초 값이 59를 지난 후 00으로 돌아가고 분 값이 1 증가하는 자리 올림 동작을 확인할 수 있었다.
- 표시되는 숫자가 지속적으로 갱신되어 4자리 7-Segment의 동적 구동도 정상적으로 이루어졌다.
- 결과적으로 초 단위 계수와 00:59 → 01:00의 분 자리 올림이 정상적으로 구현되었음을 확인할 수 있다.

보드 시연 영상:

fpga_lab3/evidence/05_mmss_clock/board/videos/05_mmss_clock_demo.mp4 at main · dalkomys777/fpga_lab3

9.6. Character LCD

- LCD 첫째 줄에 FPGA LAB3, 둘째 줄에 LCD CONTROLLER가 정확하게 표시되었다.
- 두 줄의 문자열이 지정된 위치에서 안정적으로 유지되었다.
- 1번 버튼을 누르고 있는 장면에서도 문자열이 계속 표시되었으며, 장시간 빈 화면으로 유지되는 현상은 나타나지 않았다.
- 버튼 조작 이후에도 같은 문자열이 정상적으로 유지되어 LCD 초기화 및 문자 전송 과정이 정상적으로 완료된 것을 확인했다.
- 최종적으로 LCD 초기화와 두 줄 문자열 출력이 정상적으로 구현되었다.

보드 시연 영상:

fpga_lab3/evidence/06_character_lcd/board/videos/06_character_lcd_demo.mp4 at main · dalkomys777/fpga_lab3

9.7. UART Echo

- 노트북의 터미널과 FPGA 보드가 UART 케이블로 연결된 상태를 확인했다.
- 키보드 입력에 따라 터미널에 A, AZ, AZ0가 순서대로 표시되었다.
- 입력한 문자가 터미널로 되돌아왔으므로 FPGA가 UART 데이터를 수신한 후 동일한 데이터를 다시 송신하는 Echo 기능을 확인할 수 있었다.
- 문자가 입력될 때마다 보드의 LED 패턴도 변경되어 마지막으로 수신한 문자의 데이터가 LED에 출력되는 것을 확인했다.
- 분석해보면, UART 문자 수신·재전송 및 수신 데이터의 LED 출력 기능이 정상적으로 동작했음을 알 수 있다.

보드 시연 영상:

fpga_lab3/evidence/07_uart_echo/board/videos/07_uart_echo_demo.mp4 at main · dalkomys777/fpga_lab3

10. 증빙 및 source commit 링크(Github)

vivado버전: vivado 2026.1

FPFA Part: xc7s75fgga484-1

10.1. LED PWM

VS Code simulation log:

fpga_lab3/evidence/01_led_pwm/vscode/simulation.txt at main · dalkomys777/fpga_lab3

VS Code waveform VCD:

fpga_lab3/evidence/01_led_pwm/vscode/wave.vcd at main · dalkomys777/fpga_lab3

VS Code waveform image:

fpga_lab3/evidence/01_led_pwm/vscode/wave.png at main · dalkomys777/fpga_lab3

VS Code waveform xdc:

fpga_lab3/circuits/01_led_pwm/constraints/lab3_led_pwm.xdc at main · dalkomys777/fpga_lab3

10.2. RGB LED PWM

VS Code simulation log:

fpga_lab3/evidence/02_rgb_pwm/vscode/simulation.txt at main · dalkomys777/fpga_lab3

VS Code waveform VCD:

fpga_lab3/evidence/02_rgb_pwm/vscode/wave.vcd at main · dalkomys777/fpga_lab3

VS Code waveform image:

fpga_lab3/evidence/02_rgb_pwm/vscode/wave.png at main · dalkomys777/fpga_lab3

VS Code waveform xdc:

fpga_lab3/circuits/02_rgb_pwm/constraints/lab3_rgb_pwm.xdc at main · dalkomys777/fpga_lab3

10.3. Piezo

VS Code simulation log:

fpga_lab3/evidence/03_piezo/vscode/simulation.txt at main · dalkomys777/fpga_lab3

VS Code waveform VCD:

fpga_lab3/evidence/03_piezo/vscode/wave.vcd at main · dalkomys777/fpga_lab3

VS Code waveform image:

fpga_lab3/evidence/03_piezo/vscode/wave.png at main · dalkomys777/fpga_lab3

VS Code waveform xdc:

fpga_lab3/circuits/03_piezo/constraints/lab3_piezo.xdc at main · dalkomys777/fpga_lab3

10.4. Stepper

VS Code simulation log:

fpga_lab3/evidence/04_stepper/vscode/simulation.txt at main · dalkomys777/fpga_lab3

VS Code waveform VCD:

fpga_lab3/evidence/04_stepper/vscode/wave.vcd at main · dalkomys777/fpga_lab3

VS Code waveform image:

fpga_lab3/evidence/04_stepper/vscode/wave.png at main · dalkomys777/fpga_lab3

VS Code waveform xdc:

fpga_lab3/circuits/04_stepper/constraints/lab3_stepper.xdc at main · dalkomys777/fpga_lab3

10.5. MMSS Clock

VS Code simulation log:

fpga_lab3/evidence/05_mmss_clock/vscode/simulation.txt at main · dalkomys777/fpga_lab3

VS Code waveform VCD:

fpga_lab3/evidence/05_mmss_clock/vscode/wave.vcd at main · dalkomys777/fpga_lab3

VS Code waveform image:

fpga_lab3/evidence/05_mmss_clock/vscode/wave.png at main · dalkomys777/fpga_lab3

VS Code waveform xdc:

fpga_lab3/circuits/05_mmss_clock/constraints/lab3_mmss_clock.xdc at main · dalkomys777/fpga_lab3

10.6. Character LCD

VS Code simulation log:

fpga_lab3/evidence/06_character_lcd/vscode/simulation.txt at main · dalkomys777/fpga_lab3

VS Code waveform VCD:

fpga_lab3/evidence/06_character_lcd/vscode/wave.vcd at main · dalkomys777/fpga_lab3

VS Code waveform image:

fpga_lab3/evidence/06_character_lcd/vscode/wave.png at main · dalkomys777/fpga_lab3

VS Code waveform xdc:

fpga_lab3/circuits/06_character_lcd/constraints/lab3_character_lcd.xdc at main · dalkomys777/fpga_lab3

10.7. UART echo

VS Code simulation log:

fpga_lab3/evidence/07_uart_echo/vscode/simulation.txt at main · dalkomys777/fpga_lab3

VS Code waveform VCD:

fpga_lab3/evidence/07_uart_echo/vscode/wave.vcd at main · dalkomys777/fpga_lab3

VS Code waveform image:

fpga_lab3/evidence/07_uart_echo/vscode/wave.png at main · dalkomys777/fpga_lab3

VS Code waveform xdc:

fpga_lab3/circuits/07_uart_echo/constraints/lab3_uart_echo.xdc at main · dalkomys777/fpga_lab3

전체 소스 커밋:

Complete LAB3 circuits and simulation evidence · dalkomys777/fpga_lab3@16186c9

LAB_03 github 주소:

dalkomys777/fpga_lab3: FPGA LAB3 circuits, testbenches, constraints, and simulation evidence

LAB_03 제출 태그:

fpga_lab3-reports-reports-post